

III

Halo 2 Guide

The Halo 2 proof system, from arithmetisation to recursion

Marek
m@rek.onl

Abstract

This volume of *The Zcash Arboretum* develops the proof system that Orchard uses: Halo 2, the argument by which the author of a shielded Action convinces every validator that the Action is well formed while disclosing nothing about the note, the key, or the Merkle path behind it. It assumes the *Math Guide*, whose finite fields, polynomials and evaluation domains, elliptic curves, and probability are used throughout without re-derivation, and the *Crypto Guide*, whose definitions it instantiates rather than restates: arguments of knowledge and zero knowledge, the Fiat–Shamir transform, Pedersen vector commitments, and the polynomial-commitment interface with its compilation recipe.

The running example is knowledge of x with $x^3 + x + 5 = 35$ over the field $\mathbb{F}_{97}$, a size at which every number of the example can be checked by hand; every number is computed by script. The volume reduces the example to a grid of gates, the grid to polynomials, the polynomials to a single identity, and the identity to one evaluation at a random point, at which the prover must open a commitment without sending the committed polynomial. The inner-product argument (IPA) that performs the opening has a section of its own: the problem stated without machinery, one fold written out step by step, the recursion from the prover’s and the verifier’s side, the verifier’s structured scalars, the extraction from three transcripts that proves the argument knowledge sound under the discrete-logarithm assumption on Vesta, with the hash that derives the generators modelled as a random oracle, the blinders and what they do not hide, and the running example opened on a curve of prime order 97. The deployed proof system follows: the Action circuit’s arithmetisation, the binding of the public statement, the Fiat–Shamir transcript, the reduction of many openings to one, the complete protocol, and its zero knowledge. The volume then analyses the knowledge soundness of the Fiat–Shamir-compiled argument, lists the verifier’s checks on the running example in the order he performs them, and closes with recursive composition and accumulation, which Orchard does not deploy.

Contents

1	Introduction	5
1.1	Scope and layering	5
1.2	The running example and the security notions	5
1.3	Outline	8
2	Arithmetisation: from a computation to committed polynomials	8
2.1	From computation to a grid of constraints	9
2.2	From the grid to polynomials	11
2.3	From vanishing to a single identity	12
2.4	Why checking one random point is a proof	13
2.5	PLONKish arithmetisation	15
2.6	Enforcing the wiring: the permutation argument	17
2.7	Proving a value sits in a table: lookups	22
2.8	The lookup argument	23
2.9	Why the prover cannot wriggle: binding commitments	25
3	The inner-product polynomial commitment	27
3.1	The problem: an inner product in fewer than d group elements	27
3.2	One fold, every step written out	29
3.3	The recursion: k rounds to a single scalar	32
3.4	The verifier’s work: structured scalars and the final generator	34
3.5	Why it is binding: extraction from three transcripts	36
3.6	Zero knowledge: what the blinders hide and what they do not	40
3.7	A complete toy opening, carried by script	43
3.8	Halo 2’s specifics: parameters, generators from a hash, the deferred multi-scalar multiplication, and the proof bytes	46
3.8.1	The “Halo” in Halo 2: deferring the expensive check	54
4	The Halo 2 proof system	56
4.1	From statement to circuit: arithmetisation in practice	57
4.1.1	Custom gates	57
4.1.2	The incomplete-addition gate	57
4.1.3	Lookup arguments of the Action circuit	59
4.1.4	Column polynomials and the quotient	60
4.2	Instance columns and binding the public statement	61
4.3	Removing the verifier: the Fiat–Shamir transform	63
4.4	The multipoint opening argument	65
4.5	The complete Halo 2 protocol	68
4.6	Zero knowledge: hiding the witness in Halo 2	73
4.7	Halo 2 as a compiled polynomial IOP	77

5	A rigorous treatment of knowledge soundness	80
5.1	From the interactive extractor to the non-interactive argument	80
5.2	The algebraic group model	84
5.3	Concrete versus asymptotic security	87
6	The verifier's decision and its implication chain	88
6.1	The verifier's checks, read in order	88
6.2	The Action circuit at deployed scale	91
7	Recursive proof composition and accumulation schemes	95
7.1	Recursion and its two classical obstacles	95
7.2	Accumulation schemes	96
7.3	Accumulation and the Halo trick	98
7.4	A cycle of curves for efficient native recursion	103

1 Introduction

1.1 Scope and layering

This is the third volume of *The Zcash Arboretum*, and it develops the cryptography behind Zcash’s Orchard protocol: the proof system Halo 2, with which every Orchard Action carries a zero-knowledge proof of its own well-formedness. The protocol specification fixes that use: Halo 2 is used with the Vesta curve (the *Math Guide*, §“Pallas and Vesta assembled”) to prove and verify the Orchard Action statement (protocol specification § 4.1.13, “Zero-Knowledge Proving System”), and the proving system is defined by reference to the halo2 book (§ 5.4.10.3, “Halo 2”). The statement itself (§ 4.18.4, “Action Statement (Orchard)”) is read in §1.2. Actions of the Orchard pool and of the Ironwood pool, the second shielded pool of Orchard-protocol notes, prove this statement, extended at NU6.3 by one public bit, disableCrossAddress, that when set forces the new note’s address to equal the spent note’s, with one circuit and one verifying key (protocol specification, § 4.6, “Action Descriptions”; ZIP 229; ZIP 258), so the proof system and proof format described here serve both pools.

The two volumes below this one built its material from the ground up. The *Math Guide* constructed the mathematics: finite fields, elliptic curves, polynomials and evaluation domains, probability. The *Crypto Guide* constructed the cryptographic primitives: hash functions, commitments, Σ -protocols and the Fiat–Shamir transform, zero knowledge, polynomial commitment schemes. This volume assumes all of these and defines none of them afresh; it assembles them into a proof system. In particular the *Crypto Guide* owns every definition this volume instantiates: the non-interactive argument and the SNARK, together with the polynomial interactive oracle proof and the compilation recipe that turns one into the other (§“SNARKs: succinct non-interactive arguments of knowledge”); the Fiat–Shamir transform (§“The Fiat–Shamir transform: from interactive to non-interactive”); the Pedersen vector commitment (§“Pedersen vector commitments”); and the polynomial-commitment interface, with its sketch of an inner-product opening (§“Polynomial commitment schemes”). Where a lower volume’s result is needed it is cited at the point of use, by italic volume name and section title: the volumes are separate documents, so there is no cross-document reference, and this volume forward-references only its own later sections.

1.2 The running example and the security notions

The running example is the statement

$$\text{knowledge of } x \in \mathbb{F}_{97} \text{ with } x^3 + x + 5 = 35,$$

one of whose witnesses (the *Crypto Guide*, §“Languages, relations, and witnesses”) is $x = 3$, since $27 + 3 + 5 = 35$; the others in $\mathbb{F}_{97}$ are 18 and 76, and the prover of the example holds $x = 3$. It is arithmetised in §2, committed and opened in §3, bound to its public input 35 and compiled into a proof string in §4, and verified in §6. Its field arithmetic, that of the claim, the grid, the polynomials and every scalar, is in the prime field $\mathbb{F}_{97}$ (the *Math Guide*, §“The prime field and two inversion routes”), a size at which every product can be checked by hand; the commitments of §3 are points of a curve of order 97 whose coordinates lie in a second prime field, $\mathbb{F}_{79}$. Every number printed is computed by script, and the script’s output is the source of the text.

A proof system for such a statement is required to have three properties, each a definition of the *Crypto Guide* (§“Interactive proofs, zero knowledge, and SNARKs”) used here without re-statement. *Overwhelming completeness*: an honest prover holding a witness is accepted with overwhelming probability, in the sense of the *Math Guide* (§“Polynomial, exponential, and negligible functions”). *Knowledge soundness* (the *Crypto Guide*, §“Knowledge soundness and extractors”): an efficient extractor, given rewindable access to any prover the verifier accepts with non-negligible probability, outputs a valid witness. *Zero knowledge* (the *Crypto Guide*, §“The simulation paradigm and zero knowledge”): a simulator given only the public input produces transcripts indistinguishable from those of real executions.

The volume establishes these properties to different extents. Design-stage claims are classified in three grades: *specified*, when the protocol specification or a ZIP states the property; *designed-but-unspecified*, when the design intends it but no specification, ZIP or theorem of this volume states it for the deployed system; and *open problem*, when no design is known to achieve it. The Halo 2 protocol is complete except with negligible probability: for each random challenge, a number of values at most proportional to the size of the circuit, out of the $|\mathbb{F}|$ possible, makes undefined a division that the verifier or the honest prover performs; §4.6 lists these degenerate challenges once the protocol is defined. The inner-product opening argument is knowledge sound under the discrete-logarithm assumption on Vesta, with the hash that derives its generators modelled as a random oracle (Theorem 3.5 and, for the deployed opening, Corollary 3.13). Knowledge soundness of the deployed non-interactive argument rests on an analysis in the algebraic group model and the random-oracle model whose instantiation to the deployed transcript is designed-but-unspecified (§5). Zero knowledge is recorded in §4.6 from the halo2 book’s claim for the interactive protocol, under the composition and Fiat–Shamir hypotheses stated there.

The running example has the form of the Orchard Action statement. The prover of an Action shows knowledge of a note, the keys that control it, and a Merkle path such that the note’s commitment is well formed, the path, unless the spent value is zero, leads from that commitment to the published tree root (the *Crypto Guide*, §“Merkle trees and commitments to sets”), the net value commitment is a Pedersen commitment to the old value minus the new value (the *Crypto Guide*, §“The Pedersen commitment”), the nullifier, the unique tag whose publication marks the note spent, is computed correctly, and the key that authorises the spend is a re-randomisation of the owner’s key (the *Crypto Guide*, §“Key re-randomisation and unlinkability”). The protocol specification states this as nine conditions on a public input and a private witness (§ 4.18.4, “Action Statement (Orchard)”), the five just paraphrased and four further: that the spent note’s address is derived from the same keys, that the new note’s commitment is well formed, and that two public bits, when zero, force the spent value and the created value, respectively, to zero. From NU6.3 the circuit checks a tenth condition on a further public bit, which restricts cross-address transfers and is set for every Orchard-pool Action (§ 4.6; ZIP 258). Each condition unfolds into small arithmetic facts over the field $\mathbb{F}_{p_{\text{Pallas}}}$ (the *Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”): the note commitment alone is a Sinsemilla commitment, a Sinsemilla hash to a point plus a blinding multiple of a fixed generator (the *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”), whose hash is evaluated in the circuit with the help of a lookup table. Together the conditions are laid out on the $2^{11} = 2048$ rows of the deployed Action circuit; the lookup table and the row count are given in §4, “From statement to circuit: arithmetisation in practice”. The

running example itself is a conjunction of *four* arithmetic facts, $x \cdot x = x^2$, $x^2 \cdot x = x^3$, $x^3 + x = s$ and $s + 5 = 35$, one per row of the grid that §2 builds. Both the toy and the Action circuit are instances of the constraint system that §2 defines; the toy’s constraint on each row reads that row alone and consults no table.

The chain of reductions. Five reductions connect the claim to the checks the verifier performs.

1. **Computation $\rightarrow$ grid** (§2, “From computation to a grid of constraints”). The claim is rewritten as a table of rows, each row one elementary gate ($\times$ or $+$), with copy constraints between rows forcing shared values.
2. **Grid $\rightarrow$ polynomials** (§2, “From the grid to polynomials”). Each column of the table is interpolated into a polynomial (the *Math Guide*, §“Lagrange interpolation”). The condition that every row obeys its gate becomes the condition that one polynomial $G(X)$ is zero at every row.
3. **Vanishing $\rightarrow$ identity** (§2, “From vanishing to a single identity”). The statement that G is zero at every row becomes the algebraic identity $G(X) = t(X) Z_H(X)$ for some quotient t , where Z_H is the vanishing polynomial of the set H of row names (the *Math Guide*, §“The evaluation domain H and its vanishing polynomial”). A trace that satisfies every gate yields such an identity, and a trace that violates a gate cannot. The copy constraints are not part of G ; the permutation argument enforces them (item 5).
4. **Identity $\rightarrow$ one random check** (§2, “Why checking one random point is a proof”). The verifier tests the identity at a single random point z . A false identity holds at a uniformly random point with probability at most its degree divided by the field size (the *Math Guide*, §“The Schwartz–Zippel lemma”).
5. **Random check $\rightarrow$ committed openings.** A binding commitment (the *Crypto Guide*, §“Hiding and binding”) prevents the prover from changing her polynomials after z is drawn (§2, “Why the prover cannot wriggle: binding commitments”), and the inner-product argument (IPA) of §3 opens a commitment at z without sending the polynomial; the permutation argument enforces the copy constraints (§2, “Enforcing the wiring: the permutation argument”); a dedicated public column, the instance column, fixes the public input 35 (§4, “Instance columns and binding the public statement”); and blinding hides the witness (§4, “Zero knowledge: hiding the witness in Halo 2”).

Section 6 composes the five reductions into one chain of implications, from the verifier’s accepted checks back to the claim.

The scale caveat, stated once. The field $\mathbb{F}_{97}$ is a toy. Over Orchard’s field $\mathbb{F}_{p_{\text{Pallas}}}$, whose prime has 255 bits, a *fixed* false identity of degree around 2^{14} (the degree of the deployed constraint polynomial, bounded in §4, “The complete Halo 2 protocol”) survives one fresh uniform evaluation challenge with probability around $2^{14}/2^{254} = 2^{-240}$, in place of the visible $3/97$ that §2 exhibits for the toy’s cheat. Every such number in this volume is one Schwartz–Zippel term: the probability that one random point hides one fixed lie. It is not the security level of the complete proof

system, which also rests on the binding of the commitments, the extractor of the opening argument, and the loss of the Fiat–Shamir transform, treated in §3, §4 and §5. The caveat is stated here and not re-derived; every probability printed later is read with it.

1.3 Outline

The sections follow the running example; Figure 1 shows the section in which each stage is built.

Section 2 takes the example from the computation to committed polynomials: the grid, the column polynomials, the single identity, the random point, and then the general form of the constraint system, PLONKish arithmetisation, of which the toy is an instance, followed by the permutation argument for the copy constraints and the lookup argument for facts that no low-degree gate states economically. It ends with the problem a vector commitment alone does not solve: proving the value of a committed polynomial at z without sending the polynomial, in fewer group elements than the polynomial has coefficients.

Section 3 solves that problem with the inner-product argument. It states the inner-product problem without machinery, works one fold with every algebra step written out, gives the recursion with both parties’ computations side by side, makes the verifier’s work explicit, proves the argument knowledge sound by extraction from three transcripts under the discrete-logarithm assumption with uniform generators, separates what the blinders hide from what they do not, and opens the example’s own polynomial through every round on a curve of prime order 97, by script. Its last subsection gives Halo 2’s parameters, the deployed form of the argument as a change of variables, the bytes of a proof, the deferred multi-scalar multiplication and the batching that amortises it, and accumulation, which Orchard does not deploy.

Section 4 gives the deployed proof system: the Action circuit’s arithmetisation, the instance column that binds the public statement, the Fiat–Shamir transform that replaces the verifier’s challenges by hash outputs, the reduction of many openings to one, the complete protocol in seven phases, its zero knowledge, and the compiled-polynomial-IOP pattern the protocol instantiates.

Section 5 treats knowledge soundness of the non-interactive argument: the extractor of §3 when the challenges are hash outputs, the algebraic group model in which the halo2 book proves its tight bound for the analysed protocol, and the gap between concrete and asymptotic security.

Section 6 lists the verifier’s checks on the running example in the order he performs them, the chain of implications back to the prover’s knowledge of advice satisfying every gate and every copy constraint, and the same chain at Orchard scale.

Section 7 treats recursive proof composition, accumulation schemes, the Halo trick, and the cycle of curves that makes native recursion efficient. Orchard proves with Halo 2 on the Pasta curves but does not recurse, and the section states at the outset that Orchard does not deploy its material.

2 Arithmetisation: from a computation to committed polynomials

This section carries the running example of §1 — the prover knows $x = 3$ with $x^3 + x + 5 = 35$ over $\mathbb{F}_{97}$ — from a computation to a handful of polynomials that a verifier can query at one point,

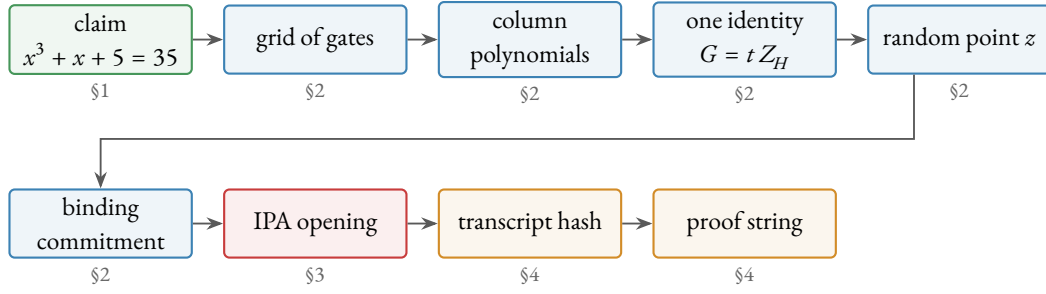


Figure 1. The running example’s path through the volume, each stage tagged with the section that owns it. The claim is fixed in this section (green). The grid of gates, the column polynomials, the single identity $G = t Z_H$, the random point z and the binding commitment are built in §2 (blue). The commitment is opened by the inner-product argument in §3 (red). The transcript hash, whose outputs replace the verifier’s challenges, and the proof string are produced in §4 (amber). Section 6 reads the chain backwards, from the accepted proof string to the claim.

toy first and general second. It ends with the property the argument has not yet provided: that the queried polynomials are fixed before the evaluation point is chosen.

2.1 From computation to a grid of constraints

The verifier cannot evaluate the computation without the witness. Arithmetisation dismantles the computation into *local* facts, each involving only a handful of values, arranged so that “every local fact holds and the values are consistently shared” is equivalent to “the whole computation ran correctly”. The dialect of arithmetisation Halo 2 uses is called *PLONKish*; §2.5 defines it once the toy has shown what it generalises.

Execution trace. Evaluate $x^3 + x + 5$ the way a machine would, naming every intermediate value:

$$x^2 = x \cdot x, \quad x^3 = x^2 \cdot x, \quad s = x^3 + x, \quad s + 5 = 35.$$

Four elementary operations, two multiplications and two additions, and each becomes one row of a table. A row has three *wires* a, b, c (two inputs and an output) and a bank of five *selectors* q_M, q_L, q_R, q_O, q_C that switch on whichever gate the row performs. The universal gate equation, imposed at every row, is

$$q_M a b + q_L a + q_R b + q_O c + q_C + \text{PI} = 0, \quad (1)$$

where PI carries the negated public input on that row (here -35 , for the target 35). A multiplication gate $a \cdot b = c$ is the choice $q_M = 1, q_O = -1$ and the rest zero; an addition gate $a + b = c$ is $q_L = q_R = 1, q_O = -1$. Filling in the four operations gives the *execution trace* of Table 1. Definition 2.7 generalises this fixed selector bank to arbitrary custom gates and lookup arguments (the halo2 book’s *PLONKish Arithmetization* chapter); §4 gives the form of the Action circuit’s gates, and one in full, under “From statement to circuit: arithmetisation in practice”.

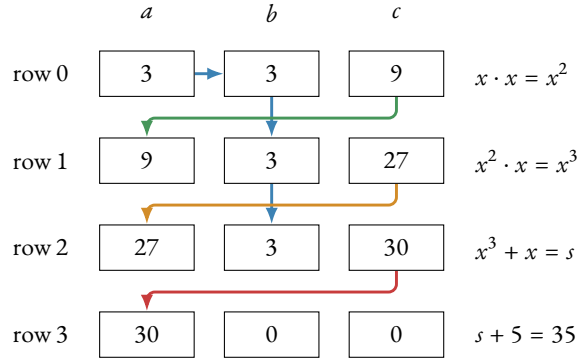
Table 1. The execution trace for $x = 3$, all entries in $\mathbb{F}_7$ (so $-1 = 96$ and $-35 = 62$). Every row satisfies the gate equation (1): row 0 reads $1 \cdot 3 \cdot 3 + (-1) \cdot 9 = 0$ and row 3 reads $1 \cdot 30 + 5 + (-35) = 0$. The selector columns q_i are fixed circuit data and PI is the negated public input, both known to the verifier; the wires a, b, c are the prover’s *advice*, her secret trace.

row	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	gate performed
0	3	3	9	1	0	0	-1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	-1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	-1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	-35	$s + 5 = 35$

Copy constraints. The table as printed does not yet encode the computation. Nothing forces the x in row 0 to equal the x in rows 1 and 2, nor the output $x^2 = 9$ of row 0 to be the input of row 1. A cheating prover could put $x = 3$ in row 0 and $x = 8$ in row 2 and satisfy each gate *in isolation* without the rows forming one computation. The missing assertions are that certain cells are *equal*, the *copy constraints* (also called *equality constraints*):

$$\underbrace{a_0 = b_0 = b_1 = b_2}_{\text{all are } x=3}, \quad \underbrace{c_0 = a_1}_{x^2=9}, \quad \underbrace{c_1 = a_2}_{x^3=27}, \quad \underbrace{c_2 = a_3}_{s=30}$$

where a_j denotes the cell of column a in row j . Figure 2 draws them as wires. They turn a list of unrelated gates into a single dataflow; enforcing them is an argument in its own right, the permutation argument of §2.6.



wires: x shared by rows 0–2 (blue); the output of each row fed to the next ($c_0 \rightarrow a_1, c_1 \rightarrow a_2, c_2 \rightarrow a_3$).

Figure 2. The four-row trace of Table 1 with the copy constraints drawn as wires. Each wire colour is one class of cells that must hold a single value: the blue wire ties the four occurrences of x , and the three coloured wires carry each row’s output c_j into the next row’s input a_{j+1} . The gates check rows; the wires make the rows one computation.

Satisfiability of the grid. The claim is now equivalent to a purely combinatorial assertion: *there exist values for the advice cells a, b, c such that (i) every row satisfies the gate equation (1) with*

the fixed selectors, and (ii) every copy constraint holds. Knowing x is exactly knowing how to fill the advice columns. The verifier knows the fixed columns and the public input; the prover must convince him that satisfying advice exists while revealing none of it. Everything that follows is machinery for doing precisely that.

2.2 From the grid to polynomials

The grid has finitely many rows, and the aim is a statement about all rows at once that the verifier can check at *one* point. A polynomial of degree less than n is determined by its values at n distinct points (*Math Guide*, §“Lagrange interpolation”), and the rest of the section relies on this.

An index set for the rows. The table has $n = 4$ rows, and four distinct field elements must name them. The chosen four are the 4th roots of unity. A *primitive n -th root of unity* is an element ω with $\omega^n = 1$ and no smaller positive power equal to 1 (*Math Guide*, §“Roots of unity”), and the *evaluation domain* it generates is $H = \{\omega^0, \dots, \omega^{n-1}\}$, a multiplicative subgroup of $\mathbb{F}^\times$ of order n (*Math Guide*, §“The evaluation domain H and its vanishing polynomial”). In $\mathbb{F}_{97}$ take $\omega = 22$: then $\omega^2 = 96 = -1$ and $\omega^4 = 1$, so

$$H = \{\omega^0, \omega^1, \omega^2, \omega^3\} = \{1, 22, 96, 75\} = \{1, 22, -1, -22\},$$

and row i lives at the point ω^i . Roots of unity are chosen because they form a cyclic group: the row after ω^i is $\omega \cdot \omega^i$, so reading a column at the next row is evaluating its polynomial at ωX ; the polynomial vanishing on H is $X^n - 1$; and interpolation over H is a fast Fourier transform (*Math Guide*, §“The fast Fourier transform”). The gate check of this subsection alone would hold over any four distinct points.

Columns become polynomials. Each column of the table is four numbers, one per row. Interpolate: for the advice column a there is a unique polynomial $a(X)$ of degree < 4 with

$$a(\omega^0) = 3, \quad a(\omega^1) = 9, \quad a(\omega^2) = 27, \quad a(\omega^3) = 30$$

(*Math Guide*, §“Lagrange interpolation”), and likewise $b(X)$, $c(X)$, the publicly known selector polynomials q_M, q_L, q_R, q_O, q_C and the public-input polynomial PI, each of degree < 4 . Computed over $\mathbb{F}_{97}$, the nine interpolants are

$$\begin{aligned} a(X) &= 90 + 61X + 22X^2 + 24X^3, & q_M(X) &= 49 + 19X + 30X^3, \\ b(X) &= 75 + 32X + 25X^2 + 65X^3, & q_L(X) &= 49 + 78X + 67X^3, \\ c(X) &= 65 + 16X + 3X^2 + 22X^3, & q_R(X) &= 73 + 24X + 73X^2 + 24X^3, \\ \text{PI}(X) &= 64 + 50X + 33X^2 + 47X^3, & q_O(X) &= 72 + 54X + 24X^2 + 43X^3, \\ & & q_C(X) &= 74 + 76X + 23X^2 + 21X^3, \end{aligned}$$

and each returns its column at the rows: b takes the values $(3, 3, 3, 0)$ on H , q_O the values $(96, 96, 96, 0)$, and PI the values $(0, 0, 0, 62)$. Interpolation is thus a bijection between columns of four values and polynomials of degree < 4 : the encoding loses no information.

The gate polynomial. Assemble the left-hand side of the gate equation (1) with the polynomials in place of the per-row numbers.

Theorem 2.1 (The gate polynomial). *Let*

$$G(X) := q_M(X) a(X) b(X) + q_L(X) a(X) + q_R(X) b(X) + q_O(X) c(X) + q_C(X) + \text{PI}(X). \quad (2)$$

Then $\deg G \leq 9$, and $G(\omega^i) = 0$ for every row i if and only if every gate of the trace is satisfied.

Proof. The term $q_M ab$ multiplies three polynomials of degree at most 3, and every other term has degree at most 6, so $\deg G \leq 9$. Evaluate G at the label ω^i of row i : each column polynomial returns that row's value there, so $G(\omega^i)$ is exactly the left-hand side of (1) for row i . It is zero precisely when gate i holds, and the conjunction over i gives the claim. $\square$

For the honest trace,

$$G(X) = 36 + 27X + 54X^2 + 50X^3 + 77X^4 + 24X^5 + 43X^6 + 47X^7 + 81X^8 + 46X^9,$$

of degree exactly 9, and it vanishes at all four points of H by construction. “Every gate is satisfied” has become “ G vanishes on H ”; the copy constraints remain separate. Four separate checks are now one statement about one polynomial.

2.3 From vanishing to a single identity

The condition “ G vanishes on H ” is still a statement about four points. This subsection replaces it by a polynomial identity that refers to no individual row.

The vanishing polynomial. The *vanishing polynomial* of H is the monic polynomial (leading coefficient 1) whose roots are exactly the points of H ,

$$Z_H(X) := \prod_{i=0}^3 (X - \omega^i) = X^4 - 1,$$

where the closed form $X^n - 1$ holds for every evaluation domain of order n (*Math Guide*, §“The evaluation domain H and its vanishing polynomial”): each ω^i is a root of $X^n - 1$, there are n distinct ones, and comparing leading coefficients finishes. The closed form lets both parties evaluate Z_H at any point with $O(\log n)$ multiplications.

Theorem 2.2 (Gate satisfaction is a quotient). *Every gate of the trace holds if and only if there is a polynomial $t(X)$ with $G(X) = t(X) Z_H(X)$.*

Proof. By Theorem 2.1, every gate holds if and only if G vanishes on H . The factor theorem (*Math Guide*, §“Roots and the factor theorem”) gives $G(\omega^i) = 0$ if and only if $(X - \omega^i) \mid G$; and because the four roots are distinct, G vanishes at all of them if and only if the product of the

four linear factors divides G (*Math Guide*, §“Lagrange interpolation”, its remark on the vanishing polynomial: a polynomial vanishes on a set of distinct nodes if and only if the product of the corresponding linear factors divides it). Thus

$$G \text{ vanishes on } H \iff (X - \omega^i) \mid G \text{ for each } i \iff Z_H(X) \mid G(X), \quad (3)$$

and divisibility is the existence of the quotient t . $\square$

Satisfaction of the *gate* constraints is precisely the existence of t ; correct wiring still requires the copy constraints. For the honest trace the division comes out even: G has degree 9, Z_H has degree 4, and

$$t(X) = \frac{G(X)}{X^4 - 1} = 61 + 70X + 43X^2 + 47X^3 + 81X^4 + 46X^5$$

is a polynomial of degree 5 with remainder zero, as it must be.

Example 2.3 (Why a cheat cannot produce t). Suppose the advice is wrong: some gate fails. Then G does not vanish on all of H , so by (3) Z_H does not divide G , and *no* polynomial t satisfies $G = t Z_H$. The dividing line is exact: the quotient exists if and only if every gate is satisfied. Concretely, take a prover who claims $x^2 = 10$ and writes $c_0 = 10$ in row 0 of Table 1 instead of 9. Row 0’s gate now reads $3 \cdot 3 - 10 = -1 \neq 0$, so the tampered gate polynomial G^* has $G^*(\omega^0) = -1 = 96$ while still vanishing at the other three rows:

$$G^*(X) = 54 + 10X + 43X^2 + 74X^3 + 83X^4 + 65X^5 + 78X^6 + 47X^7 + 81X^8 + 46X^9.$$

Dividing G^* by Z_H with remainder (*Math Guide*, §“Division with remainder”) gives the quotient $t^*(X) = 67 + 14X + 78X^2 + 47X^3 + 81X^4 + 46X^5$ and a nonzero remainder. This Euclidean quotient is an illustrative choice for t^* , not a claim that it maximises the cheat’s chances. Whatever t^* she offers as the quotient, the *gap polynomial* $D(X) := G^*(X) - t^*(X) Z_H(X)$ is nonzero, since Z_H does not divide G^* . For the Euclidean quotient D is the remainder,

$$D(X) = 24(1 + X + X^2 + X^3), \quad (4)$$

of degree 3. The identity $G^* = t^* Z_H$ is false, D is the difference of its two sides, and §2.4 shows how the verifier detects $D \neq 0$.

2.4 Why checking one random point is a proof

The prover wishes to convince the verifier that $G(X) = t(X) Z_H(X)$ as polynomials, with G built from her secret advice. The verifier does not read the polynomials: they encode the witness, and reading them would cost time at least linear in the number of rows. Instead, the two run a four-step protocol. A *commitment* is, for now, a short string that fixes a value without revealing it (*Crypto Guide*, §“Commitment schemes”); §2.9 says what a commitment to a polynomial must do and which one Halo 2 uses.

Construction 2.4 (The random-point check). 1. The prover commits to her polynomials a, b, c and to the quotient t ; the selector and public-input polynomials the verifier already knows.

2. The verifier picks a point $z \in \mathbb{F}$ *uniformly at random* and sends it.
3. The prover *opens* her commitments at z : she returns the field elements $a(z), b(z), c(z), t(z)$ together with short proofs that these are the true evaluations of the committed polynomials.
4. The verifier forms $G(z)$ from the openings and the self-evaluated public polynomials via (2), computes $Z_H(z) = z^4 - 1$, and checks the single field equation

$$G(z) \stackrel{?}{=} t(z) Z_H(z). \quad (5)$$

He accepts if and only if (5) holds.

The honest case (completeness). If the prover is honest then $G = t Z_H$ holds as polynomials, hence at every point, hence at z . With the honest trace and the draw $z = 20$,

$$Z_H(20) = 20^4 - 1 = 46, \quad t(20) = 67, \quad G(20) = 75 = 67 \cdot 46 = t(20) Z_H(20),$$

and the verifier accepts. He accepts every time, for every z .

The dishonest case (soundness). Soundness rests on the fact that a nonzero polynomial of low degree has few roots.

Theorem 2.5 (Soundness of the random-point check). *Suppose the trace is invalid, so that the prover's gate polynomial G^* does not vanish on all of H , and let t^* be any polynomial she commits to as the quotient. Set $D(X) := G^*(X) - t^*(X) Z_H(X)$. Then D is a nonzero polynomial, and if the prover's polynomials a, b, c (which determine G^*) and t^* are fixed before z is drawn and the verifier receives their true evaluations at z ,*

$$\Pr_z[\text{the verifier accepts}] = \Pr_z[D(z) = 0] \leq \frac{\deg D}{|\mathbb{F}|}.$$

Proof. Were D the zero polynomial, $G^* = t^* Z_H$ would be divisible by Z_H and so, by (3), vanish on all of H , which it does not. Hence $D \neq 0$ for every choice of t^* . By hypothesis a, b, c and t^* , and with them G^* and D , are fixed before z is drawn (the verifier never receives G^* itself; from the evaluations at z he reconstructs only the single number $G^*(z)$). The check (5) passes at z exactly when $G^*(z) - t^*(z) Z_H(z) = D(z) = 0$, that is, exactly when z is a root of D . A nonzero polynomial of degree at most $\deg D$ has at most $\deg D$ roots, so a uniformly random z is one of them with probability at most $\deg D / |\mathbb{F}|$ (*Math Guide*, §“The Schwartz–Zippel lemma”). $\square$

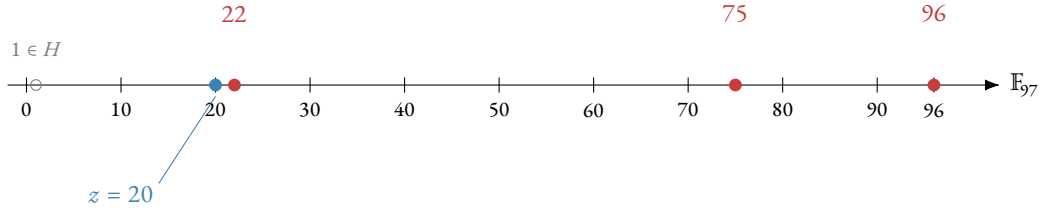
The bound $\deg D / |\mathbb{F}|$ is negligible when $|\mathbb{F}|$ is exponential in the security parameter and $\deg D$ is polynomial in it. The verifier does not recompute the trace; he tests one polynomial identity at one random point.

Example 2.6 (The random-point check on the toy). For the $x^2 = 10$ cheat of Example 2.3, $D(X) = 24(1 + X + X^2 + X^3)$ has degree 3 and so at most 3 roots; in $\mathbb{F}_{97}$ its roots are exactly $\{22, 96, 75\}$,

the three rows where the tampered gate still held (Figure 3). This particular quotient choice slips past only if the random z lands in that three-element set:

$$\Pr[\text{cheat accepted}] = \frac{3}{97}.$$

At the actual draw $z = 20 \notin \{22, 96, 75\}$ the openings give $G^*(20) = 20$ but $t^*(20) Z_H(20) = 64$; since $20 \neq 64$, the verifier rejects.



filled: the roots of the gap polynomial D , the set $H \setminus \{1\}$, where the tampered identity still holds; $D(z) = 0$ on 3 of the 97 points only. **blue:** the draw $z = 20$, where $D(20) \neq 0$ and the check rejects, as it does at every unmarked point.

Figure 3. The gap polynomial $D = 24(1 + X + X^2 + X^3)$ of the $x^2 = 10$ cheat over $\mathbb{F}_{97}$: its three roots 22, 75, 96 marked in red, the point $1 \in H$ (not a root) in grey, and the verifier's draw $z = 20$ landing elsewhere. The check (5) passes only at a root of D .

Scale. The fraction $3/97$ is large only because the field is a toy, and it concerns one particular quotient; the next paragraph bounds every cheat by $11/97$. What such a term means at the deployed field size is the scale caveat of §1.2.

Bounding the degree. The argument needs $\deg D$ small relative to $|\mathbb{F}|$, otherwise “ $\deg D$ roots out of $|\mathbb{F}|$ ” is no constraint at all. The commitment scheme enforces a degree bound on every committed polynomial (*Crypto Guide*, §“Polynomial commitment schemes”), so the prover cannot commit to a t^* of unbounded degree. The bound follows the degree the commitment admits, not the honest degree. The wires are committed at degree below $n = 4$, so $\deg G^* \leq 9$; committed as the two pieces of degree below n that the deployed rule below prescribes, t^* has degree at most 7, so $\deg(t^* Z_H) \leq 11$ and $\deg D \leq 11$. Every cheat is therefore accepted with probability at most $11/97$, and some cheat attains it, where the illustrative quotient reaches $3/97$: the t^* of degree 7 that agrees with G^*/Z_H at the eight points $2, \dots, 9$ gives D the eleven roots $2, \dots, 9, 22, 75, 96$. This is why a circuit's gates are kept to low degree, and why the quotient t , of degree 5 here and up to a few times n in general, is split into degree-bounded pieces. In the deployed system the quotient is cut into pieces of degree below n , each committed separately (the halo2 book's *Vanishing argument* chapter); “The complete Halo 2 protocol” in §4 gives the count.

2.5 PLONKish arithmetisation

The toy has met every ingredient concretely: an evaluation domain H , a generator ω , the vanishing polynomial Z_H , columns interpolated into polynomials of degree $< n$, and a gate expressed as a

polynomial identity on H . The general object can now be named.

Definition 2.7 (PLONKish constraint system). Fix a prime field $\mathbb{F}$ and an integer k with $2^k \mid |\mathbb{F}| - 1$, so that $\mathbb{F}^\times$ contains a primitive 2^k -th root of unity ω (*Math Guide*, §“Existence over finite fields: the condition $n \mid q - 1$ ”), and let $H := \{\omega^0, \dots, \omega^{n-1}\} \subset \mathbb{F}^\times$ be the multiplicative subgroup of size $n := 2^k$ that ω generates. Its vanishing polynomial is $Z_H(X) := X^n - 1$, zero exactly on H , so a congruence $p(X) \equiv 0 \pmod{Z_H(X)}$ means “ p is zero at every row ω^i ”, that is, $Z_H \mid p$. A *PLONKish circuit* C over H consists of:

1. a finite set Col of columns, each of n cells, one per row, partitioned into *fixed* columns (pre-processed circuit data, known to both parties), *advice* columns (the prover’s witness) and *instance* columns (public input); a *cell* is an element of $H \times \text{Col}$, and the circuit assigns every cell of a fixed column its value;
2. a maximum constraint degree $d_{\max}$;
3. a finite set $R \subset \mathbb{Z}$ of *rotations* containing 0; a *column expression* is a polynomial over $\mathbb{F}$ in the values $v(\omega^r X)$ for $v \in \text{Col}$ and $r \in R$, where the value of v at rotation r at row i is the cell of v in row $i + r$ taken modulo n ;
4. a finite sequence of *custom gates* $g_j = 0$, each g_j a column expression of degree at most $d_{\max}$;
5. a set $H_{\text{act}} \subseteq H$ of *active* rows: all of H in the toy, and in the deployed system the first $u < n$ rows $\omega^0, \dots, \omega^{u-1}$, the trailing rows being reserved for a boundary row and blinding (Remark 2.17 fixes u);
6. a subset $E \subseteq H_{\text{act}} \times \text{Col}$ of *equality-constrained* cells together with a permutation σ of E (a bijection $E \rightarrow E$) whose cycles, the orbits $\{\kappa, \sigma(\kappa), \sigma^2(\kappa), \dots\}$ of the cells $\kappa \in E$, are the classes of cells required to hold a common value;
7. a set of *lookup arguments*, each an input tuple $(A_0, \dots, A_{m-1})$ of column expressions and a table tuple $(S_0, \dots, S_{m-1})$ of columns.

An *assignment* gives every cell a value in $\mathbb{F}$, the fixed cells their circuit values; write v_κ for the value of cell κ . It is *satisfying* if every gate vanishes at every row of H (in the deployed system the selectors are zero outside H_{act}), $v_\kappa = v_{\sigma(\kappa)}$ for every $\kappa \in E$, and for every lookup and every active row ω^i the tuple $(A_0(\omega^i), \dots, A_{m-1}(\omega^i))$ equals $(S_0(\omega^i), \dots, S_{m-1}(\omega^i))$ for some active row $\omega^{i'}$. The *relation* $\mathcal{R}_C$ of the circuit is the set of pairs (instance-column values, advice-column values) of the satisfying assignments.

The letter d is reserved, from §3 on, for the length of a committed vector; the constraint degree is therefore written $d_{\max}$ throughout this volume.

Vanilla PLONK as the special case. The original PLONK system of Gabizon, Williamson and Ciobotaru uses a single fixed five-selector universal gate,

$$q_M(X) a(X) b(X) + q_L(X) a(X) + q_R(X) b(X) + q_O(X) c(X) + q_C(X) + \text{PI}(X) \equiv 0 \pmod{Z_H(X)},$$

with three wire polynomials a, b, c , selector polynomials q_M, q_L, q_R, q_O, q_C and the public-input polynomial PI, carrying the negated public inputs; the toy’s gate (1) is exactly this gate, with PI held in an instance column. PLONKish generalises it by permitting designer-chosen gates of higher arity and degree, each switched by its own selector $q_i(X)$, a fixed column that vanishes on the rows where the gate is inactive. In the toy the five selectors are fixed columns, a, b, c are advice, PI is the instance column, the single gate has degree 3 and reads rotation 0 only, and the copy constraints of §2.1 are the equality set E with σ cycling each class; there are no lookups until §2.7. Figure 4 shows the general shape.

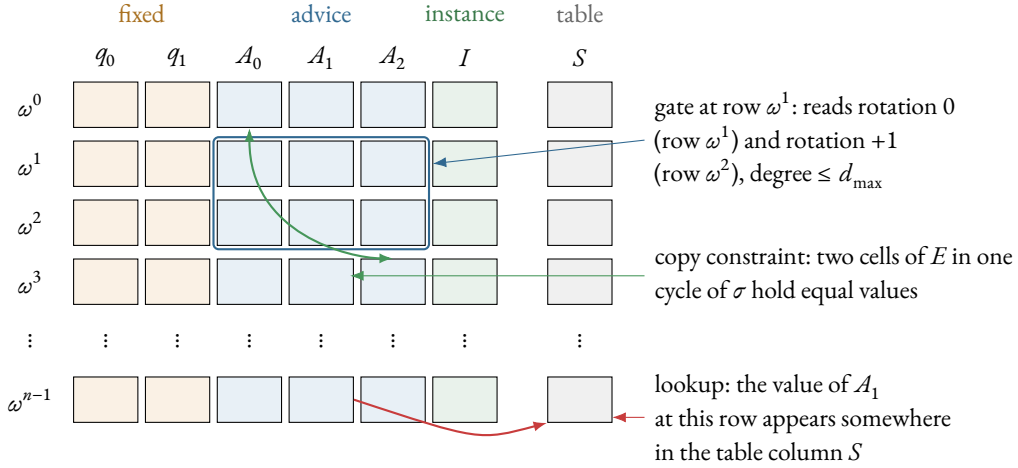


Figure 4. The PLONKish grid of Definition 2.7: columns by kind (fixed q_0, q_1 ; advice A_0, A_1, A_2 ; instance I ; a fixed table column S), rows indexed by the domain H . The outlined gate at row ω^1 reads its columns at rotations 0 and +1; the green double arrow is one cycle of the equality permutation σ joining two advice cells; the red arrow is a lookup asserting that an advice value appears in S .

Columns as committed polynomials. With the Lagrange basis $\ell_j(X)$ of H , defined by $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ for $j' \neq j$ (*Math Guide*, §“The Lagrange basis on H ”), a column with cells $v_0, \dots, v_{n-1}$ is the polynomial $v(X) = \sum_j v_j \ell_j(X)$ of degree $< n$, exactly as the toy’s columns were interpolated in §2.2. The prover commits to each advice polynomial, and the fixed polynomials are committed once for all proofs, with the inner-product polynomial commitment of §3. Every argument that follows — gates, permutation, lookup, vanishing — is a polynomial identity on these committed polynomials, verified by querying them at random points exactly as §2.4 queried G and t .

2.6 Enforcing the wiring: the permutation argument

The copy constraints of §2.1 remain to be enforced. The gate identity of §§2.2–2.4 checks each row separately; it does not assert that a_0, b_0, b_1, b_2 are the same value x , or that row 1’s input is row 0’s output, and a prover could satisfy every gate with rows that do not form one computation. The *permutation argument* reduces the equalities to the equality of two products, which is checked at random challenges.

Equalities as a permutation. Group the cells into the classes that must share a value, $\{a_0, b_0, b_1, b_2\}$, $\{c_0, a_1\}$, $\{c_1, a_2\}$, $\{c_2, a_3\}$, and let σ be the permutation of the twelve cells that cycles each class in the order written and fixes b_3 and c_3 :

$$a_0 \rightarrow b_0 \rightarrow b_1 \rightarrow b_2 \rightarrow a_0, \quad c_0 \leftrightarrow a_1, \quad c_1 \leftrightarrow a_2, \quad c_2 \leftrightarrow a_3.$$

Give every cell a distinct *label*: the cell in row j of the a -, b -, c -column gets ω^j , $k_1 \omega^j$, $k_2 \omega^j$ respectively, where k_1, k_2 place the three columns in disjoint cosets of H so that no two labels collide. Two cosets of a subgroup are equal or disjoint (*Math Guide*, §“Cosets and Lagrange’s theorem”), so it suffices that k_1, k_2 and k_2/k_1 all lie outside H ; in $\mathbb{F}_{97}$ take $k_1 = 2$, $k_2 = 3$:

$$H = \{1, 22, 96, 75\}, \quad 2H = \{2, 44, 95, 53\}, \quad 3H = \{3, 66, 94, 31\}.$$

Write λ_{cell} for a cell’s label and v_{cell} for its value. Asserting “cells in a class are equal” is then equivalent to asserting that the *multiset* (a finite collection in which an element may occur more than once; two multisets are equal when every element occurs equally often in both) of pairs $\{(v_{\text{cell}}, \lambda_{\text{cell}})\}$ is unchanged when each label is replaced by the label of the cell’s image under σ : a pair (v, λ) moves to (v, λ_σ) , and the moved pair was already present exactly when the cell at λ_σ carried the same value v . Figure 5 draws the cycles with their labels. Because labels are distinct, comparing two multisets of pairs is a job for a *product*: compress each pair to the linear form $v + \beta \lambda + \gamma$ with verifier challenges β, γ , and compare

$$\prod_{\text{cells}} (v_{\text{cell}} + \beta \lambda_{\text{cell}} + \gamma) \quad \text{against} \quad \prod_{\text{cells}} (v_{\text{cell}} + \beta \lambda_{\sigma(\text{cell})} + \gamma). \quad (6)$$

If the wiring is honest the two products run over the same multiset of factors and agree identically in β, γ . If a copy constraint is false the difference is a nonzero polynomial in β, γ , which can still vanish at one particular challenge pair, but only rarely; the converse is probabilistic, not an “if and only if” for fixed challenges. Theorem 2.11 below makes both halves precise.

The running product. A product over all cells is not a local fact, so the prover commits to a *running product*: a column Z_{perm} with $Z_{\text{perm}}(\omega^0) = 1$ whose row $j + 1$ equals row j times the ratio of row j ’s factors in (6), numerator over denominator. After the last row the product has run over every cell, and because $\omega^n = \omega^0$ the row-to-row rule at the last row forces the full product to equal $Z_{\text{perm}}(\omega^0) = 1$ again: the grand product is 1 exactly when the running product closes its loop. The deployed circuit imposes the rule cross-multiplied, “next row times denominator equals this row times numerator”, not as a field division, so a zero factor never makes the constraint undefined; the prover commits to Z_{perm} and the verifier checks the boundary value and the row-to-row update at the random point, exactly as he checked the gate identity (the halo2 book’s *Permutation argument* chapter). The identities are written out in Construction 2.10.

A single wired pair. Strip the product down to a single wired pair, $c_0 = a_1$ (both should be 9), with labels $\lambda_{c_0} = 3$ and $\lambda_{a_1} = 22$ that σ swaps. Their factors satisfy

$$(9 + \beta \lambda_{c_0} + \gamma)(9 + \beta \lambda_{a_1} + \gamma) = (9 + \beta \lambda_{a_1} + \gamma)(9 + \beta \lambda_{c_0} + \gamma),$$

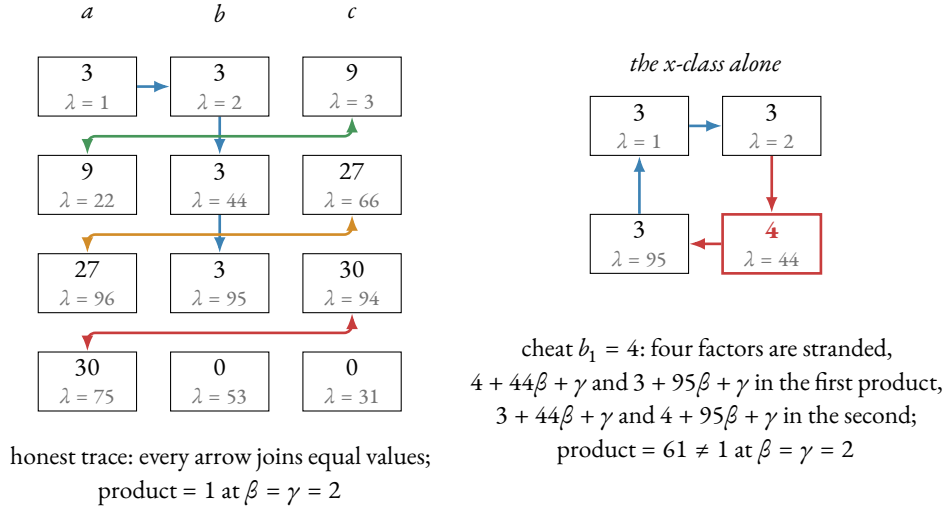


Figure 5. Left: the four copy classes of the toy as cycles of σ on the grid, every cell showing its value and its label λ (column a in H , column b in $2H$, column c in $3H$); the arrow closing the x -class cycle, from b_2 back to a_0 , is omitted. Right: the x -class with b_1 tampered to 4; the two arrows leaving and entering the tampered cell now join unequal values, so four factors of (6), two in each product, have no partner in the other product.

because the shared value 9 makes the two left-hand factors a mere reordering of the two right-hand ones: the labels may permute under σ precisely because the values agree. Now let the prover cheat the copy, $a_1 = 8 \neq 9$. The factor $8 + \beta \lambda_{a_1} + \gamma$ on the left is matched on the right only by $8 + \beta \lambda_{c_0} + \gamma$, which differs since the labels differ; the two products differ as polynomials in β, γ . A single unequal value thus leaves a factor without a partner, and the two products differ.

Example 2.8 (The permutation check on the toy). For the honest trace with $\beta = \gamma = 2$, the ratio of the two products in (6) over all twelve cells is exactly 1. Tamper one wire, setting $b_1 = 4$ and so breaking the copy $b_1 = x = 3$, and the same ratio at the same challenges becomes $61 \neq 1$: the inconsistent dataflow is exposed without any value being revealed.

The general argument. Halo 2 states the argument for any number of columns, and the statement needs one lemma on two-variable polynomials. The Schwartz–Zippel lemma of the *Math Guide* (§“The Schwartz–Zippel lemma”) is univariate; that section’s remark on the multivariate generalisation states the several-variable bound without proof, as a consequence of the univariate lemma applied once per variable, and Lemma 2.9 carries out that derivation for two variables.

Lemma 2.9 (Two variables). *Let $P \in \mathbb{F}[\beta, \gamma]$ be nonzero, of degree at most N in each variable separately. For β, γ drawn independently and uniformly from $\mathbb{F}$, $\Pr[P(\beta, \gamma) = 0] \leq 2N/|\mathbb{F}|$.*

Proof. Write $P = \sum_e c_e(\beta) \gamma^e$ with coefficients $c_e \in \mathbb{F}[\beta]$ of degree at most N . Since $P \neq 0$, some $c_{e_0} \neq 0$, and $c_{e_0}(\beta) = 0$ with probability at most $N/|\mathbb{F}|$ (*Math Guide*, §“The Schwartz–Zippel lemma”). When $c_{e_0}(\beta) \neq 0$, the polynomial $P(\beta, \cdot) \in \mathbb{F}[\gamma]$ is nonzero of degree at most N , so it vanishes at the independent uniform γ with probability at most $N/|\mathbb{F}|$. The union bound (*Math Guide*, §“The union bound and a birthday calculation”) adds the two. $\square$

Construction 2.10 (Permutation argument over m columns). Let $v_0(X), \dots, v_{m-1}(X)$ be the interpolants of the m columns that participate in equality constraints, and let $N = mn$ be the number of participating cells. Label the cell (i, j) (column i , row j) by $\delta_i \omega^j$, where $\delta_0, \dots, \delta_{m-1} \in \mathbb{F}^\times$ are chosen so that the cosets $\delta_i H$ are pairwise disjoint and all N labels are distinct; the deployed choice is $\delta_i = \delta^i$ for an element δ whose multiplicative order is the odd part of $|\mathbb{F}^\times|$ (the halo2 book's *Permutation argument* chapter). Preprocessing fixes the *permuted-label polynomials* $s_i(X)$ with $s_i(\omega^j) = \delta_{i'}$ whenever $\sigma(i, j) = (i', j')$; they are preprocessed like the fixed columns but are not circuit columns, and the verifying key — the verifier's preprocessed description of the circuit, holding commitments to every fixed column and every permuted-label polynomial — contains their commitments. After the columns are committed the verifier sends challenges $\beta, \gamma \in \mathbb{F}$, and the prover commits to a running-product polynomial $Z_{\text{perm}}(X)$ and proves the two identities, both modulo $Z_H(X)$,

$$\begin{aligned} \ell_0(X) (1 - Z_{\text{perm}}(X)) &\equiv 0, \\ Z_{\text{perm}}(\omega X) \prod_{i=0}^{m-1} (v_i(X) + \beta s_i(X) + \gamma) - Z_{\text{perm}}(X) \prod_{i=0}^{m-1} (v_i(X) + \beta \delta_i X + \gamma) &\equiv 0, \end{aligned} \quad (7)$$

which together assert

$$\prod_{i,j} (v_i(\omega^j) + \beta \delta_i \omega^j + \gamma) = \prod_{i,j} (v_i(\omega^j) + \beta s_i(\omega^j) + \gamma). \quad (8)$$

The verifier opens Z_{perm} at the random point z and at ωz , and checks the boundary and the cross-multiplied update.

The first identity fixes $Z_{\text{perm}}(\omega^0) = 1$; the second, holding at every row j , gives $Z_{\text{perm}}(\omega^{j+1}) = Z_{\text{perm}}(\omega^j) \cdot \prod_i (v_i + \beta \delta_i \omega^j + \gamma) / \prod_i (v_i + \beta s_i(\omega^j) + \gamma)$ whenever the denominator is nonzero, and at $j = n - 1$ it returns to row ω^0 , whence (8). The toy is the case $m = 3$ with $(\delta_0, \delta_1, \delta_2) = (1, 2, 3)$, which is not of the deployed form δ^i .

Theorem 2.11 (Soundness of the permutation argument). *The assignment satisfies every equality constraint (cells related by σ carry equal values) if and only if the two sides of (8) are equal as polynomials in β and γ . Consequently the honest prover always satisfies (8), and if some constraint is violated the two sides agree at uniformly random β, γ with probability at most $2N/|\mathbb{F}|$, where N is the number of participating cells.*

Proof. Index cells by κ , write v_κ and λ_κ for value and label, and set

$$P(\beta, \gamma) := \prod_{\kappa} (\gamma + v_\kappa + \beta \lambda_\kappa) - \prod_{\kappa} (\gamma + v_\kappa + \beta \lambda_{\sigma(\kappa)}) \in \mathbb{F}[\beta][\gamma].$$

Each side is a product of monic linear polynomials in γ over the ring $\mathbb{F}[\beta]$, with roots $r_\kappa := -(v_\kappa + \beta \lambda_\kappa)$ and $r'_\kappa := -(v_\kappa + \beta \lambda_{\sigma(\kappa)})$; two such roots are equal in $\mathbb{F}[\beta]$ exactly when their value-label pairs coincide, since a polynomial in β is determined by its coefficients.

If every constraint holds then $v_{\sigma(\kappa)} = v_\kappa$ for all κ , so $r'_\kappa = -(v_{\sigma(\kappa)} + \beta \lambda_{\sigma(\kappa)}) = r_{\sigma(\kappa)}$, and the second product is the first with its factors reindexed by the bijection σ : $P = 0$.

If some constraint fails, pick κ_0 with $v_{\sigma(\kappa_0)} \neq v_{\kappa_0}$. The pair $(v_{\kappa_0}, \lambda_{\sigma(\kappa_0)})$ occurs among the roots r' ; among the roots r the label $\lambda_{\sigma(\kappa_0)}$ occurs exactly once, paired with $v_{\sigma(\kappa_0)} \neq v_{\kappa_0}$, so the multisets of roots differ. Cancel every common factor from both products; a factor $(\gamma - r)$ then remains on one side, say the first, and r is not a root of the other side. Substitute $\gamma := r$, a ring homomorphism $\mathbb{F}[\beta][\gamma] \rightarrow \mathbb{F}[\beta]$ (*Math Guide*, §“Roots and the factor theorem”, evaluation is a ring homomorphism; its proof holds verbatim over the coefficient ring $\mathbb{F}[\beta]$): the first product becomes 0 and the second becomes a product of nonzero elements of $\mathbb{F}[\beta]$, which is nonzero because $\mathbb{F}[\beta]$ is an integral domain (*Math Guide*, §“Degree”, Corollary “ $F[X]$ is an integral domain”). Hence $P \neq 0$. Its degree in each variable is at most N , and Lemma 2.9 bounds the probability that the random challenges hit a zero of P by $2N/|\mathbb{F}|$. $\square$

The honest prover’s ratio is 1 whenever the denominator is nonzero. The deployed boundary rule of Remark 2.17 admits the final value 0, and the error bound changes accordingly.

Corollary 2.12 (Permutation argument under the deployed boundary rule). *Let the columns be fixed before β, γ , drawn uniformly, and let the running products be constrained on the active rows $\omega^0, \dots, \omega^{u-1}$ only, with $Z_{\text{perm}}(\omega^0) = 1$ and the value at the next row ω^u permitted to be 0 or 1 (the deployed rule; Remark 2.17 fixes $u < n$ and gives the identities). If some copy constraint fails, the probability over β, γ that some running products satisfy the update and boundary identities on the active rows is at most $3N/|\mathbb{F}|$, with N the number of participating cells.*

Proof. Suppose some running products satisfy the identities and no numerator factor $v_{\kappa} + \beta \lambda_{\kappa} + \gamma$ vanishes. Then induction from $Z_{\text{perm}}(\omega^0) = 1$ keeps every running value nonzero, so the boundary value is 1, the updates telescope, and the two products over the active rows are equal; by Theorem 2.11 that happens with probability at most $2N/|\mathbb{F}|$. For each β at most N values of γ make a numerator factor vanish, which adds $N/|\mathbb{F}|$, and the union bound gives $2N/|\mathbb{F}| + N/|\mathbb{F}| = 3N/|\mathbb{F}|$. $\square$

Cost. Excluding the final commitment check, the verifier’s work for the argument is $O(m + \log n)$: evaluating the two identities (7) at the random point z needs the $2m$ linear factors, and $\ell_0(z) = (z^n - 1)/(z - 1)$ costs $O(\log n)$ multiplications by the closed form of the Lagrange basis (*Math Guide*, §“The Lagrange basis on H ”). The complete deployed verifier still carries one check linear in n , the multi-scalar multiplication (MSM; *Math Guide*, §“The mixed inner product with group elements”) that closes every opening; §3 develops it under “The verifier’s work: structured scalars and the final generator”.

The deployed chunking. The update identity in (7) has degree $m + 1$ in the committed polynomials, and the active-row factor of Remark 2.17, which masks the rows reserved for blinding, adds one more; so a single product accommodates at most $m \leq d_{\text{max}} - 2$ columns. Columns beyond that are split into chunks of at most $d_{\text{max}} - 2$, each with its own running product, and a boundary identity $\ell_0(X)(Z_{\text{perm},a}(X) - Z_{\text{perm},a-1}(\omega^u X)) \equiv 0$ copies the value at the boundary row ω^u , the first row after the active rows $\omega^0, \dots, \omega^{u-1}$, from one product to the start of the next (the halo2 book’s *Permutation argument* chapter). The column layout is implementation-specific: the protocol specification fixes the Action statement, not its circuit. The deployed Action circuit, whose

implementation is orchard, has fifteen equality-enabled columns (ten advice, one instance, four fixed) and maximum constraint degree $d_{\max} = 9$, and therefore uses three running products of 7, 7 and 1 columns, not one.

2.7 Proving a value sits in a table: lookups

Some constraints admit no low-degree gate on the constrained cells alone. “This cell is a ten-bit number” or “this chunk indexes an entry of the Sinsemilla generator table” (Sinsemilla is the hash of the *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”) has no low-degree formula in the cell itself: written as one polynomial gate, “ $v \in \{0, \dots, 2^{10} - 1\}$ ” is $\prod_{i=0}^{2^{10}-1} (v - i) = 0$, a gate of degree 2^{10} , and a table of arbitrary entries does no better. A decomposition into ten boolean-constrained bits keeps the degree at 2 but spends ten further cells on every such value. The single gate would exceed the degree bound of §2.4: in the toy field its degree 2^{10} exceeds $|\mathbb{F}| = 97$, so the bound $\deg D / |\mathbb{F}|$ is vacuous, and at any field size it multiplies the degree of the quotient the prover must split, commit and open (§2.3). The Action circuit uses range and table constraints throughout.

Membership of every entry of a column A in a table column S is, like a copy constraint, a statement about *multisets*, but it is not a permutation: A may repeat one table entry and omit others. The prover therefore commits to A' , a rearrangement of A in which equal values occupy contiguous *runs*, and to S' , a rearrangement of S in which the first row of each run of A' holds the matching table value. Two *row-local* conditions then give membership: each row of A' either begins a run, $A'_i = S'_i$, so that its value occurs in S , or continues one, $A'_i = A'_{i-1}$. The rearrangements are certified by the running product of §2.6 without labels: labelled factors fix one wiring σ , whereas here the rearrangements are the prover’s choice, so a single running product with unlabelled factors, row by row $(A'_i + \beta)(S'_i + \gamma)$ against $(A_i + \beta)(S_i + \gamma)$, certifies that each primed column is a rearrangement of its original. Induction on the row index then gives membership for every row.

Example 2.13 (A two-bit range check on the toy grid). Constrain cells to two-bit values with the table $S = (0, 1, 2, 3)$ and the column $A = (2, 3, 2, 2)$. Grouped, $A' = (2, 2, 2, 3)$; align $S' = (2, 0, 1, 3)$, a rearrangement of S placing 2 beside the first run and 3 beside the second. Row 0 starts a run ($2 = 2$), rows 1 and 2 continue it (2 above), row 3 starts a run ($3 = 3$): every check is row-local and every polynomial low-degree, however large the table (Figure 6). The unlabelled product certifies the rearrangements: at the challenges $(\beta, \gamma) = (5, 7)$ both $\prod_i (A'_i + \beta)(S'_i + \gamma)$ and $\prod_i (A_i + \beta)(S_i + \gamma)$ equal 82. The forged column $A = (2, 7, 2, 2)$ fails the row checks: its 7 must start a run in any arrangement A' , and no rearrangement of a table without a 7 can put a 7 beside it. Over all 4 arrangements of the forged column and all 24 rearrangements of the table, the row checks pass for none.

The Action circuit uses lookups of both kinds: it range-constrains each ten-bit word of a decomposition by a lookup into the table $\{0, \dots, 2^{10} - 1\}$ (the halo2 book’s *Decomposition* chapter), and it evaluates Sinsemilla by a lookup into a fixed table whose 2^{10} rows hold an index i and the two coordinates of the i -th Sinsemilla generator (the halo2 book’s *Sinsemilla* chapter; the Orchard book’s *Circuit* chapter records the deployed uses of both). A table fact costs a few committed columns and one more running product, and the gates stay low-degree. The form of the

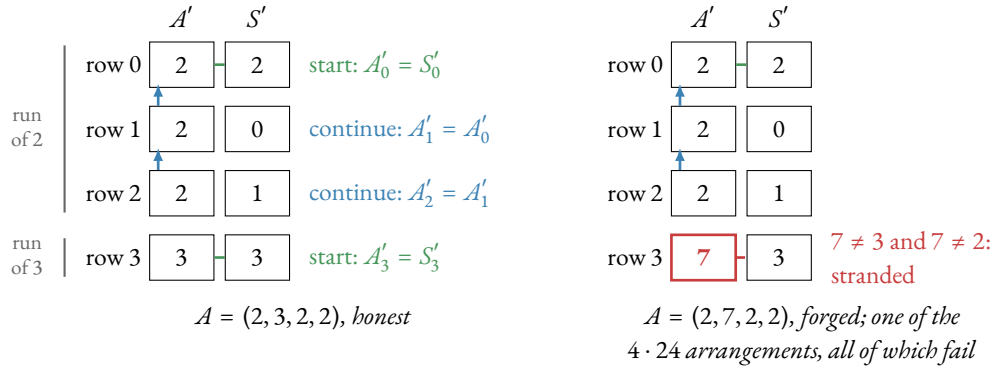


Figure 6. The lookup of Example 2.13. Left: the grouped column A' beside the aligned table S' ; a run starts where $A'_i = S'_i$ (green bar) and continues where $A'_i = A'_{i-1}$ (blue arrow to the row above). Right: the forged column with a 7; the 7 starts a run, but no entry of S' equals 7 and the row above holds 2, so both row-local checks fail at row 3.

Action circuit's gates, one gate in full, and its three lookups are given in §4 under “From statement to circuit: arithmetisation in practice”; the exact identities, the boundary row, and multi-column tables come next.

2.8 The lookup argument

Construction 2.14 (Lookup argument). The objective is that every entry of the column $A(X)$ on the active rows appears as some entry of the column $S(X)$ (the halo2 book's *Lookup argument* chapter). The prover supplies polynomials $A'(X)$ and $S'(X)$ where (i) A' is a permutation of A with like values grouped into contiguous runs, and (ii) S' is a permutation of S in which the first row of each run of like values in A' holds the matching value. The prover commits to A' and S' ; after receiving challenges β, γ she commits to a running-product column $Z_{\text{lookup}}(X)$ and proves the four identities, all modulo $Z_H(X)$,

$$\begin{aligned}
 \ell_0(X) (1 - Z_{\text{lookup}}(X)) &\equiv 0, \\
 Z_{\text{lookup}}(\omega X) (A'(X) + \beta)(S'(X) + \gamma) - Z_{\text{lookup}}(X) (A(X) + \beta)(S(X) + \gamma) &\equiv 0, \\
 \ell_0(X) (A'(X) - S'(X)) &\equiv 0, \\
 (A'(X) - S'(X))(A'(X) - A'(\omega^{-1}X)) &\equiv 0,
 \end{aligned} \tag{9}$$

the second and fourth additionally masked by the active-row factor of Remark 2.17.

Theorem 2.15 (Soundness of the lookup argument). *Let the columns A, S, A', S' be fixed before β, γ are drawn. If the four identities (9) hold on the active rows, with the boundary conditions of Remark 2.17, then except with probability at most $4n/|\mathbb{F}|$ over the challenges β, γ , every value of A on the active rows equals the value of S at some active row.*

Proof. Let the active rows be $0, \dots, u-1$ (Remark 2.17), and write $A_i := A(\omega^i)$, likewise for S, A', S' , and $Z_i := Z_{\text{lookup}}(\omega^i)$. Suppose first that no numerator factor $A_i + \beta$ or $S_i + \gamma$ with $i < u$

vanishes. The columns are fixed before the challenges, so each of the u values $-A_i$ equals β with probability $1/|\mathbb{F}|$, and likewise for γ ; by the union bound the supposition fails with probability at most $2u/|\mathbb{F}|$. Under it, the second identity at row $i < u$ reads $Z_{i+1}(A'_i + \beta)(S'_i + \gamma) = Z_i(A_i + \beta)(S_i + \gamma)$, and induction from $Z_0 = 1$ gives $Z_{i+1} \neq 0$ and $(A'_i + \beta)(S'_i + \gamma) \neq 0$ for every $i < u$. Hence $Z_u \neq 0$, the boundary condition forces $Z_u = 1$, and multiplying the identities over $i < u$ and cancelling the nonzero Z_i gives

$$\prod_{i < u} (A'_i + \beta)(S'_i + \gamma) = \prod_{i < u} (A_i + \beta)(S_i + \gamma).$$

Set $f(\beta) := \prod_i (A'_i + \beta)$, $g(\beta) := \prod_i (A_i + \beta)$, $f'(\gamma) := \prod_i (S'_i + \gamma)$, $g'(\gamma) := \prod_i (S_i + \gamma)$ and $P := ff' - gg' \in \mathbb{F}[\beta, \gamma]$. If A' is a permutation of A and S' of S then $f = g$ and $f' = g'$, so $P = 0$. If instead, say, A' is not a permutation of A , then the multisets of roots $\{-A'_i\}$ and $\{-A_i\}$ differ, so $f \neq g$ by the cancel-and-evaluate argument in the proof of Theorem 2.11 (now over the field $\mathbb{F}$ itself); comparing the coefficients of the top power of γ in $ff' = gg'$, both f' and g' being monic of the same degree, would force $f = g$, so $P \neq 0$, of degree at most u in each variable, and Lemma 2.9 bounds the probability that the challenges hide the difference by $2u/|\mathbb{F}|$. The case of S' is symmetric. The two terms together are at most $2u/|\mathbb{F}| + 2u/|\mathbb{F}| \leq 4n/|\mathbb{F}|$.

So, with that probability excepted, A' is a rearrangement of A and S' of S on the active rows. The third identity fixes row 0: $A'_0 = S'_0$, a table value. The fourth, at every later active row i , says $(A'_i - S'_i)(A'_i - A'_{i-1}) = 0$: either $A'_i = S'_i$, a table value, or $A'_i = A'_{i-1}$. (At row 0 the reference $A'(\omega^{-1}X)$ wraps around to the last row and carries no information, which is why the third identity is needed there.) By induction on i , every A'_i equals the entry of S' at some active row, hence that of S at some active row; and since every entry of A on the active rows occurs in A' there, the claim follows. $\square$

Multi-column tables. A lookup of m input expressions $A_0, \dots, A_{m-1}$ against m table columns $S_0, \dots, S_{m-1}$ is reduced to a single column by a random linear combination with a challenge θ drawn after the input and table columns are committed:

$$A_c(X) := \sum_{j=0}^{m-1} \theta^{m-1-j} A_j(X), \quad S_c(X) := \sum_{j=0}^{m-1} \theta^{m-1-j} S_j(X), \quad (10)$$

and Construction 2.14 runs on A_c and S_c .

Proposition 2.16 (Compression of a multi-column lookup). *Let θ be drawn uniformly after all input and table columns are fixed. If the input tuple at some active row equals the table tuple at no active row, then the compressed input A_c at that row equals S_c at some active row with probability at most $n(m-1)/|\mathbb{F}|$.*

Proof. For each active table row the difference of the compressed values is a polynomial in θ of degree at most $m-1$ whose coefficients are the differences of the tuple entries, not all zero; it is therefore nonzero, and the random θ zeroes it with probability at most $(m-1)/|\mathbb{F}|$ (*Math Guide*, §“The Schwartz–Zippel lemma”). A union bound over the at most n active rows gives $n(m-1)/|\mathbb{F}|$. $\square$

Variable tables, where S is itself an advice column, and range checks, where S enumerates the allowed range, are special cases of the same argument.

Remark 2.17 (Active rows and blinding). The lookup identities, like the permutation identities of §2.6, hold on the *active* rows only. The deployed system withholds the trailing rows of every column from them: a boundary row and, after it, n_{blind} blinding rows, which in every witness column (advice, and the permuted and running-product columns of the arguments) hold uniform random values, for the reason “Zero knowledge: hiding the witness in Halo 2” in §4 gives; fixed and instance columns carry no random values. Let $u := n - n_{\text{blind}} - 1$. Rows $0, \dots, u - 1$ are the active rows, row u is the boundary row, and rows $u + 1, \dots, n - 1$ are the blinding rows. (The letter u indexes a row here; in §3 it denotes a round challenge.) Let $\ell_{\text{blind}}(X)$ be the sum of the Lagrange basis polynomials of the n_{blind} blinding rows, and let $\ell_{\text{last}}(X) := \ell_u(X)$ select the boundary row. Both are computed directly from the domain by both parties; they are not committed circuit columns. Each product-update identity, and the run identity of (9), is multiplied by the factor $(1 - (\ell_{\text{last}}(X) + \ell_{\text{blind}}(X)))$, which is 1 on the active rows and 0 elsewhere, so the random rows do not violate the active-row constraints. The wrap-around that closed the loop in Construction 2.10 is gone, so the running product is pinned at the boundary row instead, by $\ell_{\text{last}}(X) (Z(X)^2 - Z(X)) \equiv 0$, which allows $Z(\omega^u) \in \{0, 1\}$ (the halo2 book’s *Lookup argument* and *Permutation argument* chapters). The value 0 is permitted so that a vanishing *numerator* factor ($A_i + \beta$ or $S_i + \gamma$; in the permutation argument $v_\kappa + \beta \lambda_\kappa + \gamma$) still leaves the honest prover a satisfying assignment: she sets the product to 0 from the next row on. The prover cannot force this event, because β, γ are drawn after her columns are committed. A *denominator* factor that vanishes at an earlier row than every vanishing numerator factor admits no satisfying running product: the update at that row j reads $0 = Z(\omega^j)$ times a nonzero numerator, and $Z(\omega^j) \neq 0$. This happens with probability at most $2u/|F|$ for a lookup and $N/|F|$ for the permutation argument, so completeness is overwhelming, not perfect. A boundary value of 0 is not a certificate of multiset equality, so soundness also excludes the challenges at which a numerator factor vanishes: the bound of Theorem 2.15 includes that term, and Corollary 2.12 adds it for the permutation argument.

2.9 Why the prover cannot wriggle: binding commitments

The random-point argument of §2.4 rests on a hypothesis: Theorem 2.5 assumed the prover’s polynomials were fixed before she saw z . If she could choose a, b, c, t *after* learning z , she would pick numbers making (5) hold and reveal nothing real. A polynomial commitment enforces that hypothesis.

What a commitment must do. A commitment to a polynomial f is a short string $C = \text{Commit}(f)$ with two properties, both developed in the *Crypto Guide*, §“Hiding and binding”:

- **Binding.** Having published C , the prover cannot later open it as two different polynomials; C pins down f . Committing first, then receiving z , then opening, leaves her answering for the *one* polynomial she fixed in advance, the situation Theorem 2.5 assumed.

- **Hiding.** The string C reveals nothing about f . This is what will let the proof be zero-knowledge, under “Zero knowledge: hiding the witness in Halo 2” in §4.

A *polynomial* commitment adds an evaluation proof: given C and a point z , the prover can prove “ $f(z) = v$ ” for the committed f without exposing f , and no prover can produce accepting proofs of two different values at the same point (*evaluation binding*, *Crypto Guide*, §“Polynomial commitment schemes”). Theorem 2.5 concerns the idealised protocol, in which the verifier receives the true evaluations of polynomials fixed in advance. Compiled with a polynomial commitment scheme that is evaluation-binding and extractable, the four-step protocol of §2.4 becomes, under compatible composition hypotheses, an argument of knowledge (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge”, Theorem “The compilation recipe, informal”): by its layered extraction, a prover who passes either answers with the true evaluations of polynomials fixed before z , where Theorem 2.5 applies, or breaks the scheme’s evaluation binding or extractability. This volume asserts no bound on the compiled error as a sum of $\deg D/|F|$ and a commitment term (Remark 4.11, §5). For Halo 2 the extractability of the openings is Theorem 3.5 and, for the deployed opening, Corollary 3.13, under the discrete-logarithm assumption on Vesta, with the generator hash modelled as a random oracle.

How Halo 2 commits. Encode f of degree $< n$ by its coefficient vector $\mathbf{f} = (f_0, \dots, f_{n-1})$; the running example’s advice polynomial becomes $\mathbf{a} = (90, 61, 22, 24)$. Let $\mathbb{G}$ be a group of prime order $|F|$ (deployed: Vesta, of order p_{Pallas}), and let $G_0, \dots, G_{n-1}, H \in \mathbb{G}$ be generators derived from a public hash, with $\mathbf{G} := (G_0, \dots, G_{n-1})$; here H is a group element, not the evaluation domain. The commitment is the single group element

$$C = \langle \mathbf{f}, \mathbf{G} \rangle + [r]H = \sum_i [f_i] G_i + [r]H,$$

the Pedersen vector commitment of the coefficient vector with blinder $r \in F$ (*Crypto Guide*, §“Pedersen vector commitments”; the mixed inner product of a scalar vector with a vector of group elements is the *Math Guide*’s, §“The mixed inner product with group elements”). The commitment is perfectly hiding for uniform r and computationally binding under the discrete-logarithm assumption in $\mathbb{G}$, with the hash modelled as a random oracle into $\mathbb{G}$ (*Crypto Guide*, §“Pedersen vector commitments”, Theorem “Properties of the vector commitment”): two openings of one C would exhibit a nontrivial discrete-logarithm relation among the G_i and H . Proving $f(z) = v$ is proving a statement about an inner product, since

$$f(z) = \sum_i f_i z^i = \langle \mathbf{f}, (1, z, z^2, \dots, z^{n-1}) \rangle,$$

and the *inner-product argument* does this in a logarithmic number of rounds, each halving the vectors, until the claim is trivial: the evaluation proof is $O(\log n)$ group elements, and it needs *no trusted setup*, because the generators come from a public hash rather than from anyone’s secret (*Crypto Guide*, §“Polynomial commitment schemes”, the inner-product alternative). Halo 2 uses exactly this one polynomial commitment scheme and no other: the inner-product argument over a group with no pairing (the halo2 book, which the protocol specification names as the definition of the system, §5.4.10.3, “Halo 2”; its *Inner product argument* chapter), on the Vesta curve, where

the protocol specification places the Action statement’s proofs (protocol specification, § 5.4.9.6, “Pallas and Vesta”). The parameters, the generators and the deployed byte layout are given at the end of §3.

Opening a vector commitment. For the argument of this section the commitment must bind, so that the prover answers for polynomials fixed before z is drawn. A vector commitment alone does not provide step 3 of the random-point check. The only opening the Pedersen construction offers is the whole vector together with its blinder, $n + 1$ field elements: sending it reveals f , and it is no shorter than f . The verifier needs $\langle \mathbf{f}, (1, z, \dots, z^{n-1}) \rangle = v$ established for a committed $\mathbf{f}$ he never sees, in fewer than n group elements. That is the inner-product argument of §3.

3 The inner-product polynomial commitment

This section constructs the inner-product polynomial commitment that opens the commitments of §2.9 at the verifier’s point. It states the problem, performs one halving step, recurses it into the full argument with both parties’ tables, gives the verifier’s closing computation, proves knowledge soundness under the discrete-logarithm assumption, proves honest-verifier zero knowledge of the masked opening, runs the running example’s polynomial through every round on a curve of prime order 97, and in the last subsection gives the parameters, the normalisation, the opening, the bytes and the deferred check of the deployed Halo 2. Beyond the *Math Guide*, the section relies on these parts of the *Crypto Guide*: the Pedersen vector commitment (§“Pedersen vector commitments”), extractors and the tree extraction of Sigma-protocols (§“Knowledge soundness and extractors”, §“The Schnorr identification protocol”, §“Sigma-protocols”), the simulation paradigm (§“The simulation paradigm and zero knowledge”), hashing to a curve (§“Hashing to a curve point”) and the standing assumptions (§“Standing assumptions”). From this volume, the generic argument takes only the cue of §2.9; the toy reuses the running example’s advice polynomial and random point (§2.2, §2.4), restated where used; the last subsection takes the deployed circuit’s field from §1.2 and the advice, permutation, lookup and quotient objects from §2, and forward-references §4 for the row count and the remaining objects of the proof; the other references to §2 mark where the argument is used.

3.1 The problem: an inner product in fewer than d group elements

The prover holds a vector $\mathbf{a} = (a_0, \dots, a_{d-1})$ of d field elements. She has published one group element,

$$P = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H = \sum_{i=0}^{d-1} [a_i] G_i + [r]H,$$

the Pedersen vector commitment of $\mathbf{a}$ with blinder r under the public generators $\mathbf{G} = (G_0, \dots, G_{d-1})$ and H (*Crypto Guide*, §“Pedersen vector commitments”; the mixed inner product $\langle \mathbf{a}, \mathbf{G} \rangle$ of a scalar vector with a vector of group elements is the *Math Guide*’s, §“The mixed inner product with group elements”). The verifier holds P , a public vector $\mathbf{b} = (b_0, \dots, b_{d-1})$ of his own,

and a number v the prover claims. He wants to be sure that

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=0}^{d-1} a_i b_i = v.$$

There is a trivial protocol: the prover sends $\mathbf{a}$ and r ; the verifier recomputes P from them, which the binding of the commitment makes a meaningful check, and then computes $\langle \mathbf{a}, \mathbf{b} \rangle$ himself. It costs $d + 1$ field elements and reveals $\mathbf{a}$. Neither is acceptable here: the vector is the prover's secret advice, and the verifier of §2.4 asks for evaluations rather than polynomials because a polynomial is long, so a proof as long as the vector gains nothing. The problem is therefore:

Convince the verifier that $\langle \mathbf{a}, \mathbf{b} \rangle = v$ for the vector $\mathbf{a}$ committed in P , using fewer than d group elements, and without revealing $\mathbf{a}$.

Why this is polynomial evaluation. Let $p(X) = a_0 + a_1X + \dots + a_{d-1}X^{d-1}$ be the polynomial with coefficient vector $\mathbf{a}$, and let x be a point. Then

$$p(x) = \sum_{i=0}^{d-1} a_i x^i = \langle \mathbf{a}, (1, x, x^2, \dots, x^{d-1}) \rangle.$$

Evaluating a committed polynomial is therefore *exactly* the problem above, with the public vector $\mathbf{b}(x) := (1, x, x^2, \dots, x^{d-1})$ of powers of the point. The running example makes it concrete. The advice polynomial of §2.2 is $a(X) = 90 + 61X + 22X^2 + 24X^3$ over $\mathbb{F}_{97}$, so $\mathbf{a} = (90, 61, 22, 24)$ and $d = 4$; the random point of §2.4 is $z = 20$, so $\mathbf{b}(20) = (1, 20, 20^2, 20^3) = (1, 20, 12, 46)$ in $\mathbb{F}_{97}$; and the value the verifier needs is $a(20) = \langle \mathbf{a}, \mathbf{b}(20) \rangle = 59$. Everything the random-point check of §2.4 asks of the prover, “open a, b, c and t at z ”, is four instances of this one problem.

Where the difficulty sits. The commitment is linear in the vector: adding two commitments commits to the sum of their vectors, and scaling one by λ commits to $\lambda\mathbf{a}$ (*Crypto Guide*, §“Pedersen vector commitments”, the remark that linearity survives). The inner product is linear in $\mathbf{a}$ too. But the two linear maps land in different places, one in the group and one in the field, and by the hiding of the commitment no public operation takes P to $\langle \mathbf{a}, \mathbf{b} \rangle$. A claim about a vector of length one is checked by sending the scalar and its blinder. The argument that follows reduces the length by *halving*. Each halving step turns a claim about vectors of length 2^j into a claim of the same shape about vectors of length 2^{j-1} , at a price of two group elements, and after $k = \log_2 d$ steps the claim is about a single scalar. The argument uses exactly this linearity.

The relation. Throughout, $\mathbb{G}$ is a cyclic group of prime order p and $\mathbb{F} = \mathbb{F}_p$ its scalar field, so that $\mathbb{G}$ is a one-dimensional $\mathbb{F}$ -vector space (*Math Guide*, §“The mixed inner product with group elements”).

Definition 3.1 (The opening relation). Fix public generators $\mathbf{G} = (G_0, \dots, G_{d-1})$ and H in $\mathbb{G} \setminus \{\mathcal{O}\}$. To open the committed polynomial p at $x \in \mathbb{F}$ with claimed value v is to prove membership

in

$$\mathcal{R}_{\text{IPA}} := \left\{ ((P, x, v); (\mathbf{a}, r)) : P = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H, v = \langle \mathbf{a}, \mathbf{b}(x) \rangle, \mathbf{a} \in \mathbb{F}^d \right\},$$

$$\mathbf{b}(x) := (1, x, \dots, x^{d-1}),$$

where (P, x, v) is the public statement and $(\mathbf{a}, r)$ the witness. The condition $\mathbf{a} \in \mathbb{F}^d$ is the degree bound $\deg p \leq d - 1$: a commitment under d generators can only hold d coefficients.

Notation for the section. The following conventions hold from here to the end of §3.

- The vector length is $d = 2^k$, and d is also the exclusive degree bound and the number of generators. The round count is $k = \log_2 d$. The letter n , the row count of Definition 2.7, is not used until the deployed parameters at the end of the section, where $d = n$; the constraint degree stays $d_{\max}$ as §2.5 fixed it. The *Crypto Guide*'s sketch of this argument indexes its generators from 0 to d and bounds the degree by d inclusively; nothing changes but the labels.
- Generators are indexed from 0, $\mathbf{G} = (G_0, \dots, G_{d-1})$, so that the coefficient a_i of X^i multiplies G_i ; the *Crypto Guide*'s "Pedersen vector commitments" indexes its generators $G_1, \dots, G_n$ instead, a shift of labels only.
- The letter H names the blinding generator $H \in \mathbb{G}$ of the Pedersen commitment; the evaluation domain $H \subset \mathbb{F}$ of §2 does not appear in this section. The letter p names the group order, and with an argument, $p(X)$, the committed polynomial.
- The letter z is the running example's opening point, $z = 20$, and x the generic one. The deployed protocol has a second challenge which its description also calls z ; this volume writes it ζ .
- The structured scalars $s_i \in \mathbb{F}$ of §3.4 are scalars; the permuted-label polynomials $s_i(X)$ of Construction 2.10 are polynomials. Type and argument disambiguate. The mask $s(X)$ of §3.6 is one further polynomial, with coefficient vector $\mathbf{m}$.
- In the byte count of §3.8, a is the number of Actions, an integer, not the vector $\mathbf{a}$.
- A vector of length 2^j splits into its *low half*, the first 2^{j-1} entries, and its *high half*, the last 2^{j-1} : $\mathbf{a} = (\mathbf{a}_{\text{lo}}, \mathbf{a}_{\text{hi}})$, and likewise for $\mathbf{G}$ and $\mathbf{b}$.

3.2 One fold, every step written out

One halving step is called a *fold*. This subsection performs one fold on a vector of length d .

Carry the claim inside the group element. The commitment P says nothing about $\mathbf{b}$ or v . Bring them in: after P is fixed, the verifier samples a further generator U uniformly from $\mathbb{G} \setminus \{\mathcal{O}\}$ and sends it. Both parties form

$$C := P + [v] U. \tag{11}$$

If the prover is honest, $C = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H + [\langle \mathbf{a}, \mathbf{b} \rangle] U$: a single group element carrying three kinds of information on three independent sets of generators, the vector on $\mathbf{G}$, the blinder on H , and the inner product on U . The claim " P commits to $\mathbf{a}$ and $\langle \mathbf{a}, \mathbf{b} \rangle = v$ " has become the claim

that C has this one *shape*, for some $\mathbf{a}$ and r , with the U -coefficient tied to the $\mathbf{G}$ -coefficients by the inner product with the public $\mathbf{b}$. The fold preserves exactly this shape while halving the vectors.

Split into halves. Write $\mathbf{a} = (\mathbf{a}_{\text{lo}}, \mathbf{a}_{\text{hi}})$, $\mathbf{G} = (\mathbf{G}_{\text{lo}}, \mathbf{G}_{\text{hi}})$ and $\mathbf{b} = (\mathbf{b}_{\text{lo}}, \mathbf{b}_{\text{hi}})$, each half of length $d/2$. An inner product splits along the halves:

$$\langle \mathbf{a}, \mathbf{G} \rangle = \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{hi}} \rangle, \quad \langle \mathbf{a}, \mathbf{b} \rangle = \langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{hi}} \rangle.$$

The fold. Let $u \in \mathbb{F}^\times$ be a nonzero challenge, and define the *folded* vectors of length $d/2$,

$$\mathbf{a}' := u \mathbf{a}_{\text{lo}} + u^{-1} \mathbf{a}_{\text{hi}}, \quad \mathbf{G}' := u^{-1} \mathbf{G}_{\text{lo}} + u \mathbf{G}_{\text{hi}}, \quad \mathbf{b}' := u^{-1} \mathbf{b}_{\text{lo}} + u \mathbf{b}_{\text{hi}}, \quad (12)$$

entry by entry: $a'_i = u a_i + u^{-1} a_{i+d/2}$, $G'_i = [u^{-1}]G_i + [u]G_{i+d/2}$ and $b'_i = u^{-1}b_i + u b_{i+d/2}$ for $0 \leq i < d/2$. The coefficient vector is folded with the weights (u, u^{-1}) and the generator and evaluation vectors with the *inverse* weights (u^{-1}, u) ; this is the symmetric fold of the Halo paper and of Bulletproofs, and the reason for the inverse weights appears in the next computation.

The inner product of the folded vectors. Expand $\langle \mathbf{a}', \mathbf{G}' \rangle$ using bilinearity of the mixed inner product (*Math Guide*, §“The mixed inner product with group elements”), one term per pair of halves:

$$\begin{aligned} \langle \mathbf{a}', \mathbf{G}' \rangle &= \langle u \mathbf{a}_{\text{lo}} + u^{-1} \mathbf{a}_{\text{hi}}, u^{-1} \mathbf{G}_{\text{lo}} + u \mathbf{G}_{\text{hi}} \rangle \\ &= u u^{-1} \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{lo}} \rangle + u u \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + u^{-1} u^{-1} \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + u^{-1} u \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{hi}} \rangle \\ &= \underbrace{\langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{lo}} \rangle + \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{hi}} \rangle}_{= \langle \mathbf{a}, \mathbf{G} \rangle} + u^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + u^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle. \end{aligned}$$

The two *diagonal* terms, low with low and high with high, have weight $u u^{-1} = 1$ and reassemble the original $\langle \mathbf{a}, \mathbf{G} \rangle$: that is what the inverse weights were for. The two *cross* terms, low with high and high with low, survive with the weights u^2 and u^{-2} . The same expansion for the scalar inner product, with $\mathbf{b}'$ folded by the same weights as $\mathbf{G}'$, gives

$$\langle \mathbf{a}', \mathbf{b}' \rangle = \langle \mathbf{a}, \mathbf{b} \rangle + u^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle + u^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle.$$

The cross terms do not depend on the challenge. The four cross inner products $\langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle$, $\langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle$, $\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle$ and $\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle$ involve only the halves, not u . The prover can therefore commit to them *before* the challenge is drawn. She samples two fresh blinders $\ell, \rho \in \mathbb{F}$ and sends the two group elements

$$L := \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + [\ell]H + [\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle] U, \quad R := \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + [\rho]H + [\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle] U, \quad (13)$$

each of the same three-part shape as C : a vector part on the generators, a blinder on H , and the matching scalar inner product on U . Only then does the verifier draw u .

The fold identity. Now add the cross terms to C with the weights the expansion produced and regroup by generator set:

$$\begin{aligned}
C + [u^2]L + [u^{-2}]R &= \langle \mathbf{a}, \mathbf{G} \rangle + [r]H + [\langle \mathbf{a}, \mathbf{b} \rangle] U \\
&\quad + u^2 \left(\langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + [\ell]H + [\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle] U \right) \\
&\quad + u^{-2} \left(\langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + [\rho]H + [\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle] U \right) \\
&= \left(\langle \mathbf{a}, \mathbf{G} \rangle + u^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + u^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle \right) \\
&\quad + [r + u^2 \ell + u^{-2} \rho]H + \left[\langle \mathbf{a}, \mathbf{b} \rangle + u^2 \langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle + u^{-2} \langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle \right] U \\
&= \langle \mathbf{a}', \mathbf{G}' \rangle + [r']H + [\langle \mathbf{a}', \mathbf{b}' \rangle] U,
\end{aligned}$$

where $r' := r + u^2 \ell + u^{-2} \rho$ and the last line substitutes the two expansions. Written as one equation, the *fold identity* is

$$C' := C + [u^2]L + [u^{-2}]R = \langle \mathbf{a}', \mathbf{G}' \rangle + [r']H + [\langle \mathbf{a}', \mathbf{b}' \rangle] U. \quad (14)$$

What has been achieved. Compare the two sides of (14) with (11). The element C' has the same shape as C , over vectors of half the length: its $\mathbf{G}'$ -part is the folded coefficient vector, its H -part a blinder, and its U -coefficient the inner product of the folded vectors. The verifier can compute C' , $\mathbf{G}'$ and $\mathbf{b}'$ entirely on his own from C, L, R, u and the public data. The prover computes $\mathbf{a}'$ and r' . The claim about $(\mathbf{a}, \mathbf{b}, \mathbf{G})$ of length d has been replaced by a claim about $(\mathbf{a}', \mathbf{b}', \mathbf{G}')$ of length $d/2$, and the exchange cost exactly two group elements and one field element of challenge. Figure 7 shows the fold and identity (14). Only completeness is claimed here; that a prover who cannot open C cannot open C' either is proved in §3.5.

The evaluation vector folds into a power vector. When $\mathbf{b} = \mathbf{b}(x)$, the high half is the low half scaled: $\mathbf{b}_{\text{hi}} = x^{d/2} \mathbf{b}_{\text{lo}}$, since $b_{i+d/2} = x^{i+d/2} = x^{d/2} x^i$. Hence

$$\mathbf{b}' = u^{-1} \mathbf{b}_{\text{lo}} + u x^{d/2} \mathbf{b}_{\text{lo}} = (u^{-1} + u x^{d/2}) (1, x, \dots, x^{d/2-1}) :$$

the folded evaluation vector is again a vector of powers of x , scaled by the scalar $u^{-1} + u x^{d/2}$. The verifier never needs to store $\mathbf{b}'$; he needs one scalar per round. This is the seed of the closed form g of §3.4.

The fold at $d = 4$, symbolically. For the running example's length, the halves are $\mathbf{a}_{\text{lo}} = (a_0, a_1)$, $\mathbf{a}_{\text{hi}} = (a_2, a_3)$, $\mathbf{G}_{\text{lo}} = (G_0, G_1)$, $\mathbf{G}_{\text{hi}} = (G_2, G_3)$, $\mathbf{b}_{\text{lo}} = (1, x)$ and $\mathbf{b}_{\text{hi}} = (x^2, x^3)$, so the cross terms of (13) read

$$\begin{aligned}
L &= [a_0]G_2 + [a_1]G_3 + [\ell]H + [a_0x^2 + a_1x^3] U, \\
R &= [a_2]G_0 + [a_3]G_1 + [\rho]H + [a_2 + a_3x] U,
\end{aligned}$$

and the folded vectors are $\mathbf{a}' = (ua_0 + u^{-1}a_2, ua_1 + u^{-1}a_3)$, $\mathbf{G}' = ([u^{-1}]G_0 + [u]G_2, [u^{-1}]G_1 + [u]G_3)$ and $\mathbf{b}' = (u^{-1} + ux^2) (1, x)$. Section 3.7 fills these in with numbers.

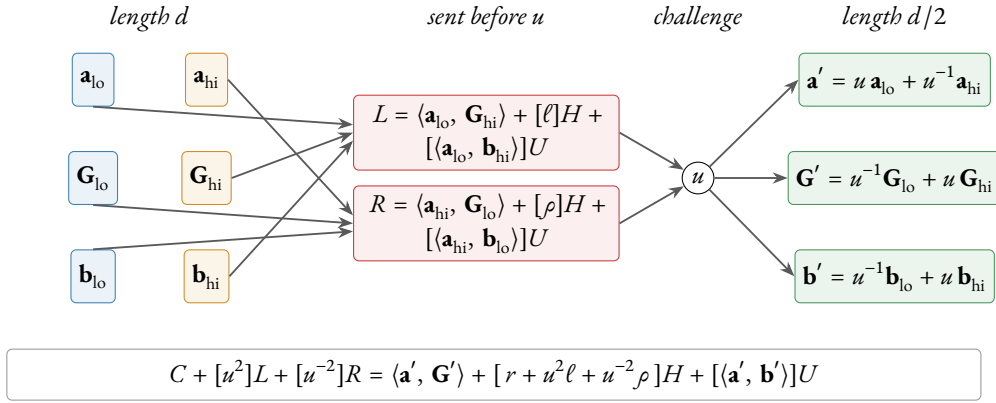


Figure 7. One fold. The three vectors of length d (left; low halves in blue, high halves in amber) determine the two cross terms L and R (red): L from $\mathbf{a}_{lo}$, $\mathbf{G}_{hi}$ and $\mathbf{b}_{hi}$, R from the complementary halves, both sent before the challenge u is drawn. The challenge then folds all three vectors to length $d/2$ (green). The identity beneath is equation (14): the folded commitment has the same three-part shape as the original, over the folded vectors, with the cross terms absorbed at weights u^2 and u^{-2} .

3.3 The recursion: k rounds to a single scalar

The folded claim of §3.2 has the shape of the original claim, so it can be folded again. After $k = \log_2 d$ folds the vectors have length one, and a length-one claim is checked by sending the scalar and its blinder. That is the whole protocol; this subsection writes it out as a construction, numbers the rounds, and lays the two parties' work side by side.

Construction 3.2 (IPA-based polynomial commitment). Let G be a cyclic group of prime order p in which the discrete logarithm problem is hard (*Crypto Guide*, §“Standing assumptions”), and let $F = F_p$. Choose $d = 2^k$ and sample the generators $\mathbf{G} = (G_0, \dots, G_{d-1})$ and H independently and uniformly from $G \setminus \{\mathcal{O}\}$, as in the *Crypto Guide*’s Construction “Pedersen vector commitment” (§“Pedersen vector commitments”). Deployed generators are instead hash-to-curve outputs on public labels, uniform when hash-to-curve is modelled as a random oracle into G (*Crypto Guide*, §“Hashing to a curve point”, Remark “Why NUMS generators are binding-safe”).

Commitment. For $p(X) = \sum_{i=0}^{d-1} a_i X^i \in F[X]$ and a blinder $r \in F$,

$$\text{Commit}(p; r) := \langle \mathbf{a}, \mathbf{G} \rangle + [r]H \in G.$$

Opening. To prove $((P, x, v); (\mathbf{a}, r)) \in \mathcal{R}_{\text{IPA}}$, the two parties proceed as follows.

1. **Setup.** The verifier samples U uniformly from $G \setminus \{\mathcal{O}\}$ and sends it. Both parties set $C^{(k)} := P + [v]U$, $\mathbf{G}^{(k)} := \mathbf{G}$ and $\mathbf{b}^{(k)} := \mathbf{b}(x)$; the prover sets $\mathbf{a}^{(k)} := \mathbf{a}$ and $r^{(k)} := r$.
2. **Rounds.** For $j = k, k-1, \dots, 1$, the vectors $\mathbf{a}^{(j)}$, $\mathbf{G}^{(j)}$, $\mathbf{b}^{(j)}$ have length 2^j and are split into halves as in §3.2.

- (a) The prover samples independent uniform $\ell_j, \rho_j \in \mathbb{F}$ and sends

$$\begin{aligned} L_j &:= \langle \mathbf{a}_{\text{lo}}^{(j)}, \mathbf{G}_{\text{hi}}^{(j)} \rangle + [\ell_j]H + [\langle \mathbf{a}_{\text{lo}}^{(j)}, \mathbf{b}_{\text{hi}}^{(j)} \rangle] U, \\ R_j &:= \langle \mathbf{a}_{\text{hi}}^{(j)}, \mathbf{G}_{\text{lo}}^{(j)} \rangle + [\rho_j]H + [\langle \mathbf{a}_{\text{hi}}^{(j)}, \mathbf{b}_{\text{lo}}^{(j)} \rangle] U. \end{aligned}$$

- (b) The verifier sends a uniform $u_j \in \mathbb{F}^\times$.

- (c) Both parties fold by (12):

$$\begin{aligned} \mathbf{G}^{(j-1)} &:= u_j^{-1} \mathbf{G}_{\text{lo}}^{(j)} + u_j \mathbf{G}_{\text{hi}}^{(j)}, & \mathbf{b}^{(j-1)} &:= u_j^{-1} \mathbf{b}_{\text{lo}}^{(j)} + u_j \mathbf{b}_{\text{hi}}^{(j)}, \\ C^{(j-1)} &:= C^{(j)} + [u_j^2]L_j + [u_j^{-2}]R_j; \end{aligned}$$

the prover also folds $\mathbf{a}^{(j-1)} := u_j \mathbf{a}_{\text{lo}}^{(j)} + u_j^{-1} \mathbf{a}_{\text{hi}}^{(j)}$ and $r^{(j-1)} := r^{(j)} + u_j^2 \ell_j + u_j^{-2} \rho_j$.

3. **Final message.** After round 1 every vector has length one. Write $a := \mathbf{a}^{(0)}$, $G^{(0)} := \mathbf{G}^{(0)}$ and $b^{(0)} := \mathbf{b}^{(0)}$ for the single entries. The prover sends the scalar a and the aggregated blinder

$$r^* := r^{(0)} = r + \sum_{j=1}^k (u_j^2 \ell_j + u_j^{-2} \rho_j).$$

4. **Check.** The verifier accepts if and only if

$$P + [v]U + \sum_{j=1}^k ([u_j^2]L_j + [u_j^{-2}]R_j) \stackrel{?}{=} [a]G^{(0)} + [r^*]H + [a \cdot b^{(0)}]U, \quad (15)$$

where the left-hand side is $C^{(0)}$ and $G^{(0)}$, $b^{(0)}$ are computed as §3.4 describes.

The round index descends so that round j begins with vectors of length 2^j ; round k is the first round and round 1 the last. The scheme instantiates the polynomial-commitment interface of the *Crypto Guide* (§“Polynomial commitment schemes”, the inner-product alternative), and the opening protocol is the inner-product argument of Bootle, Cerulli, Chaidos, Groth and Petit as refined in Bulletproofs by Bünz, Bootle, Boneh, Poelstra, Wuille and Maxwell, in the symmetric normalisation those papers use.

Completeness. The honest prover always passes. By induction on the rounds,

$$C^{(j)} = \langle \mathbf{a}^{(j)}, \mathbf{G}^{(j)} \rangle + [r^{(j)}]H + [\langle \mathbf{a}^{(j)}, \mathbf{b}^{(j)} \rangle] U \quad \text{for } j = k, k-1, \dots, 0 : \quad (16)$$

at $j = k$ this is (11) with the honest P , and the step from j to $j-1$ is the fold identity (14) applied to the round's vectors, challenge and blinders. At $j = 0$ the vectors have one entry each, the inner products are products of scalars, and (16) reads $C^{(0)} = [a]G^{(0)} + [r^*]H + [a b^{(0)}]U$, which is (15). The only way an honest run can fail is a challenge $u_j = 0$, which the verifier excludes.

Proof size. The prover sends two group elements per round and two field elements at the end: $2k$ group elements plus $O(1)$, against the $d+1 = 2^k + 1$ field elements of the trivial protocol. The

verifier keeps only the challenges $u_1, \dots, u_k$ and the received points; he need not fold $\mathbf{G}$ round by round, as the next subsection shows. Figure 8 draws the k rounds, and Tables 2 and 3 list, one row per message, what each party holds, computes and sends, so that a reader can play either role; §3.7 refills the same rows with numbers.

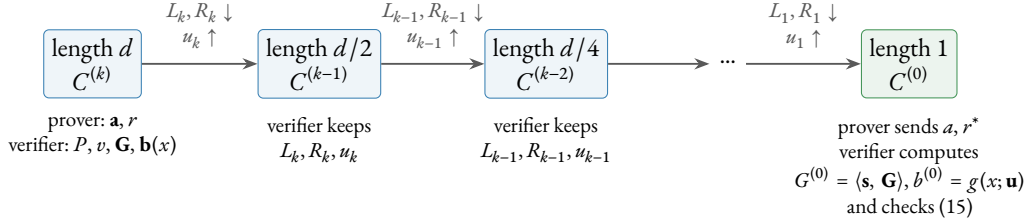


Figure 8. The k rounds of Construction 3.2. Each round halves the vector length: the prover sends two points ($\downarrow$), the verifier answers with one challenge ($\uparrow$) and keeps the three values. After k rounds one scalar and one aggregated blinder close the argument; the verifier's closing work is the subject of §3.4.

Table 2. The prover's side of Construction 3.2, one row per message. The round row repeats for $j = k, \dots, 1$.

step	receives / holds	computes and sends
setup	holds $\mathbf{a}, r, x, v$; receives U	sets $\mathbf{a}^{(k)} := \mathbf{a}, \mathbf{G}^{(k)} := \mathbf{G}, \mathbf{b}^{(k)} := \mathbf{b}(x), r^{(k)} := r$
round j	vectors of length 2^j , split into halves receives u_j	samples ℓ_j, ρ_j ; sends L_j, R_j as in Construction 3.2 folds $\mathbf{a}^{(j-1)} = u_j \mathbf{a}_{\text{lo}}^{(j)} + u_j^{-1} \mathbf{a}_{\text{hi}}^{(j)}, \mathbf{G}^{(j-1)}, \mathbf{b}^{(j-1)}$; $r^{(j-1)} = r^{(j)} + u_j^2 \ell_j + u_j^{-2} \rho_j$
final	one entry each: $a = \mathbf{a}^{(0)}, r^* = r^{(0)}$	sends a, r^*

3.4 The verifier's work: structured scalars and the final generator

The check (15) needs two quantities the verifier has not yet been told how to obtain cheaply: the final generator $G^{(0)}$ and the final evaluation scalar $b^{(0)}$. Folding $\mathbf{G}$ round by round costs $d + d/2 + \dots + 2 = 2d - 2$ scalar multiplications (two per entry of each folded vector), and folding $\mathbf{b}$ likewise many field operations. Both have closed forms in the challenges.

Theorem 3.3 (Structured scalars and the polynomial g). *For $0 \leq i < d$ write the binary expansion $i = \sum_{\ell=0}^{k-1} \beta_\ell 2^\ell$ with digits $\beta_\ell \in \{0, 1\}$, and define the structured scalars*

$$s_i := \prod_{\ell=1}^k u_\ell^{(-1)^{1-\beta_{\ell-1}}} \quad (\text{the factor is } u_\ell \text{ when } \beta_{\ell-1} = 1 \text{ and } u_\ell^{-1} \text{ when } \beta_{\ell-1} = 0). \quad (17)$$

Then, after the k folds of Construction 3.2,

$$G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = \sum_{i=0}^{d-1} [s_i] G_i, \quad b^{(0)} = \langle \mathbf{s}, \mathbf{b}(x) \rangle = g(x; u_1, \dots, u_k), \quad (18)$$

Table 3. The verifier's side of Construction 3.2, one row per message, with the closing computations of §3.4. The round row repeats for $j = k, \dots, 1$; folding C round by round is optional because (15) sums the cross terms directly.

step	receives / holds	computes and sends
setup	holds $P, x, v, \mathbf{G}, H$	samples U and sends it; sets $C^{(k)} := P + [v]U$
round j	receives L_j, R_j	samples $u_j \in \mathbb{F}^\times$ and sends it; keeps (L_j, R_j, u_j) ; optionally $C^{(j-1)} := C^{(j)} + [u_j^2]L_j + [u_j^{-2}]R_j$
final	receives a, r^*	computes $\mathbf{s}$ and $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$ (one length- d multi-scalar multiplication), $b^{(0)} = g(x; u_1, \dots, u_k)$ ($O(k)$ field operations), and checks (15)

where

$$g(X; u_1, \dots, u_k) := \prod_{\ell=1}^k (u_\ell^{-1} + u_\ell X^{2^{\ell-1}}), \quad (19)$$

a polynomial of degree $d - 1$ in X .

Proof. Follow one original generator G_m through the folds. Claim: after the rounds $k, k-1, \dots, j+1$ have been performed, G_m contributes to exactly one entry of $\mathbf{G}^{(j)}$, the entry at position $m \bmod 2^j$, with coefficient $\prod_{\ell=j+1}^k u_\ell^{(-1)^{1-\beta_{\ell-1}}}$, where $\beta_{\ell-1}$ is the digit of m at position $\ell-1$. At $j = k$ nothing has happened and the claim says G_m sits at position m with coefficient 1. For the step from j to $j-1$: round j folds the length- 2^j vector by $G_i^{(j-1)} = [u_j^{-1}]G_i^{(j)} + [u_j]G_{i+2^{j-1}}^{(j)}$ for $0 \leq i < 2^{j-1}$. The position $m \bmod 2^j$ lies in the low half $[0, 2^{j-1})$ exactly when the digit β_{j-1} of m is 0, in which case it moves to position $m \bmod 2^{j-1}$ with the extra factor u_j^{-1} ; and it lies in the high half exactly when $\beta_{j-1} = 1$, in which case it is the entry $i + 2^{j-1}$ with $i = m \bmod 2^{j-1}$ and moves to position i with the extra factor u_j . Either way the new coefficient is the old one times $u_j^{(-1)^{1-\beta_{j-1}}}$, which is the claim for $j-1$. At $j = 0$ every G_m sits in the single entry $G^{(0)}$ with coefficient s_m , which is the first half of (18). The vector $\mathbf{b}$ is folded with the same weights, so the same argument gives $b^{(0)} = \sum_m s_m b_m = \langle \mathbf{s}, \mathbf{b}(x) \rangle$.

For the closed form, expand the product (19): choosing from the ℓ -th factor either u_ℓ^{-1} (when $\beta_{\ell-1} = 0$) or $u_\ell X^{2^{\ell-1}}$ (when $\beta_{\ell-1} = 1$) produces, for each m , the monomial $s_m X^{\sum_{\ell=1}^k \beta_{\ell-1} 2^{\ell-1}} = s_m X^m$, and every $m \in [0, d)$ arises from exactly one choice. Hence $g(X) = \sum_m s_m X^m$, whose value at x is $\langle \mathbf{s}, \mathbf{b}(x) \rangle$, and whose degree is $d-1$ since the top monomial has coefficient $s_{d-1} = \prod_\ell u_\ell \neq 0$. $\square$

At $d = 4$ the scalars are $s_0 = u_1^{-1}u_2^{-1}$, $s_1 = u_1u_2^{-1}$, $s_2 = u_1^{-1}u_2$ and $s_3 = u_1u_2$, and $g(x; u_1, u_2) = (u_1^{-1} + u_1x)(u_2^{-1} + u_2x^2)$: the digit of weight 1 selects the exponent of u_1 , the round that folded pairs of neighbours, and the digit of weight 2 selects the exponent of u_2 , the round that folded the two halves of the whole vector.

The final equation, and why it checks. The verifier's closing work is now explicit. He evaluates $b^{(0)} = g(x; \mathbf{u})$ from the challenges; he computes $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$, one multi-scalar multiplication

of length d ; he adds up the left-hand side of (15) from P , U , the $2k$ received points and the challenges, $2k + 1$ scalar multiplications; and he compares it with the right-hand side, three more scalar multiplications. For the honest prover the two sides agree by the invariant (16). That a dishonest prover cannot make them agree except by knowing an opening is §3.5.

The cost of the final check. Evaluating g at x costs $3k - 2$ field multiplications: k products $u_\ell x^{2^{\ell-1}}$, $k - 1$ squarings to produce the powers $x, x^2, x^4, \dots$, and $k - 1$ multiplications of the factors together; at $k = 2$ that is four multiplications, and at Orchard’s $k = 11$ it is 31. But $G^{(0)}$ is a linear combination of *all* d generators, and computing it is a multi-scalar multiplication of length d , at Orchard’s $d = 2^{11}$ some 2048 scalar multiplications: the one part of the verifier’s work that is linear in the size of the committed vector, and the check “linear in n ” that §2.6 deferred to this subsection. Everything else the verifier does is logarithmic. The asymmetry has a name in the closed form: $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$ is nothing but $\text{Commit}(g; 0)$, the unblinded commitment to the polynomial g whose value the verifier just computed in $O(k)$ operations. Checking a *value* of g is cheap; checking its *commitment* is not. This asymmetry is the seed of the accumulation scheme of §7, and of the deferral described at the end of this section.

3.5 Why it is binding: extraction from three transcripts

This subsection shows that *any* prover that makes the verifier of Construction 3.2 accept knows an opening $(\mathbf{a}, r)$ of P with $\langle \mathbf{a}, \mathbf{b}(x) \rangle = v$, in the operational sense of the *Crypto Guide* (§“Knowledge soundness and extractors”): an extractor that runs the prover as a subroutine, rewinds it to an earlier state and feeds it fresh challenges computes the pair from its answers. The mechanism is the one that makes Schnorr’s identification protocol a proof of knowledge (*Crypto Guide*, §“The Schnorr identification protocol”): a single transcript reveals nothing, but two accepting transcripts with the same first message and distinct challenges are two linear equations, and subtracting them cancels the unknown commitment and leaves the witness. Here each round has three unknown group elements instead of one, so three transcripts are needed per round, and the k rounds stack the three-way branching into a tree, the tree special soundness of the *Crypto Guide* (§“Sigma-protocols”). The single-round step is pure linear algebra and is proved in full; the tree is an induction on it; the discrete-logarithm assumption enters once, at the root.

Representations. Fix a level j with generator vector $\mathbf{G}^{(j)}$ of length 2^j . A *representation* of an element $Q \in G$ over $(\mathbf{G}^{(j)}, H, U)$ is a triple $(\mathbf{x}; y; w) \in \mathbb{F}^{2^j} \times \mathbb{F} \times \mathbb{F}$ with

$$Q = \langle \mathbf{x}, \mathbf{G}^{(j)} \rangle + [y]H + [w]U.$$

Call the representation *well formed* at level j if its U -coordinate is the inner product of its $\mathbf{G}$ -coordinates with the round’s evaluation vector, $w = \langle \mathbf{x}, \mathbf{b}^{(j)} \rangle$. The invariant (16) says exactly that the honest prover holds a well-formed representation $(\mathbf{a}^{(j)}; r^{(j)}; \langle \mathbf{a}^{(j)}, \mathbf{b}^{(j)} \rangle)$ of $C^{(j)}$ at every level. A representation over folded generators lifts to one over the generators of the level above: if $Q = \langle \mathbf{a}', \mathbf{G}' \rangle + [y]H + [w]U$ with $\mathbf{G}' = u^{-1}\mathbf{G}_{\text{lo}} + u\mathbf{G}_{\text{hi}}$, then by bilinearity

$$Q = \langle (u^{-1}\mathbf{a}', u\mathbf{a}'), \mathbf{G} \rangle + [y]H + [w]U, \quad (20)$$

and the lift preserves well-formedness, since $\langle (u^{-1}\mathbf{a}', u\mathbf{a}'), \mathbf{b} \rangle = \langle \mathbf{a}', u^{-1}\mathbf{b}_{\text{lo}} + u\mathbf{b}_{\text{hi}} \rangle = \langle \mathbf{a}', \mathbf{b}' \rangle$. Under the discrete-logarithm assumption a prover can know at most one representation of any Q : two of them differ by a nontrivial relation among $\mathbf{G}, H$ and U , which is the binding argument of the *Crypto Guide* (§“Pedersen vector commitments”). Nothing below uses this until the root is reached.

Lemma 3.4 (Single-round recovery). *Fix one round of Construction 3.2 at level j : the verifier’s element $C := C^{(j)}$, the generators $\mathbf{G} := \mathbf{G}^{(j)}$ and evaluation vector $\mathbf{b} := \mathbf{b}^{(j)}$, and the prover’s cross terms L, R . Let u, u', u'' be three challenges for this round, and suppose that for each $v \in \{u, u', u''\}$ a representation $(\mathbf{x}_v; \gamma_v; w_v)$ of the folded element $C + [\nu^2]L + [\nu^{-2}]R$ over $(\mathbf{G}, H, U)$ is known. If the squares u^2, u'^2, u''^2 are pairwise distinct, then representations $(\mathbf{x}_L; \gamma_L; w_L)$, $(\mathbf{x}_C; \gamma_C; w_C)$ and $(\mathbf{x}_R; \gamma_R; w_R)$ of L, C and R over $(\mathbf{G}, H, U)$ are obtained by solving one 3×3 linear system with rows $(\nu^2, 1, \nu^{-2})$, and they satisfy, for each ν ,*

$$\mathbf{x}_\nu = \nu^2 \mathbf{x}_L + \mathbf{x}_C + \nu^{-2} \mathbf{x}_R, \quad \gamma_\nu = \nu^2 \gamma_L + \gamma_C + \nu^{-2} \gamma_R, \quad w_\nu = \nu^2 w_L + w_C + \nu^{-2} w_R. \quad (21)$$

If the three given representations are well formed at level j , so is the recovered representation of C .

Proof. Write the three known representations as vectors $\mathbf{Y}_\nu := (\mathbf{x}_\nu, \gamma_\nu, w_\nu) \in \mathbb{F}^{2^j+2}$ and the three unknown ones as $\mathbf{X}_L, \mathbf{X}_C, \mathbf{X}_R$. The three group equations

$$[\nu^2]L + C + [\nu^{-2}]R = \langle \mathbf{x}_\nu, \mathbf{G} \rangle + [\gamma_\nu]H + [w_\nu]U, \quad \nu \in \{u, u', u''\},$$

suggest the linear system

$$M \begin{pmatrix} \mathbf{X}_L \\ \mathbf{X}_C \\ \mathbf{X}_R \end{pmatrix} = \begin{pmatrix} \mathbf{Y}_u \\ \mathbf{Y}_{u'} \\ \mathbf{Y}_{u''} \end{pmatrix}, \quad M := \begin{pmatrix} u^2 & 1 & u^{-2} \\ u'^2 & 1 & u'^{-2} \\ u''^2 & 1 & u''^{-2} \end{pmatrix},$$

to be solved coordinate by coordinate. Multiplying the row of ν by the nonzero scalar ν^2 turns it into $(\nu^4, \nu^2, 1)$, which does not change the kernel. If $M(c_0, c_1, c_2)^\top = 0$, the polynomial $c_0W^2 + c_1W + c_2$ of degree at most 2 vanishes at $W = u^2, u'^2, u''^2$; when these squares are pairwise distinct it is zero (*Math Guide*, §“Roots and the factor theorem”, Theorem “At most d roots”), so M is invertible. When two squares coincide, two rows of M coincide and M is singular. Define $(\mathbf{X}_L, \mathbf{X}_C, \mathbf{X}_R)$ as the unique solution. Because it solves the system exactly, $\mathbf{Y}_\nu = \nu^2 \mathbf{X}_L + \mathbf{X}_C + \nu^{-2} \mathbf{X}_R$ for each ν , which is (21) read coordinate by coordinate. That the solutions are representations of L, C and R follows by taking the same linear combinations of the group equations: let $(\alpha, \alpha', \alpha'')$ be the middle row of M^{-1} , so that $\alpha(u^2, 1, u^{-2}) + \alpha'(u'^2, 1, u'^{-2}) + \alpha''(u''^2, 1, u''^{-2}) = (0, 1, 0)$ and $\mathbf{X}_C = \alpha \mathbf{Y}_u + \alpha' \mathbf{Y}_{u'} + \alpha'' \mathbf{Y}_{u''}$. Adding α times the first group equation, α' times the second and α'' times the third gives, on the left, $[0]L + [1]C + [0]R = C$, and on the right, by bilinearity, $\langle \mathbf{x}_C, \mathbf{G} \rangle + [\gamma_C]H + [w_C]U$. The first and third rows of M^{-1} do the same for L and R . For well-formedness, set $\delta_\nu := w_\nu - \langle \mathbf{x}_\nu, \mathbf{b} \rangle$ and $\delta_L := w_L - \langle \mathbf{x}_L, \mathbf{b} \rangle$, $\delta_C := w_C - \langle \mathbf{x}_C, \mathbf{b} \rangle$, $\delta_R := w_R - \langle \mathbf{x}_R, \mathbf{b} \rangle$. Subtracting the inner product of the first identity of (21) with $\mathbf{b}$ from the third gives $\delta_\nu = \nu^2 \delta_L + \delta_C + \nu^{-2} \delta_R$ for each ν , that is, $M(\delta_L, \delta_C, \delta_R)^\top = (\delta_u, \delta_{u'}, \delta_{u''})^\top$. If the given representations are well formed the right-hand side is zero, and M invertible forces $\delta_L = \delta_C = \delta_R = 0$; in particular $w_C = \langle \mathbf{x}_C, \mathbf{b} \rangle$. $\square$

Theorem 3.5 (Knowledge soundness of the opening protocol). *Let $\mathbf{G}$ be a group of prime order in which the discrete logarithm problem is hard, and let $\mathbf{G}$ and H be uniform in $\mathbf{G} \setminus \{\mathcal{O}\}$ as in Construction 3.2 (for hashed generators: hash-to-curve modelled as a random oracle into $\mathbf{G}$). The opening protocol of Construction 3.2 is knowledge sound for $\mathcal{R}_{\text{IPA}}$: there is an extractor which, given rewindable access to any prover that makes the verifier accept with non-negligible probability, runs in expected polynomial time and outputs either a witness $(\mathbf{a}, r)$ with $P = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H$ and $\langle \mathbf{a}, \mathbf{b}(x) \rangle = v$, or a nontrivial discrete-logarithm relation among $\mathbf{G}$, H and the verifier's generators U, U' . Its knowledge error is $O(\log d / |\mathbb{F}|)$.*

Proof. The proof is given at the level of this volume: the single-round step is Lemma 3.4, the tree is an induction on it, and the rewinding that supplies the sibling transcripts is the *Crypto Guide*'s (§“The Schnorr identification protocol”, the remark on extraction by rewinding, and §“Sigma-protocols”, the tree-extraction theorem cited there, which also gives the expected running time). The trees it assembles have children with pairwise distinct challenges (the *Crypto Guide*'s Definition “Tree special soundness”), while Lemma 3.4 needs pairwise distinct squares. A square has at most two square roots, so any five pairwise distinct challenges contain three with pairwise distinct squares: the rewinding assembles five children per node, and the extractor uses three of them. From a tree the extractor computes a witness or, at the root, a discrete-logarithm relation; that definition asks for a witness alone, and the difference is where the assumption enters.

Leaves. Rewind the prover to the moment after it sent (L_1, R_1) and obtain three accepting final messages (a, r^*) for three challenges u_1 with pairwise distinct squares. An accepting final message is, by (15), a representation of $C^{(0)}$ over $(G^{(0)}, H, U)$, namely $(a; r^*; a b^{(0)})$, and it is well formed at level 0: its U -coordinate is the inner product of a with the length-one vector $b^{(0)}$. Lift it to level 1 by (20): $\mathbf{x} = (u_1^{-1}a, u_1a)$.

One node. The three lifted leaves are well-formed representations of the three folded elements $C^{(1)} + [u_1^2]L_1 + [u_1^{-2}]R_1$ over $(\mathbf{G}^{(1)}, H, U)$. Lemma 3.4 at the round-1 node returns a well-formed representation of $C^{(1)}$ (and representations of L_1 and R_1 , which are not needed further).

Induction. Rewind to the moment after (L_2, R_2) , obtain three challenges u_2 with pairwise distinct squares, and under each of them repeat the two steps above. That yields, after lifting to level 2, three well-formed representations of the elements $C^{(2)} + [u_2^2]L_2 + [u_2^{-2}]R_2$, and the lemma at the round-2 node returns a well-formed representation of $C^{(2)}$. Continuing upward, after k levels the extractor holds a well-formed representation $(\mathbf{x}; y; w)$ of $C^{(k)} = P + [v]U$ over $(\mathbf{G}, H, U)$, with $w = \langle \mathbf{x}, \mathbf{b}(x) \rangle$. The tree of transcripts consumed has three children at every node and depth k , hence $3^k = d^{\log_2 3}$ leaves (Figure 9).

The root, where the assumption enters. The root representation says $P = \langle \mathbf{x}, \mathbf{G} \rangle + [y]H + [w - v]U$. If $w = v$ then $(\mathbf{x}, y)$ is a witness: $P = \langle \mathbf{x}, \mathbf{G} \rangle + [y]H$ and $\langle \mathbf{x}, \mathbf{b}(x) \rangle = w = v$. If $w \neq v$, the extractor rewinds once more, to the setup step at which the verifier sampled U , and repeats the whole extraction under an independent sample U' , obtaining $P = \langle \mathbf{x}', \mathbf{G} \rangle + [y']H + [w' - v]U'$. If $w' = v$ the witness is $(\mathbf{x}', y')$. Otherwise subtracting the two expressions for P gives

$$[w - v]U - [w' - v]U' + \langle \mathbf{x} - \mathbf{x}', \mathbf{G} \rangle + [y - y']H = \mathcal{O}$$

with the nonzero coefficient $w - v$ on U : a nontrivial relation among the independent uniform generators $\mathbf{G}, H, U, U'$, which the every-generator embedding of the *Crypto Guide* (§“Pedersen

vector commitments”, the proof of binding) turns into a discrete logarithm, the reduction embedding its challenge in the public generators and in the verifier’s samples alike. This is the only place the assumption is used.

The error. The figure does not come from a union bound over the tree. At each node the extractor needs three siblings whose challenges avoid $O(1)$ values: $\pm\nu$ for each earlier sibling’s challenge ν , so that the squares stay pairwise distinct. Each of the $k = \log_2 d$ rounds therefore contributes $O(1/|\mathbb{F}|)$, and the rounds together with the resampling of U give $O(\log d/|\mathbb{F}|)$, the theorem’s figure. Collisions among the challenges resampled inside the 3^k -leaf tree cost the extractor a rewind and a fresh sample; they affect its expected running time, not the knowledge error. A naive union bound over the roughly 3^k sibling draws would bound the tree-wide collision probability by $O(d^{\log_2 3}/|\mathbb{F}|)$, still negligible, but that is not the source of the figure. The weaker $O(d/|\mathbb{F}|)$ sometimes quoted for this argument is implied by the proved bound; a larger protocol whose own error is already linear in d loses nothing by quoting it. $\square$

What the theorem does not say. The theorem concerns the interactive protocol; the compiled argument is treated in §5. In the algebraic group model the extractor does not rewind for the representations at all: an algebraic prover hands them over with every group element it sends, and the tree collapses to a single transcript (§5, “The algebraic group model”). After the Fiat–Shamir transform has replaced the verifier’s challenges by hashes, the rewinding above must be re-proved against a prover who can query the hash adaptively and resume from any earlier transcript state (for one round, the *Crypto Guide*’s forking lemma, §“Security in the random oracle model and the forking lemma”; the multi-round case in §5), and its cost cannot be obtained by multiplying every node of the tree by an independent $1/\varepsilon$ factor. Nor does the theorem restate the binding of the commitment itself: that a single P cannot be opened as two vectors is the *Crypto Guide*’s theorem, cited in §2.9 and not reproved here.

The assumption, and what breaks it. The extractor’s alternative output is a discrete-logarithm relation among $\mathbf{G}$, H and the verifier’s U , U' , so the argument is binding as long as discrete logarithms are hard in $\mathbb{G}$ and the generators are uniform; hashed generators are uniform when hash-to-curve is modelled as a random oracle into $\mathbb{G}$ (*Crypto Guide*, §“Hashing to a curve point”, Remark “Why NUMS generators are binding-safe”), a hypothesis of Theorem 3.5. For the deployed instance the group is the Vesta curve (§3.8), and the assumption that carries the argument is the discrete logarithm on Vesta, the first of the *Crypto Guide*’s standing assumptions (§“Standing assumptions”), against which the generic attacks cost about 2^{126} operations (§“Instantiation on elliptic curves; the Pasta curves”); a quantum adversary running Shor’s algorithm lies outside those assumptions, which fix a classical adversary. Suppose the assumption fails: an algorithm computes discrete logarithms on Vesta. Then it computes the logarithms $\eta_0, \dots, \eta_{d-1}, \eta_H$ of the hashed generators to one base, and every commitment $P = [\pi]$ base opens to every vector: for any $\tilde{\mathbf{a}}$ whatsoever, the blinder $\tilde{r} := (\pi - \langle \tilde{\mathbf{a}}, \boldsymbol{\eta} \rangle) / \eta_H$ satisfies $\langle \tilde{\mathbf{a}}, \mathbf{G} \rangle + [\tilde{r}]H = P$, and the honest protocol run on $(\tilde{\mathbf{a}}, \tilde{r})$ convinces the verifier of $\langle \tilde{\mathbf{a}}, \mathbf{b}(x) \rangle$, a value the cheat chose. One commitment then opens to two vectors and to two values at one point; binding and evaluation binding fall together, and with them the premise of Theorem 2.5 that the polynomials were fixed before z was drawn.

Example 3.6 (The extractor run on the toy). The extractor is run against an honest prover for the running example’s vector $\mathbf{a} = (90, 61, 22, 24)$ opened at $z = 20$ with $v = 59$, on the prime-order-97 curve and generators of §3.7, with the blinder $r = 88$, so $P = (51, 31)$ and $C^{(2)} = P + [59]U = (43, 6)$. It fixes (L_2, R_2) , branches over three round-2 challenges $u_2 \in \{18, 81, 21\}$, and under each fixes (L_1, R_1) and branches over three round-1 challenges: $\{85, 19, 84\}$, $\{37, 43, 46\}$ and $\{38, 43, 93\}$, nine leaves in all. At the node $u_2 = 18$ the 3×3 system has determinant 67 and returns the representation $\mathbf{a}^{(1)} = (80, 0)$ with blinder 3 and U -coordinate 40 = $\langle \mathbf{a}^{(1)}, \mathbf{b}^{(1)} \rangle$, well formed as the lemma promises; the other two nodes return $(50, 41)$ and $(85, 35)$ likewise. At the root the system in u_2 has determinant 75 and returns $\mathbf{a} = (90, 61, 22, 24)$, $r = 88$ and U -coordinate $59 = v$: the extracted witness is the committed vector, its blinder, and the true value. Figure 9 draws the tree with the node $u_2 = 18$ called out.

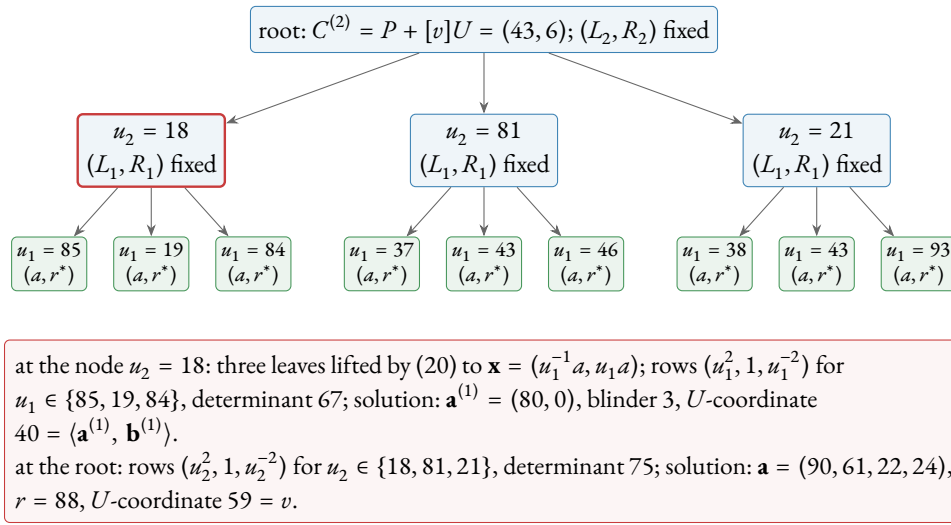


Figure 9. The extractor’s tree of accepting transcripts for $d = 4$ (Example 3.6): every node branches over three challenges, every root-to-leaf path is one accepting run of Construction 3.2, and each node’s 3×3 system of Lemma 3.4 turns its three children’s representations into its own. The red node’s system and the root’s are written out beneath.

3.6 Zero knowledge: what the blinders hide and what they do not

The transcript of one opening consists of group elements, challenges and two final scalars. The group elements are hidden by the blinders: the commitment blinder r makes P uniform in $\mathbb{G}$ whatever $\mathbf{a}$ is (perfect hiding, *Crypto Guide*, §“The Pedersen commitment” and §“Pedersen vector commitments”), and the round blinders ℓ_j, ρ_j make every L_j and R_j uniform and independent of everything else for the same reason, since each carries its own fresh $[\ell_j]H$ or $[\rho_j]H$. The aggregated blinder r^* is uniform too, being r plus a combination of the fresh ℓ_j and ρ_j . The final scalar is not hidden by these blinders.

Remark 3.7 (Folding alone is not zero knowledge). The final scalar $a = \mathbf{a}^{(0)}$ is a linear form in the committed coefficients whose coefficients the verifier knows. Follow one coefficient a_i through

the k folds exactly as the proof of Theorem 3.3 followed G_m : the coefficient side folds with the weights (u_j, u_j^{-1}) , the *inverse* of the generator side's (u_j^{-1}, u_j) , so a_i reaches the single final entry multiplied by $\prod_{\ell} u_{\ell}^{(-1)^{\beta_{\ell-1}}}$, the factor u_{ℓ} when the digit $\beta_{\ell-1}$ of i is 0 and u_{ℓ}^{-1} when it is 1. Writing $\mathbf{t}$ for this vector, $t_i = s_i^{-1}$ entry by entry, and

$$\mathbf{a}^{(0)} = \langle \mathbf{a}, \mathbf{t} \rangle, \quad t_i := \prod_{\ell=1}^k u_{\ell}^{(-1)^{\beta_{\ell-1}}} = s_i^{-1}; \quad \text{at } d = 4 : \mathbf{t} = (u_1 u_2, u_1^{-1} u_2, u_1 u_2^{-1}, u_1^{-1} u_2^{-1}). \quad (22)$$

In the toy of §3.7, with challenges $u_2 = 52$ and $u_1 = 20$, the vector is $\mathbf{t} = (70, 22, 75, 79)$, the entrywise inverse of the structured scalars $\mathbf{s} = (79, 75, 22, 70)$, and the final scalar 33 is

$$\langle (90, 61, 22, 24), (70, 22, 75, 79) \rangle.$$

One opening thus hands the verifier one linear equation in the four secret coefficients; every further opening of the same commitment under fresh challenges hands him another, and d of them determine $\mathbf{a}$ outright. Four openings of the same vector, with challenge pairs $(u_2, u_1) = (63, 29)$, $(79, 7)$, $(34, 1)$ and $(49, 74)$, end in the scalars 88, 22, 40 and 14, and solving the 4×4 system recovers $(90, 61, 22, 24)$. Blinding the commitment and the round points hides those points, not this scalar. The masked opening below therefore does not fold $\mathbf{a}$ itself: it first masks the polynomial by an independently random polynomial vanishing at the opening point, so that the linear form is taken of a vector the verifier cannot relate to $\mathbf{a}$. The mask's commitment S and the challenge that follows it are essential to the zero-knowledge argument below.

The masked opening. The masked opening runs Construction 3.2 on a masked statement.

1. **Mask.** Before any folding, the prover samples a polynomial $s(X) = \sum_{i=0}^{d-1} m_i X^i$ of degree below d with $s(x) = 0$: she draws the coefficient vector $\mathbf{m}$ uniformly from $\mathbb{F}^d$ and then replaces m_0 by $m_0 - s(x)$, which makes $\mathbf{m}$ uniform on the hyperplane $\{\mathbf{m} : \langle \mathbf{m}, \mathbf{b}(x) \rangle = 0\}$. She samples a blinder r_s and sends

$$S := \text{Commit}(s; r_s) = \langle \mathbf{m}, \mathbf{G} \rangle + [r_s]H.$$

2. **Challenge.** The verifier sends a uniform $\xi \in \mathbb{F}$.
3. **Opening.** By linearity of the commitment (*Crypto Guide*, §“Pedersen vector commitments”, the remark that linearity survives),

$$P + [\xi]S = \text{Commit}(p + \xi s; r + \xi r_s), \quad (23)$$

a commitment to the coefficient vector $\mathbf{a} + \xi \mathbf{m}$ of the polynomial $p + \xi s$, whose value at x is $p(x) + \xi s(x) = v$. The parties run Construction 3.2 on the statement $(P + [\xi]S, x, v)$ with the witness $(\mathbf{a} + \xi \mathbf{m}, r + \xi r_s)$, the verifier sampling U in its setup step, after S and ξ .

4. **Final message and check.** The prover sends the final scalar $c := (\mathbf{a} + \xi \mathbf{m})^{(0)}$ and the aggregated blinder $f := r + \xi r_s + \sum_{j=1}^k (u_j^2 \ell_j + u_j^{-2} \rho_j)$. The verifier accepts if and only if

$$P + [\xi]S + [v]U + \sum_{j=1}^k ([u_j^2]L_j + [u_j^{-2}]R_j) \stackrel{?}{=} [c]G^{(0)} + [f]H + [c \cdot b^{(0)}]U, \quad (24)$$

which is (15) for the masked statement.

Completeness is that of Construction 3.2, since the witness satisfies $\mathcal{R}_{\text{IPA}}$ for the masked statement; the mask leaves the opened value unchanged because $s(x) = 0$. The transcript of one opening is

$$(S; L_k, R_k, \dots, L_1, R_1; c, f) \quad \text{with the challenges} \quad (\xi, U, u_k, \dots, u_1).$$

The hiding argument. The final scalar is the linear form (22), taken of the masked vector:

$$c = \langle \mathbf{a} + \xi \mathbf{m}, \mathbf{t} \rangle = \langle \mathbf{a}, \mathbf{t} \rangle + \xi \langle \mathbf{m}, \mathbf{t} \rangle. \quad (25)$$

In the toy, with $\mathbf{m} = (66, 23, 23, 88)$, for which $s(20) = 0$, and $\xi = 29$, the masked vector is $(64, 49, 10, 54)$, its inner product with $\mathbf{b}(20)$ is 59, and under the challenges $u_2 = 52, u_1 = 20$ of the unmasked run the final scalar is $c = 1 = \langle (64, 49, 10, 54), \mathbf{t} \rangle$ with $\mathbf{t} = (70, 22, 75, 79)$; a second mask $\mathbf{m}' = (32, 68, 64, 88)$ under the same challenges ends in $c' = 75$, and both runs are accepted (§3.7). The same form, on a different masked vector, gives an unrelated scalar.

Proposition 3.8 (Honest-verifier zero knowledge of the masked opening). *Let $\mathbb{G}$ have prime order p , and let the verifier draw ξ uniformly from $\mathbb{F}$, U uniformly from $\mathbb{G} \setminus \{\mathcal{O}\}$ and each u_j uniformly from $\mathbb{F}^\times$. There is a simulator which, given (P, x, v) but not $(\mathbf{a}, r)$, outputs challenges and a transcript whose distribution lies within statistical distance $1/|\mathbb{F}| + (2/(|\mathbb{F}| - 1))^k$ (Math Guide, §“Statistical distance”) of the honest one. The masked opening is therefore statistical honest-verifier zero knowledge, not perfect, in the sense of the Crypto Guide (§“The simulation paradigm and zero knowledge”).*

Proof. Call a challenge tuple *good* if $\xi \neq 0$ and $\mathbf{t}$ is not a multiple of $\mathbf{b}(x)$. The vector $\mathbf{m}$ is uniform on the hyperplane $\langle \mathbf{m}, \mathbf{b}(x) \rangle = 0$, a subspace of dimension $d - 1$. The linear functional $\mathbf{m} \mapsto \langle \mathbf{m}, \mathbf{t} \rangle$ (Math Guide, §“Bilinear forms and the inner product on F^n ”) restricted to that subspace is nonzero unless $\mathbf{t}$ is a scalar multiple of $\mathbf{b}(x)$. If $\mathbf{t} = \lambda \mathbf{b}(x)$, the first entries force $\lambda = t_0 = \prod_{\ell} u_{\ell}$, and the entry at $i = 2^{\ell-1}$, whose only nonzero digit is $\beta_{\ell-1}$, reads $t_0 u_{\ell}^{-2} = t_0 x^{2^{\ell-1}}$, that is, $u_{\ell}^{-2} = x^{2^{\ell-1}}$; each of these k equations has at most two solutions u_{ℓ} , so at most 2^k challenge tuples among $(|\mathbb{F}| - 1)^k$ are excluded. A nonzero linear functional on a vector space is onto $\mathbb{F}$, and its fibres are the cosets of its kernel, all of equal size, so it carries the uniform distribution to the uniform distribution on $\mathbb{F}$. For good challenges $\xi \langle \mathbf{m}, \mathbf{t} \rangle$ is therefore uniform, and by (25) so is c , independently of $\mathbf{a}$.

In an honest run with good challenges, S and every L_j, R_j are uniform and independent (fresh blinders), c is uniform and independent of them (the mask), and f is the unique scalar that makes (24) hold once everything else is fixed, H generating $\mathbb{G}$. The simulator produces the same distribution in the reverse order, the way the Schnorr simulator chooses its commitment after the challenge (Crypto Guide, §“Sigma-protocols”): it draws the challenges as the verifier does, draws S , all L_j and all R_j but R_1 uniformly from $\mathbb{G}$, draws c and f uniformly from $\mathbb{F}$, and sets R_1 to the one point that satisfies (24), namely $R_1 := [u_1^2](\text{right-hand side} - \text{every other term of the left})$. Both distributions are uniform on the solution set of the one equation: the honest run parametrises that set by the $2k + 1$ points and c , with f determined; the simulator parametrises it by $2k$ points and the two scalars, with R_1 determined; and each parametrisation is a bijection onto that set, with p^{2k+2} points on each side since $\mathbb{G}$ and $\mathbb{F}$ both have p elements, because the determined coordinate is unique in each case. For good challenges the simulated transcript is therefore

distributed exactly as the real one. The challenges have the same distribution in both, and the tuples that are not good have probability at most $1/|\mathbb{F}| + 2^k/(|\mathbb{F}| - 1)^k$; the statistical distance is at most that probability. $\square$

The gap, stated. This argues one opening in isolation. The proof system opens many committed polynomials at once, under shared challenges, and the mask is one of several blinding devices it uses; whether the joint transcript reveals nothing is a separate question, addressed under “Zero knowledge: hiding the witness in Halo 2” in §4, where the mask, in the deployed form of §3.8, reappears as the last of those devices.

3.7 A complete toy opening, carried by script

The toy opening commits the running example’s advice polynomial, $a(X) = 90 + 61X + 22X^2 + 24X^3$ of §2.2, and opens it at the random point $z = 20$ of §2.4 with $v = a(20) = 59$: the opening the proof system needs, of a column polynomial at the verifier’s random point, never of the statement polynomial at the running example’s secret witness 3. Every value below is computed by script and can be recomputed by hand in $\mathbb{F}_{97}$ and on a curve with 97 points.

A genuine group of order 97. The vector $\mathbf{a}$ lives in $\mathbb{F}_{97}^4$, so the commitment group must have order 97: scalars of a prime-order group live in the field of its order (*Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”), exactly as the deployed circuit’s field is the scalar field of the deployed commitment curve. The group $(\mathbb{F}_{97}, +)$ has order 97 but trivially computable discrete logarithms, a division, so the toy uses an elliptic curve. For a curve over a prime field $\mathbb{F}_q$ to have exactly 97 points, the Hasse bound (*Math Guide*, §“*The group structure of $E(\mathbb{F}_p)$* ”) confines the prime q to $|q + 1 - 97| \leq 2\sqrt{q}$: $q \in \{79, 83, 89, 97, 101, 103, 107, 109, 113\}$. The case $q = 97$ is excluded on purpose: a curve with as many points as its base field has elements is *anomalous*, and its discrete logarithm is easy (*Math Guide*, §“*The elliptic-curve discrete logarithm problem*”). The first short Weierstrass curve (*Math Guide*, §“*Weierstrass equations*”) in lexicographic order of (q, A, B) with exactly 97 points is

$$E: y^2 = x^3 + 3 \quad \text{over } \mathbb{F}_{97}, \quad \#E(\mathbb{F}_{97}) = 97,$$

counted point by point; 97 is prime, so the group is cyclic and every point but $\mathcal{O}$ generates it, and its trace $79 + 1 - 97 = -17$ is not 1. It is the same kind of object as the *Math Guide*’s worked curve $y^2 = x^3 + 5$ over $\mathbb{F}_{11}$ (§“*A worked toy example*”), one size up. With 97 points every discrete logarithm on it is a short search; the toy exhibits the algebra of the argument, never its hardness.

Generators from a hash. The generators are derived from public labels by hashing to the curve, as the deployed ones are: the label and a counter are hashed to a candidate x -coordinate, and the counter is incremented until $x^3 + 3$ is a square in $\mathbb{F}_{97}$. This is a variant of the try-and-increment loop of the *Crypto Guide* (§“*Hashing to a curve point*”), which increments x itself; its constant-time refinement is irrelevant to a toy. Indexed from 0 so that the coefficient a_i multiplies G_i , the

generators are

$$\begin{aligned} G_0 &= (12, 25), & G_1 &= (17, 27), & G_2 &= (71, 26), \\ G_3 &= (49, 10), & H &= (78, 9), & U &= (53, 26). \end{aligned}$$

The hashed point U stands in for the verifier's uniform sample of Construction 3.2. No party chose any of them, so no relation among them is known in advance; a single discrete logarithm between two of them would give one, since $H = [b]G_0$ yields $[b]G_0 - H = \mathcal{O}$. On this curve such a logarithm is a short search; on Vesta its infeasibility is the assumption of §3.5.

The instance. The prover holds $\mathbf{a} = (90, 61, 22, 24)$, which takes the values $(3, 9, 27, 30)$ on the domain of §2.2, and the blinder $r = 10$; her commitment is

$$P = [90]G_0 + [61]G_1 + [22]G_2 + [24]G_3 + [10]H = (39, 25).$$

The verifier holds P , the point $z = 20$, hence $\mathbf{b}(20) = (1, 20, 20^2, 20^3) = (1, 20, 12, 46)$ in $\mathbb{F}_{97}$, and the claimed value $v = 59 = \langle \mathbf{a}, \mathbf{b}(20) \rangle$. With $d = 4$ there are $k = 2$ rounds, numbered 2 then 1.

The symmetric opening, both sides. Tables 4 and 5 refill the rows of Tables 2 and 3 with numbers, listing each party's inputs and computations; the verifier's table never uses $\mathbf{a}$.

Table 4. The prover's side of the toy opening (Table 2 with numbers): $\mathbf{a} = (90, 61, 22, 24)$ opened at $z = 20$ on the 97-point curve $y^2 = x^3 + 3$ over $\mathbb{F}_{97}$, all scalars in $\mathbb{F}_{97}$. Points are given as (x, y) with coordinates in $\mathbb{F}_{97}$.

step	receives / holds	computes and sends
setup	holds $\mathbf{a} = (90, 61, 22, 24)$, $r = 10$, $z = 20$, $v = 59$; receives $U = (53, 26)$	$\mathbf{a}^{(2)} := \mathbf{a}$, $\mathbf{G}^{(2)} := (G_0, G_1, G_2, G_3)$, $\mathbf{b}^{(2)} := (1, 20, 12, 46)$, $r^{(2)} := 10$
round 2	halves $\mathbf{a}_{\text{lo}} = (90, 61)$, $\mathbf{a}_{\text{hi}} = (22, 24)$; $\mathbf{b}_{\text{lo}} = (1, 20)$, $\mathbf{b}_{\text{hi}} = (12, 46)$ receives $u_2 = 52$ ($u_2^{-1} = 28$)	samples $\ell_2 = 44$, $\rho_2 = 16$; $\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle = 6$, $\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle = 17$; sends $L_2 = [90]G_2 + [61]G_3 + [44]H + [6]U = (74, 6)$, $R_2 = [22]G_0 + [24]G_1 + [16]H + [17]U = (38, 72)$ $\mathbf{a}^{(1)} = 52(90, 61) + 28(22, 24) = (58, 61)$; $\mathbf{b}^{(1)} = 28(1, 20) + 52(12, 46) = (70, 42)$; $\mathbf{G}^{(1)} = ([28]G_0 + [52]G_2, [28]G_1 + [52]G_3) = ((54, 40), (30, 71))$; $r^{(1)} = 10 + 52^2 \cdot 44 + 28^2 \cdot 16 = 95$
round 1	halves $a_{\text{lo}} = 58$, $a_{\text{hi}} = 61$; $b_{\text{lo}} = 70$, $b_{\text{hi}} = 42$ receives $u_1 = 20$ ($u_1^{-1} = 34$)	samples $\ell_1 = 12$, $\rho_1 = 37$; $a_{\text{lo}}b_{\text{hi}} = 11$, $a_{\text{hi}}b_{\text{lo}} = 2$; sends $L_1 = [58]G_1^{(1)} + [12]H + [11]U = (28, 25)$, $R_1 = [61]G_0^{(1)} + [37]H + [2]U = (39, 25)$ $a^{(0)} = 20 \cdot 58 + 34 \cdot 61 = 33$; $b^{(0)} = 34 \cdot 70 + 20 \cdot 42 = 19$; $G^{(0)} = [34]G_0^{(1)} + [20]G_1^{(1)} = (12, 25)$; $r^{(0)} = 95 + 20^2 \cdot 12 + 34^2 \cdot 37 = 40$
final	$a = 33$, $r^* = 40$	sends $a = 33$, $r^* = 40$

The verifier's equation (15) closes: the left-hand side $P + [59]U + [52^2]L_2 + [28^2]R_2 + [20^2]L_1 + [34^2]R_1$ is $(54, 39)$, and so is the right-hand side $[33]G^{(0)} + [40]H + [33 \cdot 19]U$. Each fold identity holds along the way: after round 2, $(72, 23) = \langle \mathbf{a}^{(1)}, \mathbf{G}^{(1)} \rangle + [95]H + [\langle \mathbf{a}^{(1)}, \mathbf{b}^{(1)} \rangle]U$, and after round 1, $(54, 39) = [33](12, 25) + [40]H + [33 \cdot 19]U$. The closed forms of Theorem 3.3 agree with the folds: $\langle \mathbf{s}, \mathbf{G} \rangle = (12, 25)$ is the $G^{(0)}$ the prover reached by folding, and

Table 5. The verifier's side of the same opening (Table 3 with numbers). He never sees $\mathbf{a}$, r , ℓ_j or ρ_j ; the structured scalars and $g(20)$ come from the two challenges alone, and $G^{(0)}$ is his one length-4 multi-scalar multiplication.

step	receives / holds	computes and sends
setup	holds $P = (39, 25)$, $z = 20$, $v = 59$, $\mathbf{G}$, $H = (78, 9)$	sends $U = (53, 26)$, a hashed point standing in for his sample; $C^{(2)} := P + [59]U = (47, 39)$
round 2	receives $L_2 = (74, 6)$, $R_2 = (38, 72)$	samples $u_2 = 52$ and sends it; keeps $(L_2, R_2, 52)$; $C^{(1)} := C^{(2)} + [52^2]L_2 + [28^2]R_2 = (72, 23)$
round 1	receives $L_1 = (28, 25)$, $R_1 = (39, 25)$	samples $u_1 = 20$ and sends it; keeps $(L_1, R_1, 20)$; $C^{(0)} := C^{(1)} + [20^2]L_1 + [34^2]R_1 = (54, 39)$
final	receives $a = 33$, $r^* = 40$	$\mathbf{s} = (34 \cdot 28, 20 \cdot 28, 34 \cdot 52, 20 \cdot 52) = (79, 75, 22, 70)$; $G^{(0)} = [79]G_0 + [75]G_1 + [22]G_2 + [70]G_3 = (12, 25)$; $b^{(0)} = g(20; 20, 52) = (34 + 20 \cdot 20)(28 + 52 \cdot 12) = 19$; right-hand side $[33]G^{(0)} + [40]H + [33 \cdot 19]U = (54, 39) = C^{(0)}$: accept

$g(20; u_1, u_2) = (u_1^{-1} + u_1 z)(u_2^{-1} + u_2 z^2) = 19$ is her $b^{(0)}$. In a group of 97 elements collisions among the printed points are frequent, and nothing follows from any of them. Here $G^{(0)}$ happens to equal G_0 and R_1 happens to equal P ; the masked run below and the runs of §3.8 show more.

A false value is rejected. Let the prover claim $v' = 60$ instead of 59. The prover's messages are those of the honest run; only $C^{(2)} = P + [60]U$ changes, by one more U . The left-hand side becomes $(62, 15)$ while the right-hand side stays $(54, 39)$, and the verifier rejects. By Theorem 3.5, a prover accepted on $v' = 60$ would yield an opening of P with value 60 and hence, beside the honest opening, a discrete-logarithm relation among the generators; on a curve of 97 points such relations are a short search, and the theorem gives no security there.

The masked opening of the same polynomial. The same P is then opened at the same z with the masked opening of §3.6. The prover samples $\mathbf{m} = (66, 23, 23, 88)$, for which $s(20) = 0$, and $r_s = 95$, and sends $S = \langle \mathbf{m}, \mathbf{G} \rangle + [95]H = (70, 71)$; the verifier answers $\xi = 29$. The masked vector $\mathbf{a} + 29\mathbf{m} = (64, 49, 10, 54)$ has inner product 59 with $\mathbf{b}(20)$, and the rounds run on it with the point U , the challenges $u_2 = 52$, $u_1 = 20$ and the round blinders of the unmasked run: the prover sends $L_2 = (36, 7)$, $R_2 = (12, 25)$, $L_1 = (51, 31)$ and $R_1 = (72, 23)$, and ends in $c = 1$ and $f = 79$. The final generator $G^{(0)} = (12, 25)$ and the final evaluation scalar $b^{(0)} = 19$ are those of the unmasked run, since they depend on the challenges alone. The verifier's equation (24) closes at $(46, 13)$ on both sides, and the false value $v' = 60$ moves the left-hand side to $(56, 70)$: rejected. A second mask $\mathbf{m}' = (32, 68, 64, 88)$ under the same challenges ends in $c' = 75$ and is accepted as well.

3.8 Halo 2’s specifics: parameters, generators from a hash, the deferred multi-scalar multiplication, and the proof bytes

Everything above holds for any prime-order group and any $d = 2^k$. This subsection, and only this one, fixes the group, the size, the generators and the normalisation of the deployed Halo 2, states its opening, and counts the bytes an Orchard proof spends on the argument. Nothing earlier in the section depends on it.

The group and the size. The Orchard Action circuit does its arithmetic in $\mathbb{F}_{p_{\text{Pallas}}}$, the base field of the Pallas curve, because the statement manipulates Pallas points and their coordinates live there (§1.2; *Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”). Its column polynomials therefore have coefficients in $\mathbb{F}_{p_{\text{Pallas}}}$, and a Pedersen commitment to such a vector needs a group whose scalar field is $\mathbb{F}_{p_{\text{Pallas}}}$: that group is the Vesta curve, whose order is p_{Pallas} (protocol specification, §5.4.9.6, “Pallas and Vesta”). The specification uses Halo 2 with the Vesta curve to prove and verify the Action statement (§4.1.13, “Zero-Knowledge Proving System”), so every P, L_j, R_j, S and $G^{(0)}$ of this section is a Vesta point and every scalar a Pallas base-field element. The other half of the Pasta cycle, Pallas’s scalar field being Vesta’s base field, is needed only by the optional native recursion of §7; Orchard’s verifier never uses it. The committed polynomials have degree below the row count $n = 2^{11} = 2048$ of the Action circuit (Definition 2.7; the count is given under “From statement to circuit: arithmetisation in practice” in §4), so here $d = n = 2^{11}$ and the argument runs $k = 11$ rounds.

Generators from a hash. The 2048 vector generators, the blinding generator H and the inner-product generator U are all outputs of hash-to-curve on one fixed domain-separation label, the vector generators indexed by their position and H and U by two further tags (*Crypto Guide*, §“Hashing to a curve point”, Construction “Nothing-up-my-sleeve generators”). With hash-to-curve modelled as a random oracle into $\mathbb{G}$ they are independent and uniform, and a relation among them yields a discrete logarithm on Vesta (the same section, Remark “Why NUMS generators are binding-safe”). No secret is generated in producing them, unlike the trapdoor of KZG (*Crypto Guide*, §“Polynomial commitment schemes”, under “KZG: evaluation in the exponent”), the point the closing subsubsection returns to. Because U is a fixed parameter rather than a verifier’s sample drawn after each P , the deployed opening re-randomises it per opening with a challenge ζ (“The deployed opening in full” below).

Challenges from the transcript. The interactive verifier of Construction 3.2 draws u_j uniformly from $\mathbb{F}^\times$, and the halo2 book’s *Protocol Description* excludes zero challenges. The deployed verifier is a hash (“Removing the verifier: the Fiat–Shamir transform” in §4), and the transcript of the halo2_proofs implementation squeezes every challenge, ξ, ζ and each u_j , as a full field element with no rejection of zero. A zero u_j would make the fold’s inversion fail, so the honest prover succeeds unless one of the k challenges is zero, an event of probability $1/|\mathbb{F}|$ per challenge, about 2^{-254} : completeness of the deployed argument is overwhelming, not literally perfect, in the sense fixed in §1.2.

Remark 3.9 (The deployed IPA normalisation). The deployed opening, stated in full below, is the masked opening of §3.6 with three changes. It folds with the weights $(1, u_j^{-1})$ on the coefficient vector and $(1, u_j)$ on $\mathbf{b}$ and $\mathbf{G}$, with cross terms $L_j = \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + \dots$ and $R_j = \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + \dots$, the halves swapped relative to (13), and the verifier’s accumulation $[u_j^{-1}]L_j + [u_j]R_j$; it scales the cross terms’ U -coordinates by a further challenge ζ drawn after ξ ; and it moves the claimed value into the commitment as $-[v]G_0$ in place of $+ [v]U$. Its final scalars are c and f , and its check is (29). The halo2 book, which the protocol specification (§ 5.4.10.3, “Halo 2”) cites for the proving system, states this opening in its chapter *Protocol Description*. Its blinding generator W is this volume’s H , its U is U , and its second challenge z is ζ , renamed because $z = 20$ is the running example’s opening point; the book’s comparison with the scheme of Bünz, Chiesa, Mishra and Spooner (chapter *Comparison to other work*) writes H , U and z for the same three objects. The chapter numbers the round challenges from the first round, $u_0, \dots, u_{k-1}$, where this volume’s descending index gives $u_k, \dots, u_1$, and states the verifier’s evaluation polynomial (its κ) as

$$\begin{aligned} g_{\text{dep}}(X) &= \prod_{i=0}^{k-1} (1 + u_{k-1-i} X^{2^i}) \quad \text{in its numbering,} \\ g_{\text{dep}}(X) &= \prod_{\ell=1}^k (1 + u_{\ell} X^{2^{\ell-1}}) \quad \text{in this volume’s,} \end{aligned} \tag{26}$$

with structured scalars $s_i = \prod_{\ell: \beta_{\ell-1}=1} u_{\ell}$, beside the symmetric $g(X) = \prod_{\ell=1}^k (u_{\ell}^{-1} + u_{\ell} X^{2^{\ell-1}})$ of (19) with $s_i = \prod_{\ell} u_{\ell}^{(-1)^{1-\beta_{\ell-1}}}$.

The change of variables. The two folds are one argument. Run both on the same vector, the same generators and the same evaluation vector, with the deployed challenge of every round the square of the symmetric one, $u_{\text{dep}} := u_{\text{sym}}^2$. In one round, writing u for u_{sym} ,

$$\begin{aligned} \mathbf{a}'_{\text{sym}} &= u \mathbf{a}_{\text{lo}} + u^{-1} \mathbf{a}_{\text{hi}} = u (\mathbf{a}_{\text{lo}} + u^{-2} \mathbf{a}_{\text{hi}}) = u \mathbf{a}'_{\text{dep}}, \\ \mathbf{G}'_{\text{sym}} &= u^{-1} \mathbf{G}_{\text{lo}} + u \mathbf{G}_{\text{hi}} = u^{-1} (\mathbf{G}_{\text{lo}} + u^2 \mathbf{G}_{\text{hi}}) = u^{-1} \mathbf{G}'_{\text{dep}}, \\ \mathbf{b}'_{\text{sym}} &= u^{-1} \mathbf{b}_{\text{lo}} + u \mathbf{b}_{\text{hi}} = u^{-1} (\mathbf{b}_{\text{lo}} + u^2 \mathbf{b}_{\text{hi}}) = u^{-1} \mathbf{b}'_{\text{dep}}, \end{aligned}$$

so $\mathbf{a}'_{\text{dep}} = u^{-1} \mathbf{a}'_{\text{sym}}$, $\mathbf{G}'_{\text{dep}} = u \mathbf{G}'_{\text{sym}}$ and $\mathbf{b}'_{\text{dep}} = u \mathbf{b}'_{\text{sym}}$, and the scalings cancel in every inner product: $\langle \mathbf{a}'_{\text{dep}}, \mathbf{G}'_{\text{dep}} \rangle = \langle \mathbf{a}'_{\text{sym}}, \mathbf{G}'_{\text{sym}} \rangle$ and $\langle \mathbf{a}'_{\text{dep}}, \mathbf{b}'_{\text{dep}} \rangle = \langle \mathbf{a}'_{\text{sym}}, \mathbf{b}'_{\text{sym}} \rangle$. The cross terms agree, up to their H -blindings, once the naming is unwound: the deployed L carries $\langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle$, which the symmetric presentation calls R , and its weight $u_{\text{dep}}^{-1} = u^{-2}$ is the symmetric weight on R ; likewise for the other cross term. The single-round form holds for one round on identical inputs; over several rounds the change of variables is *cumulative*, because each round’s inputs already carry the previous rounds’ factors:

$$\mathbf{a}_{\text{dep}}^{(j)} = \left(\prod_{\ell > j} u_{\ell} \right)^{-1} \mathbf{a}_{\text{sym}}^{(j)}, \quad \mathbf{G}_{\text{dep}}^{(j)} = \left(\prod_{\ell > j} u_{\ell} \right) \mathbf{G}_{\text{sym}}^{(j)}, \quad \mathbf{b}_{\text{dep}}^{(j)} = \left(\prod_{\ell > j} u_{\ell} \right) \mathbf{b}_{\text{sym}}^{(j)}, \tag{27}$$

with u_{ℓ} the symmetric challenges of the rounds already run. Each deployed factor of (26) is a symmetric factor scaled, $1 + u^2 X^m = u (u^{-1} + u X^m)$, so after k rounds

$$g_{\text{dep}} = \left(\prod_{\ell=1}^k u_{\ell} \right) g_{\text{sym}}, \quad s_{\text{dep}} = \left(\prod_{\ell=1}^k u_{\ell} \right) s_{\text{sym}}, \quad a_{\text{dep}}^{(0)} = \left(\prod_{\ell=1}^k u_{\ell} \right)^{-1} a_{\text{sym}}^{(0)},$$

while $G^{(0)}$ and $b^{(0)}$ carry the product itself, so that $[a^{(0)}]G^{(0)}$ and $a^{(0)}b^{(0)}$ agree in both normalisations, and the two verifier equations coincide once each run attaches ℓ_j and ρ_j to the same cross product. Table 6 prints both runs on the toy’s data and keeps the blinder names instead, so the deployed run attaches each blinder to the other cross product, and its equation differs from the symmetric one only in the H -term ($r^* = 33$ against 40); with the blinders exchanged, the deployed run also closes at $(54, 39)$ with $r^* = 40$. What the substitution does *not* do is identify the challenge distributions: a uniform deployed challenge is not the square of a uniform symmetric one (“Extraction in the deployed rows” below), so the deployed rows are analysed with their own lemma, and the deployed opening is knowledge sound by the argument of §3.5 with Lemma 3.10 at every node and two branchings, over ξ and ζ , in place of the resampled U at the root: Corollary 3.13.

Table 6. The two folds on the same $\mathbf{a} = (90, 61, 22, 24)$, the same generators and the same $\mathbf{b}(20)$, with $u_{\text{dep}} = u_{\text{sym}}^2$ in every round and the same round blinders. The last column is the cumulative factor of (27): after round 2 it is 52, after round 1 it is $52 \cdot 20 = 70$ in $\mathbb{F}_{97}$. The cross terms differ as points because the deployed naming attaches each blinder to the other cross product; every folded quantity differs by the stated factor only.

	symmetric fold	deployed fold	relation
round 2: challenge	$u_2 = 52$	$u_2 = 52^2 = 85$	
cross terms sent	$L_2 = (74, 6), R_2 = (38, 72)$	$L_2 = (78, 70), R_2 = (71, 26)$	halves swapped
$\mathbf{a}^{(1)}$	$(58, 61)$	$(72, 59)$	$52^{-1} \cdot$
$\mathbf{G}^{(1)}$	$((54, 40), (30, 71))$	$((53, 53), (41, 6))$	$52 \cdot$
$\mathbf{b}^{(1)}$	$(70, 42)$	$(51, 50)$	$52 \cdot$
round 1: challenge	$u_1 = 20$	$u_1 = 20^2 = 12$	
cross terms sent	$L_1 = (28, 25), R_1 = (39, 25)$	$L_1 = (43, 6), R_1 = (70, 71)$	halves swapped
$\mathbf{a}^{(0)}$	33	85	$(52 \cdot 20)^{-1} = 70^{-1} \cdot$
$\mathbf{G}^{(0)}$	$(12, 25)$	$(34, 53)$	$70 \cdot$
$\mathbf{b}^{(0)}$	19	69	$70 \cdot$
structured scalars $\mathbf{s}$	$(79, 75, 22, 70)$	$(1, 12, 85, 50)$	$70 \cdot$
$g(20)$	$(u_1^{-1} + u_1 z)(u_2^{-1} + u_2 z^2) = 19$	$(1 + u_1 z)(1 + u_2 z^2) = 69$	$70 \cdot$
aggregated blinder r^*	40	33	
verifier’s equation	$(54, 39) = (54, 39)$	$(5, 72) = (5, 72)$	same $[a^{(0)}]G^{(0)}, a^{(0)}b^{(0)}$

Extraction in the deployed rows. The deployed fold changes the rows of the extractor’s linear system.

Lemma 3.10 (Single-round recovery in the deployed rows). *Fix one round at level j as in Lemma 3.4, but let the round fold the coefficient vector with the weights $(1, u^{-1})$ and the generator and evaluation vectors with the weights $(1, u)$,*

$$\mathbf{a}' := \mathbf{a}_{\text{lo}} + u^{-1}\mathbf{a}_{\text{hi}}, \quad \mathbf{G}' := \mathbf{G}_{\text{lo}} + u\mathbf{G}_{\text{hi}}, \quad \mathbf{b}' := \mathbf{b}_{\text{lo}} + u\mathbf{b}_{\text{hi}},$$

with the cross terms named with the halves swapped relative to (13),

$$L := \langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle + [\ell]H + [\langle \mathbf{a}_{\text{hi}}, \mathbf{b}_{\text{lo}} \rangle] U, \quad R := \langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle + [\rho]H + [\langle \mathbf{a}_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle] U,$$

and the folded commitment $C' := C + [u^{-1}]L + [u]R$, so that the fold identity reads $C' = \langle \mathbf{a}', \mathbf{G}' \rangle + [r + u^{-1}\ell + u\rho]H + [\langle \mathbf{a}', \mathbf{b}' \rangle]U$ by the same expansion as in §3.2. Then the conclusions

of Lemma 3.4 hold with rows $(\nu^{-1}, 1, \nu)$ and the identities $\mathbf{x}_\nu = \nu^{-1}\mathbf{x}_L + \mathbf{x}_C + \nu\mathbf{x}_R$ (likewise for y, w), provided only that u, u', u'' are pairwise distinct and nonzero. The deployed cross terms also scale their U -coordinates by a challenge $\zeta \neq 0$ drawn before the rounds (“The deployed opening in full” below), which the lemma covers verbatim with U replaced by $[\zeta]U$.

Proof. The fold identity is the expansion of §3.2 with the weights changed: the diagonal terms of $\langle \mathbf{a}_{\text{lo}} + u^{-1}\mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} + u\mathbf{G}_{\text{hi}} \rangle$ carry the weights $1 \cdot 1$ and $u^{-1}u = 1$, and the cross terms $\langle \mathbf{a}_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle$ and $\langle \mathbf{a}_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle$ carry u^{-1} and u , which is why they are absorbed as $[u^{-1}]L + [u]R$ with L holding the high-with-low product. The lift (20) becomes $\mathbf{x} = (\mathbf{a}', u\mathbf{a}')$, and preserves well-formedness since $\langle (\mathbf{a}', u\mathbf{a}'), \mathbf{b} \rangle = \langle \mathbf{a}', \mathbf{b}_{\text{lo}} + u\mathbf{b}_{\text{hi}} \rangle$. The matrix now has rows $(\nu^{-1}, 1, \nu)$; multiplying the row of ν by $\nu \neq 0$ gives $(1, \nu, \nu^2)$, and the same root count, now for a polynomial of degree at most 2 in ν itself, shows that the matrix is invertible exactly when the challenges are pairwise distinct. Everything else is as in the proof of Lemma 3.4. $\square$

The two row shapes impose different hypotheses: two challenges $u' = -u$ have equal squares, so a triple containing them makes the symmetric matrix singular, while the deployed matrix remains invertible whenever the three challenges are distinct and nonzero. Over $\mathbb{F}_{97}$ the triple $(3, 94, 5)$, in which $94 = -3$, has squares $(9, 9, 25)$: the symmetric rows are $(9, 1, 54)$, $(9, 1, 54)$ and $(25, 1, 66)$, two of them equal, with determinant 0; the deployed rows are $(65, 1, 3)$, $(32, 1, 94)$ and $(39, 1, 5)$, with determinant $28 \neq 0$. Remark 3.9 shows that the deployed fold is the symmetric fold under the substitution $u_{\text{dep}} = u_{\text{sym}}^2$, but that relabelling does not carry the hypotheses across: the squaring map $\mathbb{F}^\times \rightarrow \mathbb{F}^\times$ is two-to-one onto the squares, so a uniform deployed challenge is not the square of a uniform symmetric one, and the condition “pairwise distinct squares” is a condition on three symmetric challenges that has no counterpart for three uniform deployed ones. Each shape is therefore analysed with its own rows. In the proof of Theorem 3.5 with Lemma 3.10 in place of Lemma 3.4, three pairwise distinct nonzero challenges suffice at each node, and each sibling avoids the earlier siblings’ challenges and 0.

Example 3.11 (The extractor in the deployed rows). On the data of Example 3.6, the extractor with rows $(u^{-1}, 1, u)$ and the fold of Lemma 3.10 branches over its own challenges: $u_2 \in \{78, 83, 56\}$ at the root, and under them the round-1 triples $\{31, 71, 92\}$, $\{91, 50, 47\}$ and $\{95, 14, 96\}$. The node systems have determinants 70, 41 and 33 and the root system 54, and the extractor recovers the same $\mathbf{a} = (90, 61, 22, 24)$, $r = 88$ and U -coordinate $59 = v$.

The deployed opening in full. The deployed opening is the masked opening of §3.6 with the deployed fold, a further challenge ζ , and the claimed value moved into the commitment. Let $\mathbf{e}_0 := (1, 0, \dots, 0)$, and write $\mathbf{t}$ for the coefficient-side vector and $\mathbf{s}$ for the structured scalars of the deployed fold throughout the rest of this subsection.

1. **Mask.** The prover samples $s(X)$ with $s(x) = 0$ and r_i , and sends $S := \text{Commit}(s; r_i) = \langle \mathbf{m}, \mathbf{G} \rangle + [r_i]H$, as in §3.6.
2. **Two challenges.** The verifier sends a uniform ξ , and then a uniform ζ .

3. **The masked commitment.** Both parties replace P by

$$P' := P + [\xi]S - [v]G_0, \quad (28)$$

which, by linearity of the commitment (*Crypto Guide*, §“Pedersen vector commitments”, the remark that linearity survives), commits with blinder $r + \xi r_s$ to the coefficient vector

$$\mathbf{a}' := \mathbf{a} + \xi \mathbf{m} - v \mathbf{e}_0$$

of the polynomial $p + \xi s - v$, whose value at x is $p(x) + \xi s(x) - v = 0$. The claim “ $p(x) = v$ ” has become the claim “ P' commits to a vector whose inner product with $\mathbf{b}(x)$ is 0”: the value is folded into the commitment, and the opening claims the value 0.

4. **Rounds.** The parties run the k rounds on $(\mathbf{a}', \mathbf{G}, \mathbf{b}(x))$ with the fold of Lemma 3.10, weights $(1, u_j^{-1})$ on the coefficients and $(1, u_j)$ on generators and evaluation vector, and with the U -coordinates of the cross terms scaled by ζ :

$$\begin{aligned} L_j &:= \langle \mathbf{a}'_{\text{hi}}^{(j)}, \mathbf{G}_{\text{lo}}^{(j)} \rangle + [\ell_j]H + [\zeta \langle \mathbf{a}'_{\text{hi}}^{(j)}, \mathbf{b}_{\text{lo}}^{(j)} \rangle] U, \\ R_j &:= \langle \mathbf{a}'_{\text{lo}}^{(j)}, \mathbf{G}_{\text{hi}}^{(j)} \rangle + [\rho_j]H + [\zeta \langle \mathbf{a}'_{\text{lo}}^{(j)}, \mathbf{b}_{\text{hi}}^{(j)} \rangle] U; \end{aligned}$$

the verifier accumulates $[u_j^{-1}]L_j + [u_j]R_j$, and the prover’s blinder accumulates $f := r + \xi r_s + \sum_j (u_j^{-1} \ell_j + u_j \rho_j)$.

5. **Final message and check.** The prover sends the final scalars $c := \mathbf{a}'^{(0)}$ and f . The verifier accepts if and only if

$$P + [\xi]S - [v]G_0 + \sum_{j=1}^k ([u_j^{-1}]L_j + [u_j]R_j) \stackrel{?}{=} [c]G^{(0)} + [c \cdot b^{(0)} \cdot \zeta]U + [f]H, \quad (29)$$

where $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$ and $b^{(0)} = \langle \mathbf{s}, \mathbf{b}(x) \rangle = g_{\text{dep}}(x)$ for the structured scalars $s_i = \prod_{\ell: \beta_{\ell-1}=1} u_\ell$ of (26).

Completeness is the invariant (16) again, with $\mathbf{a}'$ in place of $\mathbf{a}$, the value 0 in place of v , and every U -coordinate scaled by ζ ; the final U -coordinate is $\zeta \langle \mathbf{a}'^{(0)}, \mathbf{b}^{(0)} \rangle = c b^{(0)} \zeta$. Subtracting $[v]G_0$ moves the claimed value into the commitment, so that the argument opens to 0 and the check carries no term in v on U ; the mask leaves the opened value unchanged in either form because $s(x) = 0$. The halo2 book gives the reason in its comparison with the scheme of Bünz, Chiesa, Mishra and Spooner, which adds the value on the inner-product generator instead (chapter *Comparison to other work*): G_0 is a fixed base, so $[v]G_0$ is the cheaper multiplication inside a recursive verifier. The factor ζ has a reason of its own. The deployed U is a fixed public generator rather than a verifier’s sample drawn after P (“Generators from a hash” above): a prover who knew U in advance could plant a U -component in P or S , and drawing ζ after both re-randomises the generator that carries the inner product, so that a planted component would have to have anticipated ζ ; Corollary 3.13 makes this precise.

Hiding in the deployed opening. The final scalar is the linear form of (22) for this fold, taken of the masked vector:

$$c = \langle \mathbf{a} + \xi \mathbf{m} - v \mathbf{e}_0, \mathbf{t} \rangle = \langle \mathbf{a} - v \mathbf{e}_0, \mathbf{t} \rangle + \xi \langle \mathbf{m}, \mathbf{t} \rangle, \quad t_i = \prod_{\ell: \beta_{\ell-1}=1} u_\ell^{-1}, \quad (30)$$

the coefficient side folding with the inverse weights of the generator side as always. Unmasked, the deployed fold leaks this form as the symmetric one does (Remark 3.7): four deployed openings of (90, 61, 22, 24) without the mask, with challenge pairs $(u_2, u_1) = (16, 85), (8, 16), (69, 95)$ and $(11, 41)$, end in the scalars 70, 24, 73 and 63, and solving the 4×4 system recovers the vector. Masked, with $\mathbf{m} = (66, 23, 23, 88)$, $\xi = 29$ and the challenges $u_2 = 85, u_1 = 12$, the masked vector is $(5, 49, 10, 54)$, its inner product with $\mathbf{b}(20)$ is 0, and the final scalar is $c = 20 = \langle (5, 49, 10, 54), \mathbf{t} \rangle$ with $\mathbf{t} = (1, 89, 8, 33)$; a second mask $\mathbf{m}' = (32, 68, 64, 88)$ under the same challenges ends in $c' = 46$, and both runs are accepted. The transcript of one opening is

$$(S; L_k, R_k, \dots, L_1, R_1; c, f) \quad \text{with the challenges} \quad (\xi, \zeta, u_k, \dots, u_1).$$

Corollary 3.12 (Honest-verifier zero knowledge of the deployed opening). *Let $\mathbb{G}$ have prime order p , and let the verifier draw ξ, ζ uniformly from $\mathbb{F}$ and each u_j uniformly from $\mathbb{F}^\times$. There is a simulator which, given (P, x, v) but not $(\mathbf{a}, r)$, outputs challenges and a transcript of the deployed opening whose distribution lies within statistical distance $1/|\mathbb{F}| + (|\mathbb{F}| - 1)^{-k}$ of the honest one.*

Proof. The proof of Proposition 3.8, with the deployed $\mathbf{t}$ and with (29) in place of (24). Call a challenge tuple *good* if $\xi \neq 0$ and $\mathbf{t}$ is not a multiple of $\mathbf{b}(x)$; now $\mathbf{t} = \lambda \mathbf{b}(x)$ forces $\lambda = t_0 = 1$ and then $u_\ell^{-1} = x^{2^{\ell-1}}$ for every ℓ , at most one challenge tuple among $(|\mathbb{F}| - 1)^k$. For good challenges c is uniform and independent of $\mathbf{a}$ by (30) and the argument of that proof. The simulator draws the challenges, S , every L_j and every R_j but R_1 , and c and f uniformly, and sets R_1 to the one point that satisfies (29), namely $R_1 := [u_1^{-1}](\text{right-hand side} - \text{every other term of the left})$; the counting of that proof shows that the two distributions agree for good challenges, and the tuples that are not good have probability at most $1/|\mathbb{F}| + (|\mathbb{F}| - 1)^{-k}$. $\square$

The deployed opening of the toy polynomial. The toy's P of §3.7 is opened at $z = 20$ with the deployed opening, using the round challenges $u_2 = 85$ and $u_1 = 12$ of Table 6; Table 7 lists both sides with every intermediate value. The verifier's equation (29) closes at $(5, 72)$ on both sides, with $g(20) = (1 + 12 \cdot 20)(1 + 85 \cdot 20^2) = 69$ and $\mathbf{s} = (1, 12, 85, 12 \cdot 85) = (1, 12, 85, 50)$ in the deployed shape, and the false value $v' = 60$ moves the left-hand side to $(67, 31)$ against $(5, 72)$ on the right: rejected once more. The deployed run and the masked symmetric run of §3.7 open the same commitment to the same value with entirely different numbers; Remark 3.9 relates the two folds. The unmasked deployed run of Table 6 and this masked opening both close at $(5, 72)$, and that run's $R_1 = (70, 71)$ equals the mask commitment S : further collisions of the kind noted in §3.7, accidents of a 97-element group, not consequences of the mask.

Corollary 3.13 (Knowledge soundness of the deployed opening). *Let $\mathbb{G}$, $\mathbf{G}$ and H be as in Theorem 3.5, and let U be a fixed generator, uniform in $\mathbb{G} \setminus \{\mathcal{O}\}$ like them. The deployed opening, with the check (29), is knowledge sound for $\mathcal{R}_{\text{IPA}}$: there is an extractor which, given rewindable*

Table 7. The deployed opening of the same $P = (39, 25)$ at $z = 20$ with the deployed mask: the fold of Lemma 3.10, the cross terms named with the halves swapped, the U -coordinates scaled by ζ , the value 0 opened for $P + [\xi]S - [v]G_0$, and the final scalars c and f . The round challenges are those of Table 6.

step	prover	verifier
mask	samples $\mathbf{m} = (66, 23, 23, 88)$, for which $s(20) = 0$, and $r_j^* = 95$; sends $S = \langle \mathbf{m}, \mathbf{G} \rangle + [95]H = (70, 71)$	receives S ; sends $\xi = 29$, then $\zeta = 12$
masked vector	$\mathbf{a}' = \mathbf{a} + 29\mathbf{m} - 59\mathbf{e}_0 = (5, 49, 10, 54)$; $\langle \mathbf{a}', \mathbf{b}(20) \rangle = 0$; blinder $f := 10 + 29 \cdot 95$	$P' := P + [29]S - [59]G_0$
round 2	halves $\mathbf{a}'_{\text{lo}} = (5, 49)$, $\mathbf{a}'_{\text{hi}} = (10, 54)$; $\ell_2 = 44$, $\rho_2 = 92$; cross terms $\langle \mathbf{a}'_{\text{hi}}, \mathbf{G}_{\text{lo}} \rangle = [10]G_0 + [54]G_1 = (34, 53)$, $\langle \mathbf{a}'_{\text{lo}}, \mathbf{b}_{\text{lo}} \rangle = 23$, $\langle \mathbf{a}'_{\text{lo}}, \mathbf{G}_{\text{hi}} \rangle = [5]G_2 + [49]G_3 = (54, 39)$, $\langle \mathbf{a}'_{\text{lo}}, \mathbf{b}_{\text{hi}} \rangle = 83$; sends $L_2 = (34, 53) + [44]H + [12 \cdot 23]U = (17, 52)$, $R_2 = (54, 39) + [92]H + [12 \cdot 83]U = (47, 40)$ folds $\mathbf{a}^{(1)} = (5, 49) + 8(10, 54) = (85, 93)$, $\mathbf{b}^{(1)} = (1, 20) + 85(12, 46) = (51, 50)$, $\mathbf{G}^{(1)} = (G_0 + [85]G_2, G_1 + [85]G_3) = ((53, 53), (41, 6))$; $f = 10 + 29 \cdot 95 + 8 \cdot 44 + 85 \cdot 92 = 73$	sends $u_2 = 85(85^{-1} = 8)$; keeps $(L_2, R_2, 85)$; $P^{(1)} := P' + [8]L_2 + [85]R_2 = (64, 5)$
round 1	halves $a'_{\text{lo}} = 85$, $a'_{\text{hi}} = 93$; $\ell_1 = 95$, $\rho_1 = 56$; cross terms $[93]G_0^{(1)} = (58, 8)$, $93 \cdot 51 = 87$, $[85]G_1^{(1)} = (24, 9)$, $85 \cdot 50 = 79$; sends $L_1 = (58, 8) + [95]H + [12 \cdot 87]U = (46, 13)$, $R_1 = (24, 9) + [56]H + [12 \cdot 79]U = (24, 70)$ folds $c = a'^{(0)} = 85 + 89 \cdot 93 = 20$, $b^{(0)} = 51 + 12 \cdot 50 = 69$, $G^{(0)} = G_0^{(1)} + [12]G_1^{(1)} = (34, 53)$; $f = 73 + 89 \cdot 95 + 12 \cdot 56 = 82$	sends $u_1 = 12(12^{-1} = 89)$; keeps $(L_1, R_1, 12)$; $P^{(0)} := P^{(1)} + [89]L_1 + [12]R_1 = (5, 72)$
final	sends $c = \mathbf{a}'^{(0)} = 20$, $f = 82$	$g(20) = (1 + 12 \cdot 20)(1 + 85 \cdot 20^2) = 69$; $\mathbf{s} = (1, 12, 85, 50)$, $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = (34, 53)$; left: $P' + [85^{-1}]L_2 + [85]R_2 + [12^{-1}]L_1 + [12]R_1 = (5, 72)$; right: $[20]G^{(0)} + [20 \cdot 69 \cdot 12]U + [82]H = (5, 72)$; accept ; with $v' = 60$: left $(67, 31) \neq (5, 72)$: reject

access to any prover that makes the verifier accept with non-negligible probability, runs in expected polynomial time and outputs either a witness $(\mathbf{a}, r)$ for (P, x, v) or a nontrivial discrete-logarithm relation among $\mathbf{G}$, H and U . Its knowledge error is $O(\log d / |\mathbb{F}|)$.

Proof. The extractor branches over two values $\xi_1 \neq \xi_2$ after S , under each over two nonzero values $\zeta_1 \neq \zeta_2$, and under each of those over the rounds as in the proof of Theorem 3.5, with three pairwise distinct nonzero challenges per node; the tree has $4 \cdot 3^k$ leaves, and the rewinding and running time are as in that proof.

Rounds. Fix ξ and $\zeta \neq 0$, and write $U_\zeta := [\zeta]U$. With U_ζ in place of U the rounds are those of Lemma 3.10, so the induction in the proof of Theorem 3.5 returns a representation $(\mathbf{x}; y; w)$ of $P' = P + [\xi]S - [v]G_0$ over $(\mathbf{G}, H, U_\zeta)$ with $w = \langle \mathbf{x}, \mathbf{b}(x) \rangle$, that is, the representation $(\mathbf{x}; y; \zeta w)$ over $(\mathbf{G}, H, U)$.

The challenge ζ . Under one ξ , the values ζ_1 and ζ_2 give representations $(\mathbf{x}_1; y_1; \zeta_1 w_1)$ and $(\mathbf{x}_2; y_2; \zeta_2 w_2)$ of the same P' over $(\mathbf{G}, H, U)$. If they differ, their difference is a nontrivial relation among $\mathbf{G}$, H and U . Otherwise $\mathbf{x}_1 = \mathbf{x}_2 =: \mathbf{x}$, so $w_1 = w_2 =: w$ and $(\zeta_1 - \zeta_2)w = 0$, whence $w = 0$: $P' = \langle \mathbf{x}, \mathbf{G} \rangle + [y]H$ with $\langle \mathbf{x}, \mathbf{b}(x) \rangle = 0$.

The challenge ξ . The values ξ_1 and ξ_2 follow the same S and give $P + [\xi_i]S - [v]G_0 = \langle \mathbf{x}_i, \mathbf{G} \rangle + [y_i]H$ with $\langle \mathbf{x}_i, \mathbf{b}(x) \rangle = 0$ for $i = 1, 2$. Subtracting, $S = \langle \mathbf{m}, \mathbf{G} \rangle + [r]H$ with $\mathbf{m} := (\mathbf{x}_1 - \mathbf{x}_2) / (\xi_1 - \xi_2)$, $r := (y_1 - y_2) / (\xi_1 - \xi_2)$ and $\langle \mathbf{m}, \mathbf{b}(x) \rangle = 0$. Substituting back, $P = \langle \mathbf{a}, \mathbf{G} \rangle + [r]H$ with $\mathbf{a} := \mathbf{x}_1 - \xi_1 \mathbf{m} + v \mathbf{e}_0$ and $r := y_1 - \xi_1 r$, and $\langle \mathbf{a}, \mathbf{b}(x) \rangle = 0 - 0 + v \cdot 1 = v$, the first entry of $\mathbf{b}(x)$ being 1. This is the witness.

The error. Beyond the $O(\log d / |\mathbb{F}|)$ of the rounds, the two branchings at the root and the exclusion of $\zeta = 0$ each cost $O(1 / |\mathbb{F}|)$. A relation among $\mathbf{G}$, H and U yields a discrete logarithm by the every-generator embedding, as at the root of Theorem 3.5. $\square$

The bytes. An Orchard proof is a byte string. The proofs of the $a \geq 1$ Action descriptions that one transaction carries in one pool, here called a *bundle*, are aggregated into a single proof (protocol specification, § 4.6, “Action Descriptions”); a transaction with Actions in both the Orchard and the Ironwood pool carries one such proof for each. Its length is $2720 + 2272a$ bytes (ZIP 225, the `sizeProofsOrchard` field; ZIP 229, the fields `sizeProofsOrchard` and `sizeProofsIronwood`; the consensus rule is the protocol specification’s, § 7.5, “Action Description Encoding and Consensus”), 4992 bytes for a single Action. Every Vesta point and every field element in it occupies 32 bytes. Table 8 lists by kind, from the per-object counts of the Action circuit whose verifying key the consensus rules fix (protocol specification, § 4.6, “Action Descriptions”), every one of those bytes: a per-Action block of 22 points and 49 scalars, repeated for each Action, and a shared block of 33 points and 52 scalars. The inner-product argument’s share is the last three rows of the shared block: the blinding commitment S , the $k = 11$ pairs (L_j, R_j) , which are 22 points, and the two scalars c and f : 23 points, 736 bytes, or 800 bytes with the scalars, once per bundle, because the complete protocol collapses every opening of every polynomial into one opening (“The multi-point opening argument” in §4). This is the only part of the proof that grows with n , and it grows as $2 \log_2 n + 1$ points. Everything else grows with the circuit’s structure and not with n : one commitment per advice column, two permuted-column commitments and one running product per

lookup, the permutation running products and quotient chunks, the multipoint commitment, and one field element per claimed evaluation.

Table 8. The bytes of an Orchard proof, $2720 + 2272a$ for a Actions (ZIP 225, ZIP 229), laid out as points and scalars of 32 bytes each. The three rows marked **IPA** are this section’s: 23 points and 2 scalars, 800 bytes, shared by the whole bundle. The advice, permutation, lookup and quotient rows are the objects of §2; the random polynomial is constructed under “The complete Halo 2 protocol” and the multipoint polynomial under “The multipoint opening argument”, both in §4.

kind	count	bytes	what it is
<i>Per-Action block, once per Action</i>			
point	10	320	advice column commitments, one per advice column
point	3	96	permuted lookup input commitments A' , one per lookup (Construction 2.14)
point	3	96	permuted lookup table commitments S' , one per lookup (Construction 2.14; unrelated to the mask commitment S)
point	3	96	permutation running-product commitments: the 15 equality-enabled columns in chunks of 7 (§2.6)
point	3	96	lookup running-product commitments, one per lookup
scalar	1	32	instance column evaluation at x
scalar	25	800	$25 = 5 \cdot 2 + 5 \cdot 3$ advice evaluations, one per queried rotation of each advice column (§4.4)
scalar	8	256	permutation product evaluations: 3 products at x and ωx , the first 2 also at $\omega^{-6}x$, the rotation to the boundary row $n - n_{\text{blind}} - 1$ of Remark 2.17, with $n_{\text{blind}} = 5$ blinding rows (§4.6)
scalar	15	480	lookup evaluations: product at x and ωx ; A' at x and $\omega^{-1}x$; S' at x
total		2272	22 points and 49 scalars
<i>Shared block, once per bundle</i>			
point	1	32	random polynomial commitment: a random polynomial added to blind the quotient (the complete protocol, §4)
point	8	256	quotient chunk commitments: constraint degree $d_{\text{max}} = 9$, so $d_{\text{max}} - 1 = 8$ chunks of degree below n (Theorem 4.7)
scalar	29	928	fixed column evaluations at x , one per fixed column
scalar	15	480	permuted-label polynomial evaluations at x , one per equality-enabled column (Construction 2.10)
scalar	1	32	random polynomial evaluation at x
point	1	32	multipoint polynomial commitment: the one polynomial into which every query collapses (Construction 4.4)
scalar	5	160	multipoint evaluations: the value of each query group’s combined polynomial at the reduction’s fresh challenge point, one per distinct set of query points; five sets (§4.4)
point	1	32	IPA : the blinding commitment S
point	22	704	IPA : the $k = 11$ pairs (L_j, R_j)
scalar	2	64	IPA : the final scalars c and f
total		2720	33 points and 52 scalars

3.8.1 The “Halo” in Halo 2: deferring the expensive check. The verifier of this section is logarithmic everywhere but in one place. He reads $2k + 1$ points and two scalars, evaluates g in $3k - 2$ field multiplications, and forms the left-hand side of (29) with $2k + 2$ scalar multiplications ($[\xi]S$, $[v]G_0$ and the $2k$ round terms); then he needs $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$, one combination of *all* 2048 generators, group arithmetic proportional to the size of the whole circuit (§3.4). Two ways

of amortising that cost follow, batch verification and accumulation, the second not deployed by Orchard; between them, the paragraph “Transparency: no trusted setup” records why the linear cost is accepted.

Batch verification. The consensus rules require each Action proof to be valid for the verifying key of the Orchard circuit (protocol specification, § 4.6, “Action Descriptions”); validity includes the check (29) with its multi-scalar multiplication of length d , and the specification composes no proofs, so the verifier pays that multiplication itself rather than passing it on. Several proofs can share it (*Crypto Guide*, § “Polynomial commitment schemes”, Remark “KZG versus IPA, and the Halo lineage”). Written as one multi-scalar multiplication that must evaluate to $\mathcal{O}$, equation (29) has $d + 2k + 4$ terms: the generators $G_0, \dots, G_{d-1}, U, H, P, S$ and the $2k$ points L_j, R_j . The equations $E_1, \dots, E_b$ of b proofs are checked together by testing $\sum_i [\mu_i]E_i = \mathcal{O}$ for fresh uniform $\mu_i \in \mathbb{F}$ drawn by the verifier.

Lemma 3.14 (Randomised batch check). *Let $\mathbb{G}$ have prime order, let $E_1, \dots, E_b \in \mathbb{G}$, and let $\mu_1, \dots, \mu_b \in \mathbb{F}$ be independent and uniform. If some $E_i \neq \mathcal{O}$, then $\Pr[\sum_j [\mu_j]E_j = \mathcal{O}] = 1/|\mathbb{F}|$.*

Proof. Fix i with $E_i \neq \mathcal{O}$ and condition on every μ_j with $j \neq i$. Since $\mathbb{G}$ has prime order, E_i generates it, so $\mu \mapsto [\mu]E_i$ is a bijection $\mathbb{F} \rightarrow \mathbb{G}$, and the sum equals $\mathcal{O}$ for exactly one value of μ_i . $\square$

Because $\mathbf{G}$, U and H are shared, their terms merge, and b equations combine into one multi-scalar multiplication of $d + 2 + b(2k + 2)$ terms, against $b(d + 2k + 4)$ when checked separately. In the complete protocol P is itself a combination of commitments (§4); those of the verifying key are shared as well, and each further proof adds its own points, the $22a + 33$ of Table 8, and the instance commitments computed from its public inputs, one per Action (§4.2). The $d = 2048$ generator terms are paid once per batch. A batch check is not accumulation: it emits nothing that a later proof could consume, and it verifies the batch and stops. Both take linear combinations of deferred equations; their interfaces and guarantees differ, and the accumulation scheme of Bünz, Chiesa, Mishra and Spooner, sketched below under “Accumulation (not deployed by Orchard)”, is not part of the specified protocol.

Transparency: no trusted setup. The generators are outputs of a public hash-to-curve map on fixed inputs (this subsection, “Generators from a hash”); the scheme has no trapdoor and requires no trusted setup, unlike KZG, whose trapdoor permits forgeries (*Crypto Guide*, § “Polynomial commitment schemes”, under “KZG: evaluation in the exponent”). The price is the $2k + 1$ points of Table 8, against the constant-size opening proof of KZG, and the linear verifier. ZIP 224’s motivation names the generation of the structured reference string of Zcash’s earlier proving system a point of risk within the protocol and asks for a proving system that does not require one (ZIP 224, Motivation).

Accumulation (not deployed by Orchard). In a recursive verifier, a circuit that verifies a proof, the multi-scalar multiplication $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = \text{Commit}(g; 0)$ of §3.4 would cost d scalar multiplications inside the circuit. Instead the claim is combined with a running claim of the

same shape, $G'^{(0)} = \text{Commit}(g'; 0)$, by a random linear combination with a challenge ρ drawn after both (a verifier challenge, not a round blinder ρ_j of this section; §7 keeps the letter): $G^{(0)} + [\rho] G'^{(0)} = \text{Commit}(g + \rho g'; 0)$ holds for all ρ when both inputs hold and for at most one ρ otherwise. The combined claim, returned to a single tuple by a prover-assisted opening, is the *accumulator* carried along the chain of proofs, and a single multi-scalar multiplication outside any circuit at the end discharges all combined claims. The soundness of an accumulation scheme for a closely related inner-product commitment is cited in §7 from Bünz, Chiesa, Mishra and Spooner, under the extractability of its openings, which they conjecture from the binding of the Pedersen commitment; the cycle of curves that keeps the recursive arithmetic in the right fields is the other half of Pasta. Both are developed in §7, under “Accumulation and the Halo trick” and “A cycle of curves for efficient native recursion”, where recursion and accumulation over the cycle are classified designed-but-unspecified in the sense of §1.2: the protocol specification uses Halo 2 only to prove and verify Action statements and composes no proofs.

4 The Halo 2 proof system

This section defines the Halo 2 proof system as deployed for the Orchard Action statement: the Action circuit, the instance column, the transcript, the multipoint opening, the protocol, and its zero knowledge. It combines the arithmetisation of §2, which reduces a circuit to committed polynomials and identities among them (the gates, the permutation argument for the wiring, the lookup argument for table membership), with the polynomial commitment of §3, which opens a committed polynomial at a point in logarithmically many group elements and is binding under the discrete logarithm on Vesta. The abstractions the *Crypto Guide* owns, the non-interactive argument and the SNARK, the polynomial interactive oracle proof, the Fiat–Shamir transform and the polynomial-commitment interface, are recalled at the point of use. Halo 2 is a *transparent* PLONKish argument: the parameters of its commitment scheme are derived by hashing to the curve and carry no trapdoor (§3.8), and for the Orchard Action circuit its polynomial commitment works over Vesta. The protocol specification defines Action proofs as proofs of the Halo 2 proving system described in the halo2 book (protocol specification, § 5.4.10.3, “Halo 2”); this section cites the book, by chapter, as the specification of the protocol, and names an implementation only for a behaviour that neither document fixes.

The subsections treat, in order, the Action circuit as a PLONKish circuit (its gates, one gate in full, its lookup arguments and its column polynomials); the instance column; the Fiat–Shamir transform and the deployed transcript; the multipoint opening argument; the complete protocol in seven phases; its zero knowledge; and the compiled-polynomial-IOP pattern the protocol instantiates. Recursion is not deployed by Orchard and is treated in §7.

Notation. The constraint degree is $d_{\max}$, as Definition 2.7 fixed it; the letter d is the committed vector length of §3, which equals the row count n throughout this section. The letter H carries three meanings; two come from the sections this one joins: §2 writes H for the evaluation domain and §3 for the blinding generator of the Pedersen commitment. Here the domain appears only through its vanishing polynomial $Z_H(X) = X^n - 1$, its Lagrange basis ℓ_j and its rows ω^j ; a bare H inside a commitment formula is the blinding generator, as in §3. In §4.3 and the security state-

ments the letter H also names the transcript hash, modelled as a random oracle, as in the *Crypto Guide*; there it is applied to arguments, $H(\cdot)$, or called the hash or the oracle. The letter x is the evaluation challenge of phase 5; the running example’s witness is written by its value 3 here and in §6.

4.1 From statement to circuit: arithmetisation in practice

Definition 2.7 says what a PLONKish circuit is; the deployed instance is the Orchard Action circuit. The protocol specification fixes the statement it proves, the Action statement (protocol specification, § 4.18.4, “Action Statement (Orchard)”), and requires every Action proof to verify under the verifying key identified by the current Orchard circuit version (protocol specification, § 4.6, “Action Descriptions”, consensus rules). No specification or ZIP fixes the parameters of the circuit behind that key; they are those of its implementation, orchard. As a PLONKish circuit in the sense of Definition 2.7 it has the field $\mathbb{F}_{p_{\text{Pallas}}}$, $n = 2^{11}$ rows, ten advice columns, one instance column and twenty-nine fixed columns, among them its selector columns and the three columns of its lookup table; the constraint-degree bound $d_{\text{max}} = 9$; and three lookup arguments into that table, of 2^{10} rows. Its custom gates encode the conditions of the Action statement, using among others complete and incomplete addition on Pallas, the distinction the *Math Guide* draws in § “Explicit affine formulas” (its remark on complete versus incomplete addition). This subsection gives the form of the gates, one gate in full, the three lookup arguments, and the column polynomials on which every argument of §2 operates.

Key and witness. The verifying key is a deterministic function of the commitment parameters and the circuit description: the values of the fixed columns, the gates, the lookup arguments and the permutation σ of Definition 2.7. For the Action circuit it is a fixed consensus parameter, the key the consensus rule above identifies. The prover’s witness is the content of the advice columns. A copy constraint places two cells in one cycle of σ (Construction 2.10), and the public inputs are bound to advice cells in this way (§4.2).

4.1.1 Custom gates. Every custom gate of the deployed circuit, a gate in the sense of Definition 2.7, has the form

$$q \cdot c(v_1(\omega^{r_1} X), \dots, v_s(\omega^{r_s} X)) = 0,$$

where $v_1, \dots, v_s$ are columns read at rotations $r_1, \dots, r_s$, c is a polynomial, and the *selector* q is a polynomial in the fixed columns that is zero on every row where the gate does not apply (the halo2 book’s *PLONKish Arithmetization* chapter). The gate is required to vanish at every row; on the rows where q is zero, the blinding rows among them, it holds whatever the cells contain. The universal gate of vanilla PLONK is the special case of §2.5.

4.1.2 The incomplete-addition gate. The incomplete-addition gate of the Action circuit constrains $R = P + Q$ for affine points $P = (x_p, y_p)$ and $Q = (x_q, y_q)$ of Pallas with $x_p \neq x_q$. It has one selector q_{add} and four equality-enabled advice columns x_p, y_p, x_{qr} and y_{qr} ; it reads x_p, y_p, x_q, y_q at the current row and x_r, y_r at the next row of the *same* two columns x_{qr}, y_{qr} , so that Q occupies the

selector row and R the row below it. Its two constraints are

$$q_{\text{add}} \cdot ((x_r + x_q + x_p)(x_p - x_q)^2 - (y_p - y_q)^2) = 0, \quad (31)$$

$$q_{\text{add}} \cdot ((y_r + y_q)(x_p - x_q) - (y_p - y_q)(x_q - x_r)) = 0 \quad (32)$$

(the halo2 book's *Incomplete and complete addition* chapter).

Theorem 4.1 (The gate is the chord formula with the slope eliminated). *Let $P = (x_p, y_p)$ and $Q = (x_q, y_q)$ be affine points of a short Weierstrass curve with $x_p \neq x_q$, and set $\lambda := (y_p - y_q)/(x_p - x_q)$. For a pair $(x_r, y_r) \in \mathbb{F}^2$ the following are equivalent:*

1. *the bracketed polynomials of (31) and (32) both vanish;*
2. *$x_r + x_q + x_p = \lambda^2$ and $y_r + y_q = \lambda(x_q - x_r)$;*
3. *$(x_r, y_r) = P + Q$ by the chord formulas of the Math Guide (§“Explicit affine formulas”), $x_r = \lambda^2 - x_p - x_q$ and $y_r = \lambda(x_p - x_r) - y_p$.*

Proof. (i) $\iff$ (ii). Since $x_p - x_q \neq 0$, dividing the first bracket by $(x_p - x_q)^2$ and the second by $(x_p - x_q)$ are invertible operations: the first bracket vanishes exactly when $x_r + x_q + x_p = (y_p - y_q)^2 / (x_p - x_q)^2 = \lambda^2$, and the second exactly when $y_r + y_q = \lambda(x_q - x_r)$. The gate stores the cleared forms because a constraint must be a polynomial in the cells, and a division is not.

(ii) $\iff$ (iii). The x -equations are the same equation rearranged. For the y -equations, the chord formula uses the slope of the line through P and Q , which is λ whichever endpoint is subtracted from which, and the identity $\lambda(x_p - x_q) = y_p - y_q$ gives $\lambda x_p - y_p = \lambda x_q - y_q$; hence $\lambda(x_p - x_r) - y_p = \lambda x_q - y_q - \lambda x_r = \lambda(x_q - x_r) - y_q$, which is (ii)'s y -equation with y_q moved across. $\square$

On the 97-point toy curve of §3.7, $y^2 = x^3 + 3$ over $\mathbb{F}_{99}$, the chord formulas give $P + Q = (58, 71)$ for $P = (12, 25)$ and $Q = (71, 53)$, and both brackets are 0 at $(x_r, y_r) = (58, 71)$. On Pallas, with $P = (-1, 2)$ and $Q = [2]P$, both brackets are again 0 at the chord sum.

Degrees. The bracket of (31) has degree three in the cells and that of (32) degree two; with a selector column taking values in $\{0, 1\}$ the constraints have degrees four and three. The deployed key combines selectors into shared fixed columns (the halo2 book's *Selector combining* chapter), which can raise both degrees, keeping them within the deployed bound $d_{\text{max}} = 9$.

Layout. The selector q_{add} is 1 on row i , which holds P and Q , each coordinate copy-constrained to the cell it came from; row $i + 1$ holds R (Table 9). The slope λ occupies no cell: the identities (31)–(32) verify the sum without it, which is what Theorem 4.1 says. By that theorem the two constraints force $(x_r, y_r) = P + Q$, a point of the curve, and $R \neq \mathcal{O}$ because $x_p \neq x_q$ excludes $Q = -P$.

The excluded inputs. If $Q = P$, both brackets vanish identically, since every term carries a factor $x_p - x_q$ or $y_p - y_q$, and (x_r, y_r) is unconstrained. If $Q = -P \neq \mathcal{O}$, the bracket of (31) reduces to $-(2y_p)^2$, which is nonzero because Pallas, of prime order, has no point of order two, and no

Table 9. The cells of one incomplete addition: two rows of four advice columns and the selector. The four input cells x_p, y_p, x_q, y_q on the selector row i are copy-constrained to the cells they came from; the result cells x_r, y_r occupy the x_{qr}, y_{qr} columns of row $i + 1$, where the gate’s next-rotation queries find them. Dashes mark cells the gate does not use. The gate does not read q_{add} on row $i + 1$. When additions are chained, as in fixed-base scalar multiplication, row $i + 1$ is the selector row of the next addition, and R is that addition’s Q .

row	x_p	y_p	x_{qr}	y_{qr}	q_{add}
i	x_p	y_p	x_q	y_q	1
$i + 1$	–	–	x_r	y_r	–

assignment satisfies the gate. The Action circuit applies this gate only to non-identity inputs with $x_p \neq x_q$: it sums the window multiples of fixed-base scalar multiplication for every window but the last, an offset in the window table excluding the identity and the doubling case there, and adds the last window, whose sum can be the identity or a doubling, with the complete-addition gate (the halo2 book’s *Fixed-base scalar multiplication* chapter).

4.1.3 Lookup arguments of the Action circuit. A gate asserts only polynomial identities. “This value has ten bits” and “this point is the j -th Sinsemilla generator” are set memberships, and §2.7 showed why no low-degree gate on the cell alone states them; the circuit states them as lookup arguments. Each is one instance of Construction 2.14, the input expressions playing $A_0, \dots, A_{m-1}$ and the table columns $S_0, \dots, S_{m-1}$, compressed by the challenge θ as in (10) whenever $m > 1$.

The table. The Action circuit has three lookup arguments, one range check and two Sinsemilla generator lookups, and all three use one table. It holds, for $0 \leq j < 2^{10}$,

$$\text{row } j : \quad (j, X(S[j]), Y(S[j])), \quad S[j] := \text{GroupHash}(\text{"z.cash:SinsemillaS", LE}_{32}(j)),$$

in three table columns $(t_{\text{idx}}, t_x, t_y)$, where $\text{LE}_\ell(v)$ is the ℓ -bit little-endian encoding of the integer v , the generator table S is defined by the protocol specification (§ 5.4.1.9, “Sinsemilla Hash Function”), and the hash GroupHash that derives it is the *Crypto Guide*’s (§ “Sinsemilla: an algebraic hash-based commitment”; the hash-to-curve pipeline is § “Hashing to a curve point”). The first two rows are:

row	coordinates of $S[j]$, hexadecimal
$j = 0$	$X = 0\text{db}5218\text{b } e6881\text{f}0\text{f } 1431\text{d}4\text{ea } 7\text{d}4\text{af}c7\text{b } 29\text{a}05\text{baf } \text{bede}62\text{b}5\text{ } 5\text{a}91\text{eb}91\text{ } 2044\text{ea}5\text{f}$ $Y = 2\text{f}0\text{f}40\text{c}2\text{ } \text{f}152\text{a}01\text{c } 9\text{caf}6629\text{ } 8493\text{d}5\text{d}0\text{ } 944\text{a}041\text{c } 2\text{e}65\text{ba}01\text{ } 17\text{c}24\text{f}76\text{ } \text{bf}8\text{e}6483$
$j = 1$	$X = 211112\text{b}4\text{ } \text{b}3\text{e}19518\text{ } \text{c}8\text{fd}33\text{eb } 39175404\text{ } \text{e}6770\text{ca } 24649\text{b}8\text{f } \text{ce}4\text{ae}33\text{e } \text{a}108\text{af}91$ $Y = 06\text{c}59939\text{ } 93\text{adb}03\text{b } \text{a}38\text{a}3\text{e}79\text{ } \text{cd}5\text{a}35\text{fe } \text{b}43\text{c}744\text{a } 670\text{e}19\text{bc } 1\text{d}83\text{c}293\text{ } \text{f}810\text{c}5\text{ee}$

Unused table rows. On every active row $j \geq 2^{10}$ (Remark 2.17), each of t_{idx}, t_x and t_y holds its row-0 value, so that every active row of the table is a genuine entry, $(0, X(S[0]), Y(S[0]))$ or $(j, X(S[j]), Y(S[j]))$. Rows left at 0 would add the tuple $(0, 0, 0)$, which is no entry, since row 0 is the only entry with index 0 and $X(S[0]) \neq 0$, and which an input tuple could then match.

The range check. The range-check lookup is

$$q_{\text{lookup}} \cdot \left(q_{\text{running}} \cdot (z_i - 2^{10} z_{i+1}) + (1 - q_{\text{running}}) \cdot z_i \right) \mapsto t_{\text{idx}}, \quad (33)$$

with z_i and z_{i+1} the running-sum advice column at the current and next rotations. With $q_{\text{running}} = 1$ the input is a running-sum step: the prover witnesses $z_{i+1} = (z_i - a_i) \cdot 2^{-10}$ on successive rows, so the expression recovers the limb $a_i = z_i - 2^{10} z_{i+1}$, and membership in $t_{\text{idx}} = \{0, \dots, 2^{10} - 1\}$ asserts that each limb has ten bits. With $q_{\text{running}} = 0$ the cell itself is looked up. Every range check of the Action circuit is either an instance of this lookup, a limb of fewer than ten bits being looked up together with its shift to ten bits, or, for a range of $R \leq 8$ values, the gate $q \cdot w(w-1) \dots (w-R+1) = 0$; the three-bit windows of fixed-base scalar multiplication and the single bits use the gate (the halo2 book’s *Decomposition* chapter).

The Sinsemilla generator lookup. The Sinsemilla lookup is the three-column instance

$$\left(q_{S1} \cdot m_{i+1}, \quad q_{S1} \cdot x_p + (1 - q_{S1}) \cdot X(S[0]), \quad q_{S1} \cdot y_p + (1 - q_{S1}) \cdot Y(S[0]) \right) \mapsto (t_{\text{idx}}, t_x, t_y), \quad (34)$$

where $m_{i+1} = z_i - 2^{10} \cdot q_{\text{run}} \cdot z_{i+1}$ recovers the ten-bit message word from the hash’s own running-sum decomposition of its message, and $q_{\text{run}} := q_{S2} - q_{S2}(q_{S2} - 1) = q_{S2}(2 - q_{S2})$ is an expression in a fixed column $q_{S2} \in \{0, 1, 2\}$ that evaluates to 1 on the interior rows of a message piece and to 0 on its final row, where the trailing running sum is implicitly zero. The coordinate y_p is not a cell but a polynomial expression in the queried cells equal to the y -coordinate of the generator added (the halo2 book’s *Sinsemilla* chapter).

Expression-valued inputs. Lookup inputs may be arbitrary expressions over queried cells, not merely cells. The Sinsemilla lookup asserts that the generator added is $S[m_{i+1}]$ with no advice cell holding that generator’s y -coordinate. The $(1 - q_{S1})$ branches set the input tuple of every row with $q_{S1} = 0$ to table row 0, so the assertion of Construction 2.14, that every active row’s input tuple appears in the table, holds with nothing to exempt.

4.1.4 Column polynomials and the quotient. Every column, advice, fixed and instance, is a vector of $n = 2^k$ field elements, and its polynomial is its interpolant over the rows, of degree $< n$: the column polynomials of §2.5. The trailing rows of each advice column hold uniformly random field elements, the blinding rows of §4.6, on which every selector is zero and which Remark 2.17 exempts from the permutation and lookup arguments.

The quotient. Each gate polynomial is evaluated with the committed column polynomials substituted for the queried cells, a query at rotation r reading $v(\omega^r X)$, and the resulting polynomials, together with the permutation identities (7) and the lookup identities (9), are folded with powers of a challenge y and divided by $Z_H(X) = X^n - 1$. The result is the quotient $b(X)$ of phase 4 of “The complete Halo 2 protocol” (§4.5), where the folding and the division are written out (the halo2 book’s *Vanishing argument* chapter).

The running example. The running example of §2.2 stands at the same point: it interpolated a four-row trace into a, b, c , the five selector polynomials and PI, and assembled the one polynomial $G(X)$ of degree 9 whose divisibility by $Z_H = X^4 - 1$, with the quotient t of degree 5, the rest of the protocol certifies.

The deployed parameters. For the Action circuit $k = 11$, $n = 2^{11} = 2048$ and $d_{\max} = 9$, the bound holding for its gates and for its permutation and lookup identities. The quotient therefore splits into $d_{\max} - 1 = 8$ chunks of degree below n , derived in §4.5. The two arguments this construction leans on are the ones §2 proved: the permutation argument of §2.6 stands behind every copy constraint, and the lookup argument of §2.8 behind the lookups (33) and (34).

4.2 Instance columns and binding the public statement

Definition 2.7 lists instance columns beside fixed and advice columns, and the arguments of §2 treat them like any other column. This subsection states how the instance column binds a proof to its public statement.

The public input of the running example. In the running example the target 35 is public, so it is not advice but *instance* data: it sits in the public-input polynomial $\text{PI}(X)$ of (2), which the verifier builds himself from the column $(0, 0, 0, -35)$; its interpolant $64 + 50X + 33X^2 + 47X^3$ of §2.2 takes the value $-35 = 62$ on row 3 and 0 elsewhere. A prover who claimed that $X^3 + X + 5 = 36$ has a root would face a *different* G , one with -36 in place of -35 , and her honest trace would no longer vanish on the rows: the fourth gate would read $30 + 5 - 36 \neq 0$. The public input enters the polynomial G whose divisibility Theorem 2.5 tests, and an accepted proof attests the statement for the verifier’s public values.

Definition 4.2 (Instance polynomial). An *instance column* is a public-input column: its n entries $\pi = (\pi_0, \dots, \pi_{n-1}) \in \mathbb{F}^n$ are not part of the witness, and both parties agree on them before verification. Like every other column it is interpolated over the rows in the Lagrange basis,

$$\pi(X) := \sum_{j=0}^{n-1} \pi_j \ell_j(X) \in \mathbb{F}[X], \quad \deg \pi < n,$$

with $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ for $j' \neq j$ (§2.5; the *Math Guide*, §“The Lagrange basis on H ”). The public values occupy designated rows, and every other π_j is 0.

Binding by copy constraints. The deployed circuit binds the public values through the permutation argument of §2.6, not through a gate. The instance column is equality-enabled, so it participates in the permutation σ ; it is one of the fifteen columns counted under “The deployed chunking” there, beside the ten advice and four fixed ones. Each designated instance cell is copy-constrained to an advice cell that the circuit’s constraints read (the halo2 book’s *Permutation argument* chapter for the mechanism), so a satisfying assignment exists only when those advice cells hold the public values the verifier supplied (Figure 10). The alternative, a dedicated gate

$q(X)(a(X) - \pi(X)) \equiv 0 \pmod{Z_H(X)}$ with a selector q active exactly on the designated rows, is not used: the copy constraint reuses the permutation argument, at no extra gate degree.

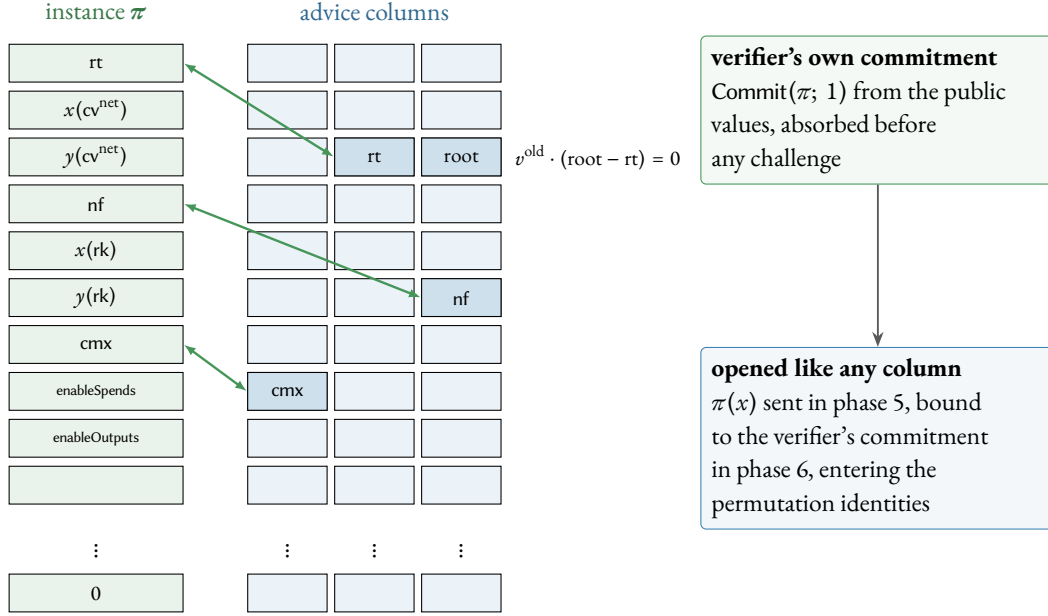


Figure 10. The instance column of the Action circuit. Rows 0 to 8 hold the primary input in the encoding of the protocol specification (§ 4.18.4), the curve points cv^{net} and rk as two coordinates each; the further input `disableCrossAddress` of § 4.6 occupies a row that no normative text fixes, drawn empty; every other row is 0. Each designated cell is copy-constrained to an advice cell the circuit's constraints read; three such constraints are drawn (green double arrows): nf and cmx are copied to the cells where the circuit computes them, rt to a cell that a gate ties to the computed Merkle root. The verifier commits to the column himself, with the fixed blinding factor 1, and absorbs that commitment before the first challenge.

The instance commitment. The verifier does not take $\pi(X)$ from the prover. *Both* parties compute the commitment to each instance polynomial from the instance values (the halo2 book's *Circuit commitments* chapter), with the public blinding factor 1 in place of a random one so that the two computations agree, and absorb it into the transcript before the first challenge is drawn, in phase 1 of §4.5; every Fiat–Shamir challenge thereby depends on the public input. The book fixes the factor 1 for the fixed columns only; for the instance columns it is the choice of the implementation, `halo2_proofs`. Thereafter the instance column is treated like any other: its claimed evaluation $\pi(x)$ appears among the phase-5 evaluations, enters the permutation identities as one of the v_i of Construction 2.10, and the multipoint opening of phase 6 binds it to the instance commitment. Because the verifier computed that commitment himself, from the instance values he verifies against, the prover has no freedom over π : an evaluation inconsistent with the verifier's own values fails the opening argument, by the evaluation binding recalled in §2.9.

The public input of the Action statement. The Action statement's public input is the tuple $(rt, cv^{net}, nf, rk, cmx, enableSpends, enableOutputs)$ (protocol specification, § 4.18.4, "Action Statement (Orchard)"): the anchor rt , the root of the note-commitment tree the spent note is

proved against (the *Crypto Guide*, §“Append-only and incrementally updatable trees”); the net value commitment cv^{net} ; the nullifier nf ; the randomised verification key rk ; the extracted commitment $cmx = \text{Extract}_p(cm)$ of the output note (the *Crypto Guide*, §“A shielded spend, end to end”); and the flags enableSpend s and enableOutput s, which permit non-zero-valued spends and outputs. The primary-input note of the same section encodes the tuple as the sequence

$$[rt, x(cv^{\text{net}}), y(cv^{\text{net}}), nf, x(rk), y(rk), cmx, \text{enableSpend}, \text{enableOutput}] \in \mathbb{F}^9,$$

each integer read as an element of $\mathbb{F} = \mathbb{F}_{p_{\text{Pallas}}}$. The Action description specifies the proof with an eighth component, $\text{disableCrossAddress} := 1 - \text{enableCrossAddress}$, where $\text{enableCrossAddress}$ permits the output note’s receiver to differ from the input note’s (protocol specification, § 4.6, “Action Descriptions”); from NU6.3 the proof of every Action of either pool is verified with this further public input, which is 1 for every Orchard-pool Action (ZIP 258). Rows 0 to 8 of the instance column hold the nine values above; no normative text fixes the row of the further input, and every other row is 0 (Figure 10). Each is copy-constrained to an advice cell the circuit’s constraints read: cv^{net} , nf , rk and cmx to the cells where the circuit computes them; rt to a cell tied to the computed Merkle root by the gate $v^{\text{old}}(\text{root} - rt) = 0$, which encodes the state-ment’s condition that $v^{\text{old}} = 0$ or the Merkle path is valid; the two enable flags to cells read by the gates $v^{\text{old}}(1 - \text{enableSpend}) = 0$ and $v^{\text{new}}(1 - \text{enableOutput}) = 0$, which encode, for flags in $\{0, 1\}$, the conditions $v^{\text{old}} = 0$ or $\text{enableSpend} = 1$, and $v^{\text{new}} = 0$ or $\text{enableOutput} = 1$; and $\text{disableCrossAddress}$ to a cell read by a gate that, when it is nonzero, equates the coordinates of the receivers (g_d, pk_d) of the two notes. That condition is the meaning § 4.6 gives the flag; the list of conditions in § 4.18.4 does not yet include it.

Binding to the instance. The verifier computes the instance commitment from the public values π itself, and the copy constraints tie the designated cells to the advice cells the constraints read; so a witness extracted from an accepting proof satisfies the relation at π , not at other public values π' the prover may target. The probabilistic conclusion, that a prover who knows no witness for π , whatever witnesses she holds for other instances, convinces the verifier with at most negligible probability, holds under the knowledge soundness whose status §5 classifies: designed-but-unspecified for the deployed transcript, resting on the discrete logarithm on Vesta or on the algebraic group model, with the transcript hash modelled as a random oracle. Under that hypothesis an Action proof that verifies against an anchor, a nullifier and a value commitment certifies knowledge of a witness consistent with those values.

4.3 Removing the verifier: the Fiat–Shamir transform

The protocols of §§2–3 are public-coin: every verifier message is uniformly random and independent of the prover’s messages, a field element (the point z of the random-point check, the pair β, γ of the permutation and lookup arguments, the challenges u_j of each opening round) or, in Construction 3.2, the group element U ; and the verifier’s decision is a deterministic function of the public input and the transcript. Their soundness requires only that each challenge be drawn after the commitments it tests are fixed.

The transform. The Fiat–Shamir transform replaces the i -th challenge c_i by

$$c_i := H(x, m_1, c_1, \dots, c_{i-1}, m_i),$$

the hash of the public statement x and of the transcript prefix, the prover’s messages $m_1, \dots, m_i$ and the earlier challenges, under a fixed hash function H (the *Crypto Guide*, §“The Fiat–Shamir transform: from interactive to non-interactive”). The ordering of §2.9 is enforced because each commitment is an input to the hash that derives the challenge testing it. The toy of §2.4 drew $z = 20$ as an interactive challenge; the deployed prover derives every challenge from a hash. The two kinds of challenge are kept apart in what follows: the interactive one is uniform by assumption, the hashed one only in the model in which the hash is a random oracle.

Grinding. A prover against the compiled argument may vary the inputs of H and inspect the resulting challenges. The bound on that freedom uses round-by-round soundness (Canetti, Chen, Holmgren, Lombardi, Rothblum, Rothblum and Wichs, *Fiat–Shamir: From Practice to Theory*), which Remark 4.11 recalls.

Proposition 4.3 (Fiat–Shamir of a round-by-round sound protocol). *Let Π be a public-coin interactive protocol whose partial transcripts for a false statement carry a doomed labelling with error ε : the empty transcript is doomed, a doomed complete transcript is rejected, and from a doomed partial transcript, whatever the prover’s next message, the next challenge leaves it doomed except with probability at most ε . With H modelled as a random oracle, a prover making at most Q distinct oracle queries, the verifier’s recomputations counted among them, makes the verifier of the Fiat–Shamir transform of Π accept with probability at most $Q\varepsilon$.*

Proof. Each query is a transcript prefix followed by a prover message, and its answer is uniform and independent of everything before the query. The verifier accepts only a transcript that is not doomed, and the empty transcript is doomed, so acceptance requires some query on a doomed prefix whose answer leaves the doomed set; each query does so with probability at most ε , and the union bound over the at most Q queries gives $Q\varepsilon$. $\square$

The proposition has the form of the halo2 book’s extraction theorem (*Protocol Description* chapter, “Witness-extended Emulation”, its term $q\varepsilon$) and of the state-restoration analysis of Ghoshal and Tessaro that §5 cites, where its application to the deployed transcript is classified. For the toy’s identity check, a single challenge with error $11/97$ (§2.4), ten queries exhaust the bound. This volume establishes no round-by-round error for Halo 2, whose binding is computational. At Orchard scale the identity check’s term is about 2^{-240} (one Schwartz–Zippel term, in the sense §1.2 fixed), and budgets of 2^{40} , 2^{64} and 2^{80} queries multiply that term alone to 2^{-200} , 2^{-176} and 2^{-160} ; these agree with the $Q\varepsilon_{\text{bad}}$ of the halo2 book’s algebraic-group-model bound quoted in §5.2, proved for the abstract protocol, whose transfer to the deployed transcript §5 classifies. The permutation argument’s term over (β, γ) , $3N/|F|$ with $N = 15 \cdot 2042$ participating cells (Corollary 2.12), is larger, about $2^{-237.5}$.

The hash must take the whole transcript prefix as input: a statement or commitment left out can be chosen after the challenge is seen, the “weak Fiat–Shamir” failure the *Crypto Guide* records in §“The Fiat–Shamir transform: from interactive to non-interactive”. The bound holds with H

modelled as a random oracle, a heuristic. The *Crypto Guide* proves the one-round theorem, with its forking-lemma extractor and its programmed-oracle simulator, and states that a multi-round protocol such as this one needs a multi-round theorem whose hypotheses match the protocol at hand; §5 weighs that theorem against the deployed transcript.

Non-interactivity. The compiled argument is non-interactive: the proof is one static object, commitments, evaluations and the opening’s scalars, and verification recomputes the challenges by the same hash and runs the same checks. The protocol specification defines a Halo 2 proof as a byte sequence (protocol specification, § 5.4.10.3, “Halo 2”, and its “Encoding of Halo 2 Proofs”), so a full node verifies an Action proof from the transaction alone.

The deployed transcript. Neither the protocol specification nor the halo2 book fixes the transcript that instantiates the transform; it is that of the implementation, halo2.proofs, and is as follows. The state is a BLAKE2b-512 hash state with the sixteen-byte personalisation Halo2-Transcript (the *Crypto Guide*, §“Domain separation and personalisation”). Absorbing a point $P = (x, y) \neq \mathcal{O}$ of Vesta appends the bytes $0x01 \parallel \text{LE}_{256}(x) \parallel \text{LE}_{256}(y)$; the identity is not absorbable. Absorbing a scalar $s \in \mathbb{F}$ appends $0x02 \parallel \text{LE}_{256}(s)$. Squeezing a challenge appends $0x00$ to the state, which keeps it, and returns the 64-byte digest of the state read as a little-endian integer and reduced modulo p_{Pallas} , with no rejection of any value (§3.8). The proof carries each point as its 32-byte compressed encoding, the x -coordinate with the parity of y in the top bit (protocol specification, § 5.4.9.6, “Pallas and Vesta”), and each scalar as $\text{LE}_{256}(s)$. Before any prover message both parties absorb, as a scalar, a digest of the verifying key, its BLAKE2b hash reduced into $\mathbb{F}$, and then the instance commitments of §4.2, which therefore never appear in the proof. Every prover message is then absorbed in the order the protocol fixes, and each challenge is squeezed at its prescribed point, after everything it must depend on. The proof is the sequence of the prover’s messages in the phase order of Table 10, counted by kind in Table 8; the verifier replays the absorptions and recomputes every challenge. Figure 12 in §4.5 draws the timeline.

4.4 The multipoint opening argument

When the verifier has drawn his evaluation point x , the prover has committed to several dozen polynomials and claimed their values at x and at rotations of x : a running product at x and ωx , a permuted lookup input at x and $\omega^{-1}x$, an advice column at every rotation its gates read. Opening each claim with the argument of §3 would cost $2k + 1 = 23$ points per claim, and a one-Action proof makes 95 claims about 69 committed polynomials, the 94 evaluations at x and its rotations in Table 8 and the quotient’s ($49a + 46$ claims for a Actions). Halo 2 reduces all of them to a single opening of a single polynomial at a single point. The reduction combines random linear combinations, division by vanishing polynomials and one random-point check. The letters q and f below follow the halo2 book’s *Multipoint opening argument* chapter and are unrelated to the selector polynomials q_i of §2, which are among the polynomials being folded; nor is this f the final blinder f of the deployed opening (§3.8), which phase 7 sends. The book’s *Protocol Description* chapter writes the values $q_i(x_3)$ as u , the letter this volume reserves for the round challenges of the inner-product argument; they are written $\bar{q}_i$ here. The index i ranges over sets of query points, not over columns.

Construction 4.4 (Multipoint opening). Let $p_1, \dots, p_N$ be polynomials of degree $< n$ with commitments $C_1, \dots, C_N$ under Construction 3.2, each p_ν queried at a finite set $T_\nu \subset \mathbb{F}$ of points, and let $\hat{p}_\nu(t)$ for $t \in T_\nu$ be the values the prover has claimed. Let $T^{(1)}, \dots, T^{(m)}$ be the distinct sets among the T_ν and $I_i := \{\nu : T_\nu = T^{(i)}\}$ the i -th group, enumerated as $\nu_{i,0}, \nu_{i,1}, \dots$; write $Z_i(X) := \prod_{t \in T^{(i)}} (X - t)$.

1. **Fold each group (x_1).** The verifier sends $x_1 \in \mathbb{F}$. For each group i the prover forms $q_i(X) := \sum_e x_1^e p_{\nu_{i,e}}(X)$, the verifier forms the same combination of commitments, $Q_i := \sum_e [x_1^e] C_{\nu_{i,e}}$, which commits to q_i by linearity (*Crypto Guide*, §“Pedersen vector commitments”), and both form the combined claimed values $\hat{q}_i(t) := \sum_e x_1^e \hat{p}_{\nu_{i,e}}(t)$ for $t \in T^{(i)}$. Both interpolate $r_i(X)$, the unique polynomial of degree $< |T^{(i)}|$ with $r_i(t) = \hat{q}_i(t)$ on $T^{(i)}$ (the *Math Guide*, §“Lagrange interpolation”); the verifier can do this from the claimed values alone.
2. **Divide out the points (x_2).** The verifier sends $x_2 \in \mathbb{F}$. The prover forms the quotients $f_i(X) := (q_i(X) - r_i(X))/Z_i(X)$, which are polynomials exactly when the combined values $\hat{q}_i(t)$ equal $q_i(t)$ for every $t \in T^{(i)}$, in particular when every claimed value is true, and combines them into

$$f(X) := \sum_{i=1}^m x_2^{i-1} f_i(X),$$

samples a fresh blinder r_f and sends $F := \text{Commit}(f; r_f)$.

3. **A fresh point (x_3).** The verifier sends $x_3 \in \mathbb{F}$. The prover sends $\bar{q}_i := q_i(x_3)$ for $i = 1, \dots, m$.
4. **The value f must take.** The verifier computes

$$v_f := \sum_{i=1}^m x_2^{i-1} \frac{\bar{q}_i - r_i(x_3)}{Z_i(x_3)},$$

the value of $f(x_3)$ if every f_i is a polynomial and every $\bar{q}_i$ is honest; he can, since r_i and Z_i are his to evaluate, and he rejects if some $Z_i(x_3)$ is 0.

5. **One polynomial (x_4).** The verifier sends $x_4 \in \mathbb{F}$. Both parties form

$$f^*(X) := f(X) + \sum_{i=1}^m x_4^i q_i(X), \quad F^* := F + \sum_{i=1}^m [x_4^i] Q_i, \quad v^* := v_f + \sum_{i=1}^m x_4^i \bar{q}_i,$$

and run one opening of Construction 3.2 on the statement (F^*, x_3, v^*) , the prover’s witness being the coefficient vector of f^* and the blinder $r_f + \sum_i x_4^i r_{Q_i}$, where r_{Q_i} is the blinder of Q_i , the x_1 -combination of the blinders of the polynomials in group i . The verifier accepts the original claims if and only if that opening accepts.

Two remarks on the shape. Nothing is sent between x_1 and x_2 , so the deployed transcript squeezes them back to back; they are two challenges because they play two roles, one keeping the polynomials of a group linearly independent and one keeping the quotients of different groups independent. And the reduction is free for the verifier in group operations except for the combinations of commitments, which are a few scalar multiplications per committed polynomial; the

one expensive step, the final multi-scalar multiplication, is paid once, inside the single opening. Figure 11 draws the data flow.

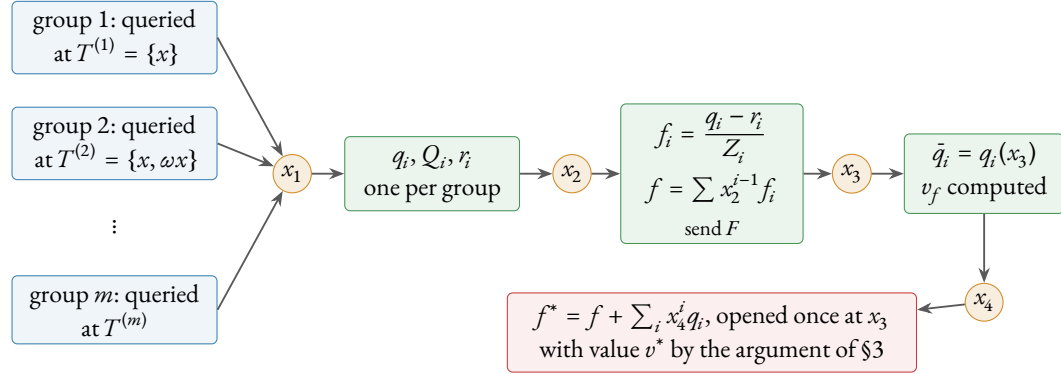


Figure 11. The multipoint reduction of Construction 4.4. Committed polynomials are grouped by the set of points at which they are queried (blue), folded within each group by x_1 into q_i , quotiented by the group’s vanishing factor into f_i , combined across groups by x_2 into f , which is committed as F ; after the fresh point x_3 the prover sends the $q_i(x_3)$, and x_4 folds f and the q_i into f^* (red), whose single opening at x_3 discharges every original claim. Challenges are amber.

Completeness. If every claimed value is true, then $q_i - r_i$ vanishes on $T^{(i)}$ and is divisible by Z_i (the *Math Guide*, §“Roots and the factor theorem”), each f_i is a polynomial of degree $< n$, so is f , and $f(x_3) = v_f$ whenever no $Z_i(x_3)$ is 0; $f^*(x_3) = v^*$ then holds, and the honest opening of §3 accepts. The verifier’s rejection when x_3 is one of the query points, an event of probability at most $|\bigcup_i T^{(i)}|/|\mathbb{F}|$, is the reduction’s only completeness defect.

Lemma 4.5 (Soundness of the multipoint reduction). *In Construction 4.4, let the committed polynomials p , and the claimed values be fixed before x_1 is drawn, let $\hat{f}$ be a polynomial of degree $< n$ committed in F , fixed before x_3 is drawn, let $x_1, \dots, x_4$ be independent and uniform in $\mathbb{F}$, and let $Z(X) := \prod_{t \in \bigcup_i T^{(i)}} (X - t)$. If some claimed value is false, $\hat{p}_{v_0}(t_0) \neq p_{v_0}(t_0)$ with v_0 in group i_0 , then the probability that the verifier computes v_f and the polynomial $\hat{f} + \sum_i x_4^i q_i$ committed in F^* takes the value v^* at x_3 is at most*

$$\frac{|I_{i_0}| + 2m + n - 1 + \deg Z}{|\mathbb{F}|}.$$

Proof. The argument takes one Schwartz–Zippel term per challenge (the *Math Guide*, §“The Schwartz–Zippel lemma”). *The fold.* The difference $\hat{q}_{i_0}(t_0) - q_{i_0}(t_0) = \sum_e x_1^e (\hat{p}_{v_0,e}(t_0) - p_{v_0,e}(t_0))$ is a nonzero polynomial in x_1 of degree below $|I_{i_0}|$, fixed before x_1 is drawn, so except with probability $|I_{i_0}|/|\mathbb{F}|$ the folded claim is false: $r_{i_0}(t_0) \neq q_{i_0}(t_0)$. *The combination.* Each $T^{(i)}$ is contained in $\bigcup_{i'} T^{(i')}$, so each Z/Z_i is a polynomial. The polynomial

$$M(X) := \hat{f}(X) Z(X) - \sum_{i=1}^m x_2^{i-1} (q_i(X) - r_i(X)) \frac{Z(X)}{Z_i(X)}$$

is fixed once x_2 is drawn and F is sent, before x_3 . At t_0 the term $\hat{f}(t_0) Z(t_0)$ and every term with $t_0 \notin T^{(i)}$ vanish, and

$$M(t_0) = - \sum_{i: t_0 \in T^{(i)}} x_2^{i-1} (q_i(t_0) - r_i(t_0)) \prod_{t \notin T^{(i)}} (t_0 - t),$$

the product over $t \in \bigcup_{i'} T^{(i')} \setminus T^{(i)}$: a polynomial in x_2 of degree below m , fixed before x_2 is drawn, whose coefficient at $x_2^{i_0-1}$ is nonzero. Except with probability $m/|\mathbb{F}|$ over x_2 , then, $M(t_0) \neq 0$ and M is a nonzero polynomial of degree at most $n - 1 + \deg Z$. *The last fold.* Acceptance means $(\hat{f}(x_3) - v_f) + \sum_i x_4^i (q_i(x_3) - \bar{q}_i) = 0$, a polynomial equation in x_4 of degree at most m whose coefficients are fixed before x_4 is drawn. Unless every coefficient is zero, that is, unless $\hat{f}(x_3) = v_f$ and $\bar{q}_i = q_i(x_3)$ for every i , a uniform x_4 satisfies it with probability at most $m/|\mathbb{F}|$. *The fresh point.* If $\hat{f}(x_3) = v_f$ and every $\bar{q}_i = q_i(x_3)$, then, since v_f is computed only when $Z(x_3) \neq 0$, multiplying the first equation by $Z(x_3)$ gives $M(x_3) = 0$, which a uniform x_3 achieves for a nonzero M with probability at most $(n - 1 + \deg Z)/|\mathbb{F}|$. A union bound over the four terms gives the stated bound. $\square$

The bound is one more Schwartz–Zippel allowance, of the order of $n/|\mathbb{F}|$. The opening’s own knowledge error, $O(\log d/|\mathbb{F}|)$ by Corollary 3.13, lies below it, and that corollary’s witness-or-relation form supplies the binding opening the lemma assumes.

The deployed instance. In the Action circuit the point sets are five. Every fixed, instance and permuted-label polynomial, the random polynomial of phase 4 and the collapsed quotient are queried at $\{x\}$; the advice columns, five at $\{x, \omega x\}$ and five at $\{\omega^{-1}x, x, \omega x\}$, according to the rotations their gates read; the permuted lookup input A' at $\{x, \omega^{-1}x\}$ and the permuted table S' at $\{x\}$; the lookup running products and the last permutation running product at $\{x, \omega x\}$; and the first two permutation running products at $\{x, \omega x, \omega^{-6}x\}$, the last rotation reaching the boundary row of Remark 2.17 that chains one product into the next. The sets $\{x\}$, $\{x, \omega x\}$, $\{x, \omega^{-1}x\}$, $\{\omega^{-1}x, x, \omega x\}$ and $\{x, \omega x, \omega^{-6}x\}$ give the five values $\bar{q}_i$, the multipoint evaluations of Table 8, and the whole reduction adds one point and five scalars to the proof.

4.5 The complete Halo 2 protocol

The pieces are now assembled into the Halo 2 protocol (the halo2 book’s *Protocol Description* chapter, instantiated with the arguments of its *Proving system* chapters), in seven phases. Throughout, $\mathbb{F}$ is the constraint field, $\mathbb{F}_{p_{\text{Pallas}}}$ for Orchard; $\mathbf{G}$ is the commitment group, the Vesta curve, whose order is p_{Pallas} (§3.8); the rows are $\omega^0, \dots, \omega^{n-1}$ for a primitive n -th root of unity ω , with $n = 2^k$; and every commitment is $\text{Commit}(p; r) = \langle \mathbf{p}, \mathbf{G} \rangle + [r]H$ of Construction 3.2, H the blinding generator. The protocol has a preprocessed setup and seven phases: (1) advice commitments; (2) lookup compression, challenge θ ; (3) running products, challenges β, γ ; (4) the quotient, challenge y ; (5) the evaluation challenge x ; (6) the multipoint opening, challenges x_1, x_2, x_3, x_4 ; (7) the inner-product opening. Every challenge is squeezed from the transcript of §4.3 at the point named; “the verifier sends” below means that. Table 10 lists the messages with their Orchard counts and Figure 12 draws the timeline.

Setup. The circuit fixes the columns, gates, lookup arguments and the equality permutation σ , as §4.1 described; its selectors and lookup tables are fixed columns. Key generation interpolates every fixed column and every permuted-label polynomial $s_i(X)$ of Construction 2.10 and commits to them with the fixed blinding factor 1 (for the fixed columns, the halo2 book’s *Circuit commitments* chapter); these commitments, with the circuit description, form the verifying key. The transparent parameters of the commitment scheme, the n generators $\mathbf{G}$, the blinding generator H and the inner-product generator U hashed to the curve as §3.8 described, together with the domain data, are held separately. All of it is reproducible from public algorithms and the circuit; no secret setup trapdoor exists. Verification is a deterministic function of the parameters, the verifying key, the instance values and the proof.

Phase 1: advice commitments. Both parties first absorb the digest of the verifying key and the instance commitments of §4.2 into the transcript, so that every subsequent challenge depends on the circuit and on the public input. The prover constructs the advice columns $\mathbf{w}_1, \dots, \mathbf{w}_{n_a} \in \mathbb{F}^n$, which with the fixed and instance columns form a satisfying assignment in the sense of Definition 2.7, fills the reserved trailing rows of each with uniformly random field elements, the blinding rows of §4.6, interpolates each to $w_i(X)$ of degree $< n$, and sends $\text{Commit}(w_i; r_i)$ with fresh blinders r_i . The verifier absorbs the commitments.

Phase 2: lookup compression, challenge θ . The verifier sends θ . For each lookup the prover compresses the input expressions and table columns by powers of θ as in (10), forms the permuted columns A' and S' of Construction 2.14, and sends their two commitments; the verifier absorbs them.

Phase 3: running products, challenges β, γ . The verifier sends β and γ . The prover commits to the permutation running products, one per chunk of $d_{\max} - 2$ columns as “The deployed chunking” in §2.6 described, and to one lookup running product Z_{lookup} per lookup, and sends the commitments; the verifier absorbs them. The order is not a convenience. The permuted columns A' and S' must be fixed *before* the challenges that randomise the products are drawn, since Theorem 2.15 bounds the prover’s success probability only over challenges drawn after her columns; a prover who saw β, γ first could choose the permutation against them.

Phase 4: the quotient, challenge y . Before y is drawn the prover commits to a uniformly random polynomial of degree $< n$, sends the commitment, and the verifier absorbs it; its role is the zero-knowledge one explained in §4.6. The verifier then sends y . The prover assembles the *gate equation*, one polynomial

$$C(X) := \sum_{t=0}^{T-1} y^t g_t(X),$$

the y -weighted combination of the T constraint polynomials g_t of the circuit: each constraint of each custom gate with its selector factor, the permutation identities (7) and the chunk-boundary identities, the lookup identities (9), and the boundary identities of Remark 2.17, each masked by its active-row factor where that remark requires. If every constraint holds then every g_t vanishes on the rows, so C does, and $Z_H \mid C$ by (3): the *quotient* $b(X) := C(X)/Z_H(X) = C(X)/(X^n - 1)$

is a polynomial. If some constraint fails at some row, Lemma 4.6 bounds the probability that C nonetheless vanishes on the rows, since γ is drawn after every polynomial in the g_t is committed and after θ , β and γ .

Lemma 4.6 (Batching by γ). *Let $g_0, \dots, g_{T-1} \in \mathbb{F}[X]$ be fixed, and let γ be uniform in $\mathbb{F}$. If some g_t does not vanish on the rows, then $C = \sum_{t=0}^{T-1} \gamma^t g_t$ vanishes on the rows with probability at most $(T-1)/|\mathbb{F}|$.*

Proof. Let $g_{t_0}(\omega^j) \neq 0$. Then $C(\omega^j) = \sum_t \gamma^t g_t(\omega^j)$ is a polynomial in γ of degree at most $T-1$ with the nonzero coefficient $g_{t_0}(\omega^j)$, so it vanishes for at most $T-1$ values of γ (the *Math Guide*, §“The Schwartz–Zippel lemma”). $\square$

Theorem 4.7 (Degree of the quotient). *Let every column polynomial and every permuted-label polynomial s_i have degree $< n$, and every constraint polynomial degree at most $d_{\max}$, counting as degree one each column polynomial at a rotation, each s_i , each of ℓ_0 , ℓ_{last} and ℓ_{blind} , and X (as in “The deployed chunking” of §2.6). Then*

$$\deg b \leq d_{\max}(n-1) - n = (d_{\max} - 1)(n-1) - 1.$$

Proof. A constraint of degree at most $d_{\max}$ is a sum of products of at most $d_{\max}$ such factors, each of degree at most $n-1$ (rotations do not change degrees); so $\deg g_t \leq d_{\max}(n-1)$ and $\deg C \leq d_{\max}(n-1)$. Division by Z_H , of degree n , subtracts n . The two forms agree since $(d_{\max} - 1)(n-1) - 1 = d_{\max}(n-1) - (n-1) - 1 = d_{\max}(n-1) - n$. $\square$

For the deployed $d_{\max} = 9$ and $n = 2048$ the bound is $\deg b \leq 16375$, far above the degree bound $n-1 = 2047$ that the commitment of Construction 3.2 supports with $d = n$ generators, so b cannot be committed as one polynomial. The prover therefore uses a fixed proof shape of $d_{\max} - 1$ chunks $b_0, \dots, b_{d_{\max}-2}$, each of degree $< n$, with

$$b(X) = \sum_{i=0}^{d_{\max}-2} X^{ni} b_i(X),$$

retaining leading zero chunks when the actual degree is smaller, and commits to each chunk separately with its own blinder; there are 8 chunks for Orchard. The count is right because b has at most $(d_{\max} - 1)(n-1) = (d_{\max} - 1)n - (d_{\max} - 1) < (d_{\max} - 1)n$ coefficients. Once the evaluation point x is drawn, the verifier recombines the chunk commitments, weighted by powers of x^n , into one commitment $\sum_i [x^{ni}] \text{Commit}(b_i)$ to the polynomial $\sum_i x^{ni} b_i(X)$, whose value at x is $b(x)$; no chunk is ever opened on its own (the halo2 book’s *Protocol Description* chapter, steps 5 to 8, and note 1 of its “Zero-knowledge and Completeness”).

Phase 5: the evaluation challenge x . The verifier sends x . The prover evaluates every committed polynomial the verifier’s identities query, *except* the quotient chunks, at x and at the rotated points the identities reference, ωx for the running products and for any gate reading the next row, $\omega^{-1}x$ for the permuted lookup input A' and for gates reading the previous row, and $\omega^{-6}x$, the

deployed boundary row, for all but the last permutation product, and sends the claimed values as field elements. The deployed proof also carries the evaluations at x of the fixed columns and of the permuted-label polynomials, which the same opening binds to the verifying key's commitments, and of the random polynomial, bound to its phase-4 commitment. No evaluation of h is sent: the verifier computes the expected quotient value $C(x)/(x^n - 1)$ from the sent evaluations, evaluating $\ell_0, \ell_{\text{last}}, \ell_{\text{blind}}$ and Z_H at x himself, and enforces it against the collapsed chunk commitment inside the multipoint opening of phases 6 and 7, where the claimed evaluations themselves are bound to their commitments as well. The gate equation is thus checked as the identity $C(x) = h(x) Z_H(x)$, the deployed form of (5).

Phase 6: the multipoint opening, challenges $x_1, \dots, x_4$. The parties run Construction 4.4 on every committed polynomial with a claimed evaluation: the advice columns, the instance column, the fixed and permuted-label polynomials, the permuted lookup columns, the running products, the random polynomial, and the collapsed quotient with its expected value, the last two joining the $\{x\}$ group. The prover sends the commitment F after x_1, x_2 and the five values $\tilde{q}_i$ after x_3 ; after x_4 both parties hold the statement (F^*, x_3, v^*) .

Phase 7: the inner-product opening. The parties execute the deployed opening of §3.8, the masked argument in the normalisation of Remark 3.9, on (F^*, x_3, v^*) . The prover sends the mask commitment S ; the verifier sends ξ and ζ ; then for $k = 1$ rounds the prover sends the pair (L_j, R_j) and the verifier answers with u_j ; finally the prover sends the scalars c and f . The verifier accepts if and only if the deployed equation (29) holds with $P := F^*, v := v^*$ and $x := x_3$, its one multi-scalar multiplication of length n included. Its symmetric counterpart is the masked check (24), which differs from it by the three changes of that remark: the fold normalisation, a change of variables; the factor ζ on U ; and the value carried as $+ [v]U$ rather than $- [v]G_0$. Neither it nor (15) is the deployed equation. This is the proof's end; Table 8 counts its bytes by kind, and nothing else is sent.

Accumulation, not deployed. In the recursive variant of Halo 2 the verifier is itself a circuit, and that circuit defers the one expensive step of phase 7, the claim $G^{(0)} = (\mathbf{s}, \mathbf{G}) = \text{Commit}(g_{\text{dep}}; 0)$ of (29), the deployed form (Remark 3.9) of the claim $G^{(0)} = \text{Commit}(g; 0)$ of §3.4, into an accumulator folded with the accumulators of earlier proofs; a single decider at the end verifies all accumulated claims. Orchard does not do this. The protocol specification uses Halo 2 only to prove and verify Action statements and composes no proofs (protocol specification, § 4.1.13, “Zero-Knowledge Proving System”), the Orchard design proposal states that it makes no use of the recursion (ZIP 224), and the verifier pays every deferred multiplication itself; several proofs may share it by the randomised batch check of Lemma 3.14. The variant is developed as the coda of §7.

Costs. The prover computes one multi-scalar multiplication of length n per commitment she sends before the opening, $22a + 10$ for a bundle of a Actions: per Action the ten advice columns, the six permuted lookup columns and the three permutation and three lookup products; shared, the random polynomial, the eight quotient chunks and the multipoint polynomial f . She also

computes the a instance commitments, as the verifier does (§4.2). The opening adds one of length n , for its mask commitment S , and the $2k$ cross terms of its rounds, two in each, of lengths $n/2, n/4, \dots, 1$ in successive rounds. The verifier performs $O(a + \log n)$ field and group operations for a proof of a Actions: he commits to the a instance columns (at most ten nonzero values each), recomputes the challenges, evaluates each Action's gate equation at x , combines the $23a + 46$ commitments of phase 6, and runs the k rounds of phase 7; to this he adds the single deferred multi-scalar multiplication of length n , which is the linear term that §3.8.1 discussed and that a batch verifier amortises across proofs with fresh random scalars; that batching is distinct from the accumulation above.

Table 10. The seven phases as absorb-and-squeeze events, with the Orchard counts of Table 8. Each challenge is listed in the phase that draws it, and the prover column states which messages follow it. The per-Action messages repeat for each of the a Actions of a bundle and the shared ones appear once, giving $2720 + 2272a$ bytes. The phase-5 scalars per Action are the instance evaluation, 25 advice evaluations, 8 permutation-product and 15 lookup evaluations; the shared 45 are 29 fixed, 15 permuted-label and 1 random-polynomial evaluation.

phase	prover sends	verifier squeezes	Orchard count
before 1	— (both absorb the key digest and the instance commitments)	—	0 bytes
1	advice commitments	—	10 points per Action
2	after θ , the A' and S' commitments, per lookup	θ	3 + 3 points per Action
3	after β, γ , the permutation products and the lookup products	β, γ	3 + 3 points per Action
4	random polynomial commitment; then, after γ , the quotient chunks	γ	1 + 8 points, shared
5	after x , the claimed evaluations at x and its rotations	x	49 scalars per Action; 45 shared
6	after x_1, x_2 , the commitment F ; then, after x_3 , the values $\bar{q}_i$	x_1, x_2, x_3 ; then x_4	1 point, 5 scalars, shared
7	S ; then, after ξ, ζ , the pairs (L_j, R_j) for $j = k, \dots, 1$, each followed by u_j ; then c, f	ξ, ζ, u_j after each pair	1 + 22 points, 2 scalars, shared

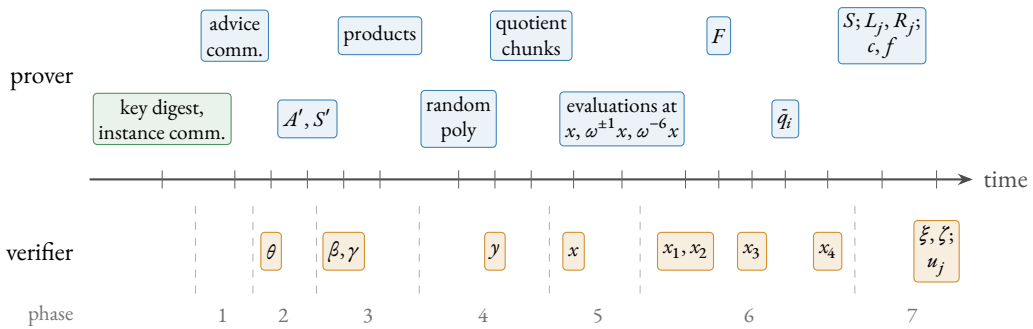


Figure 12. The transcript of §4.3 as a timeline. Prover messages (blue, staggered for legibility) are absorbed above the line; the data both parties hold in advance (green) is absorbed but never sent; challenges (amber) are squeezed below the line, each after everything it must depend on; dashed lines separate the phases of Table 10. The proof is the sequence of blue boxes read left to right.

4.6 Zero knowledge: hiding the witness in Halo 2

Knowledge soundness, classified in §5, concerns what a convincing prover knows; zero knowledge concerns what the proof reveals. Halo 2 is zero-knowledge through two devices, hiding commitments and blinded polynomials, assembled into a simulator. This subsection states what each device achieves on its own, and states the claim for the whole protocol as a cited proposition with its hypotheses.

Evaluations of the running example. The prover of the running example sends commitments and the evaluations $a(20)$, $b(20)$, $c(20)$, $t(20)$ at the random point. An evaluation is a linear equation in the secret coefficients: $a(20) = 90 + 61 \cdot 20 + 22 \cdot 20^2 + 24 \cdot 20^3 = 59$ is one equation in the four unknowns (90, 61, 22, 24), and evaluations at four distinct points determine the column, hence the secret witness 3. Halo 2 therefore blinds: the witness trace reserves independent random rows, and every commitment carries an independent random blinding term. Adding random coefficients to a column polynomial would not do: a column polynomial is determined by its n row values, and a random coefficient on X^i changes every row, breaking the gates; a random multiple of the Lagrange polynomial ℓ_j of a reserved row changes row j alone, which is what a blinding row is. The random values violate no constraint: every selector is zero on the reserved rows, so the gates hold there, and the permutation and lookup identities carry the active-row factor of Remark 2.17. Each column’s openings are then uniform (Lemma 4.8); the joint statement is the proposition of “The simulator” below, whose simulator is one in the sense of the *Crypto Guide* (§“The simulation paradigm and zero knowledge”).

Hiding commitments. The commitment of Construction 3.2 is *perfectly hiding*. The prover commits a polynomial p as

$$\text{Commit}(p; r) = \langle \mathbf{p}, \mathbf{G} \rangle + [r] H, \quad r \text{ uniform in } \mathbb{F},$$

with $\mathbf{p}$ the coefficient vector, $\mathbf{G}$ the generator vector, and H the blinding generator of §3, of no known discrete-logarithm relation to $\mathbf{G}$; the term $[r]H$ makes the commitment a uniform group element for every p , since H generates $\mathbb{G}$, so a commitment by itself leaks nothing about what was committed. This is the perfect hiding of the Pedersen commitment, proved in the *Crypto Guide* (§“The Pedersen commitment” and §“Hiding and binding”) and recalled in §2.9. Every commitment the prover sends in phases 1 to 6, the advice columns, the permuted lookup columns A' and S' , the permutation and lookup running products, the random polynomial, the quotient chunks and the multipoint polynomial f , has this form with a fresh r (the halo2 book’s *Protocol Description* chapter, which blinds every group element of the protocol, its note 4). The verifying key’s commitments and the instance commitments carry the fixed blinding factor 1 instead: they commit public data both parties recompute (§4.5, §4.2), so there is nothing to hide, and a random blinder would prevent the two computations from agreeing.

Blinding rows. The prover opens commitments, and an opening reveals evaluations. To keep those evaluations from revealing the witness, the last n_{blind} rows of the table, the *blinding rows*, carry no circuit data: no gate applies there, and each advice column holds uniformly random field

elements on them. By Lemma 4.8 below, a column opened at m distinct points outside the rows needs $n_{\text{blind}} \geq m$. The halo2 book's *Protocol Description* chapter accordingly requires $n_e + 1$ random evaluations of a polynomial opened at n_e rotations of x , the extra one covering its opening at the point x_3 of the multipoint reduction. In the Action circuit a polynomial is opened at most four times: at three rotations of x , for some advice columns and the first two permutation products, and at x_3 . The deployed circuit takes $n_{\text{blind}} = 5$, one more than that, a parameter fixed by the implementation, `halo2_proofs`; with the boundary row of Remark 2.17 its usable rows number $n - (n_{\text{blind}} + 1) = 2042$. The permutation and lookup running products are blinded the same way, their trailing rows random, and the remark's active-row factor exempts every one of those rows from every argument, so the random values do not violate a constraint (Figure 13; the halo2 book's *Permutation argument* and *Lookup argument* chapters, their "Zero-knowledge adjustment" sections).

Lemma 4.8 (Openings of a column with blinding rows). *Let $v(X) = \sum_{j=0}^{n-1} v_j \ell_j(X)$ be a column polynomial over the rows $\omega^0, \dots, \omega^{n-1}$, let B be a set of n_{blind} row indices, the blinding rows, and let $z_1, \dots, z_m \in \mathbb{F}$ be distinct points, none of them a row. With the values on the other rows held fixed, the map $(v_j)_{j \in B} \mapsto (v(z_1), \dots, v(z_m))$ from $\mathbb{F}^{n_{\text{blind}}}$ to $\mathbb{F}^m$ is affine, and its linear part has rank $\min(m, n_{\text{blind}})$ (Math Guide, §"Bases, dimension, and linear maps"). Consequently, if $m \leq n_{\text{blind}}$ and the blinding values are uniform and independent, the m evaluations are jointly uniform on $\mathbb{F}^m$ and independent of the values on the other rows.*

Proof. The map is $v(z_i) = \sum_{j \notin B} v_j \ell_j(z_i) + \sum_{j \in B} v_j \ell_j(z_i)$, a constant plus the linear map with matrix $L := [\ell_j(z_i)]_{i \leq m, j \in B}$. The Lagrange basis on the rows has the closed form $\ell_j(X) = \omega^j (X^n - 1) / (n(X - \omega^j))$ (the *Math Guide*, §"The Lagrange basis on H "), so $L = D_1 K D_2$ with $D_1 := \text{diag}(z_i^n - 1)$, nonzero on the diagonal because no z_i is a row, $D_2 := \text{diag}(\omega^j / n)$, nonzero because n is invertible in $\mathbb{F}$, and $K := [1 / (z_i - \omega^j)]$. Hence $\text{rank } L = \text{rank } K$. Every square submatrix of K is nonsingular: take rows I and columns J with $|I| = |J| = s$, and suppose coefficients c_j , $j \in J$, not all zero, satisfy $\sum_{j \in J} c_j / (z_i - \omega^j) = 0$ for all $i \in I$. Over the common denominator, $\sum_{j \in J} c_j (X - \omega^j) = N(X) / \prod_{j \in J} (X - \omega^j)$ with $N(X) := \sum_{j \in J} c_j \prod_{j' \in J, j' \neq j} (X - \omega^{j'})$ of degree $< s$; the polynomial N vanishes at the s distinct points z_i , so it is the zero polynomial (the *Math Guide*, §"Roots and the factor theorem"), and then $N(\omega^j) = c_j \prod_{j' \neq j} (\omega^j - \omega^{j'}) = 0$ forces $c_j = 0$ for every $j \in J$, since the rows are distinct: a contradiction. Thus K has an invertible square submatrix of size $\min(m, n_{\text{blind}})$, and its rank, at most $\min(m, n_{\text{blind}})$, equals $\min(m, n_{\text{blind}})$.

If $m \leq n_{\text{blind}}$ the linear part is onto $\mathbb{F}^m$, so the affine map is onto, and every fibre is a coset of the same kernel, of the same size $|\mathbb{F}|^{n_{\text{blind}} - m}$. A uniform point of $\mathbb{F}^{n_{\text{blind}}}$ therefore lands in each fibre with the same probability: the image is uniform on $\mathbb{F}^m$. The constant term depends on the other rows but not the distribution, so the evaluations are independent of them. $\square$

On the toy domain $\{1, 22, 96, 75\}$ of §2.2, the rank of the lemma's map was computed for every choice of $m \leq 3$ distinct points among the 93 outside it and for 2000 random choices of four: with $n_{\text{blind}} = 2$ blinding rows the rank is 1 for one point and 2 for two, and the map is onto; for three points the rank stays 2 and it is not; with $n_{\text{blind}} = 3$ the ranks are 1, 2, 3 for one, two and three points, onto each time, and 3 for four points, no longer onto. The threshold is exact: the map

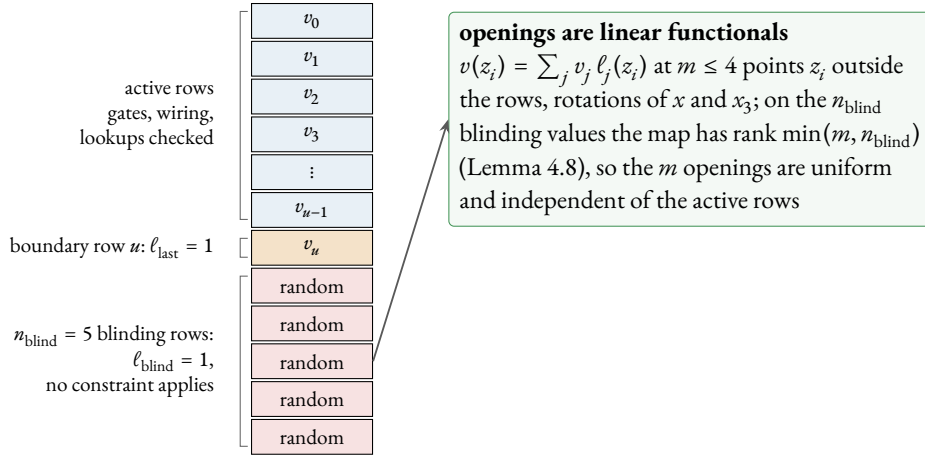


Figure 13. One column of the deployed table: the $u = n - 6 = 2042$ active rows $0, \dots, u - 1$ (blue) on which every argument is checked, the boundary row u selected by ℓ_{last} (amber) that pins the running products, and the $n_{\text{blind}} = 5$ blinding rows (red) selected by ℓ_{blind} , filled with uniform random values and exempted from every constraint by the active-row factor of Remark 2.17. Each opening of the column at a point outside the rows is a linear functional of all n row values, and Lemma 4.8 says the blinding rows alone make at most n_{blind} such openings jointly uniform.

is a bijection at $m = n_{\text{blind}}$ and fails first at $m = n_{\text{blind}} + 1$, so $n_{\text{blind}} \geq m$ is the condition, and the deployed $n_{\text{blind}} = 5$ exceeds the at most $m = 4$ openings of any column by one. One hypothesis of the lemma is that the points lie outside the rows: an opening *at* a row would return a cell, witness or blinder, and the halo2 book’s *Protocol Description* chapter accordingly excludes challenges in the domain (its note 5). The deployed transcript squeezes full field elements with no rejection, and Corollary 4.10 bounds the effect.

Scope of the lemma. It is a *marginal* statement about one column. The proof opens many related polynomials under shared challenges, and the joint distribution of all their evaluations, subject to the public relations among them, the gate equation, the running-product updates, the lookup identities, is not a consequence of the lemma applied column by column. The composition is addressed under “The simulator” below, and its status is stated there.

The quotient and the random polynomial. The quotient chunks admit no blinding-row treatment: C determines them entirely, and the rows of h mean nothing. Their commitment blinders, together with the blinding-row randomness they inherit through C , protect them instead, and their single collapsed opening at x is the publicly computable value $C(x)/(x^n - 1)$, which reveals nothing the verifier did not already hold. There is one more place the quotient could leak: the multipoint reduction evaluates its $\{x\}$ -group polynomial q_1 at the fresh point x_3 , and q_1 contains the collapsed quotient, whose value at x_3 is not a public function of anything. This is why phase 4 commits a uniformly random polynomial before γ and opens it at x alongside the quotient: it joins the same group with the same x_1 -weighting, so $q_1(x_3)$ carries a uniform summand, and the value $\tilde{q}_1$ the prover sends is uniform whatever the quotient’s value at x_3 is. The protocol commits that random polynomial for this reason (the halo2 book’s *Protocol Description* chapter,

step 3 and note 2).

Composition. In each phase the verifier sees only two kinds of object: hiding commitments, which are uniform group elements, and a bounded number of evaluations of the committed polynomials at the transcript’s challenge points. The blinding rows supply randomness which, by Lemma 4.8, makes the openings of each column uniform on its own; a full composition proof must show that this randomness masks the *joint* evaluations subject to the public relations, and must account for the correlations among advice, products, quotient and multipoint polynomials. The final inner-product opening uses the deployed mask of §3.8, the random polynomial vanishing at x_3 whose commitment S opens phase 7, in addition to the commitment and round blinders, because Remark 3.7 showed that a final commitment blinder alone would not hide the folded coefficient; the opening of one polynomial was proved statistically honest-verifier zero knowledge there, and that proof is one ingredient of the composition, not the composition.

The simulator. Zero knowledge requires a simulator for the *complete joint transcript*, given only the public input. The usual proof samples suitably blinded commitments and evaluations satisfying the verifier’s equations and then programs the random oracle so that the Fiat–Shamir challenges come out as required, in the manner of the *Crypto Guide*’s one-round simulator (§“The Fiat–Shamir transform: from interactive to non-interactive”). For the interactive protocol the halo2 book exhibits such a simulator and claims the following; this volume records the claim and does not reprove it.

Proposition 4.9 (Zero knowledge of the interactive protocol, cited). *The interactive Halo 2 protocol is perfect special honest-verifier zero knowledge, the simulator being given the verifier’s challenges in advance (the Crypto Guide, §“Sigma-protocols”), for challenge tuples in which every challenge is nonzero and outside the rows and every factor $1 + u_\ell x_3^{2^{\ell-1}}$ of $g_{\text{dep}}(x_3)$ in (26) is nonzero (the halo2 book’s Protocol Description chapter, “Zero-knowledge and Completeness”). The simulator commits to uniformly random polynomials in place of every polynomial the prover commits in phases 1 to 3 (the book’s a_i : the advice columns, the permuted lookup columns and the running products), the quotient chunks and the multipoint polynomial, and uses its foreknowledge of ξ to choose the mask $s(X)$ so that $p + \xi s - v$ vanishes at x_3 .*

Corollary 4.10 (Unrestricted challenges). *Let every challenge be uniform in $\mathbb{F}$ with no value excluded, as the deployed transcript draws them. Then the transcripts of the interactive protocol and of the simulator of Proposition 4.9, run on the same challenges, are within statistical distance $N_{\text{ch}}(n+2)/|\mathbb{F}|$, where $N_{\text{ch}} = 22$ is the number of challenges, $\theta, \beta, \gamma, y, x, x_1, \dots, x_4, \xi, \zeta$ and the $k = 11$ values u_ℓ ; for $n = 2^{11}$ this is $45100/p_{\text{Pallas}} \approx 2^{-238.5}$.*

Proof. Each challenge has at most $n+2$ excluded values: 0, the n rows, and, for u_ℓ , the value that makes its factor of $g_{\text{dep}}(x_3)$ zero. The challenges have the same distribution in both experiments; on the event that none takes an excluded value the two transcripts are identically distributed by Proposition 4.9, and by a union bound that event fails with probability at most $N_{\text{ch}}(n+2)/|\mathbb{F}|$, which therefore bounds the statistical distance. $\square$

Perfect hiding of each commitment and Lemma 4.8 are ingredients of that argument, not by themselves a proof that the joint distribution matches a real execution. For the compiled argument, the *Crypto Guide*’s compilation recipe (Theorem “The compilation recipe, informal”, §“SNARKs: succinct non-interactive arguments of knowledge”) yields zero knowledge when the polynomial IOP is properly blinded and the corresponding simulation theorem applies; for a multi-round protocol that is a multi-round simulation theorem in the programmable random-oracle model, which this volume neither proves nor cites from a source that proves it for this protocol. Non-interactive zero knowledge of the deployed proof is therefore stated, not proved.

The three properties. Subject to the hypotheses just named, Halo 2 targets three guarantees for the relation $\mathcal{R}_C$ of a PLONKish circuit (Definition 2.7), the three that §1.2 demanded of a proof. *Overwhelming completeness*: an honest prover holding a satisfying witness convinces the verifier except on negligible transcript degeneracies, a zero inner-product challenge (§3.8, probability $1/|\mathbb{F}|$ per challenge), a point x_3 among the query points of the multipoint reduction (“Completeness” in §4.4), an evaluation point x on a row, where the verifier’s division by $x^n - 1$ in phase 5 is undefined (probability $n/|\mathbb{F}|$), or challenges β, γ at which a running-product denominator factor vanishes at an earlier row than any numerator factor does (Remark 2.17; probability at most $N/|\mathbb{F}|$ for the permutation argument on N participating cells and $2u/|\mathbb{F}|$ per lookup on u active rows). *Knowledge soundness*: an efficient extractor, given rewindable access to any prover the verifier accepts with non-negligible probability, outputs a satisfying witness, so that an accepted proof certifies that one is *known*. Corollary 3.13 supplies the extractor for the opening layer; §5 lifts it to the non-interactive argument, whose challenges are transcript hashes, and classifies what that lifting establishes. *Zero knowledge*: a witness-free simulator reproduces the proof, exactly for restricted challenges (Proposition 4.9) and within the distance of Corollary 4.10 for the deployed ones. Knowledge soundness is the property of Action proofs that the Orchard balance and nullifier arguments consume, together with the binding signature (protocol specification, § 4.14, “Balance and Binding Signature (Orchard)”) and the consensus nullifier rules (protocol specification, § 3.9, “Nullifier Sets”); zero knowledge is the property the privacy argument consumes.

4.7 Halo 2 as a compiled polynomial IOP

The protocol of §4.5 factors into two layers: a *polynomial interactive oracle proof* for the relation, and a polynomial commitment scheme, in the sense of the *Crypto Guide* (§“Polynomial commitment schemes”), that realises the proof’s oracles. Every phase of the protocol belongs to one of the two layers.

The polynomial IOP, recalled. The *Crypto Guide* defines the object (§“SNARKs: succinct non-interactive arguments of knowledge”, under “Polynomial interactive oracle proofs”): a public-coin interactive protocol in which, in place of ordinary messages, the prover in each round sends one or more *oracles* for polynomials $p_i \in \mathbb{F}[X]$ of bounded degree, the verifier answers with uniformly random field elements, and at the end the verifier *queries* each oracle at a small number of points, receiving the evaluations, and accepts or rejects as a deterministic function of the challenges, the evaluations and the public input. Completeness, soundness and knowledge soundness are those of interactive arguments, with oracle access in place of reading the messages,

and the soundness analysis is purely algebraic, resting on the Schwartz–Zippel lemma. The one respect in which a polynomial IOP differs from a vanilla interactive argument is that the verifier never reads a prover message in full: it queries a few evaluations of each oracle. That is what renders the verifier succinct even when the polynomials have large degree; the message length is never read, only as many field elements as there are queries.

The compiler, recalled. The *Crypto Guide*’s compilation recipe (the same section) turns a polynomial IOP into a non-interactive argument with a polynomial commitment scheme (Setup, Commit, Open, Eval, Verify) and a random oracle H . Setup runs the commitment scheme’s setup. The prover simulates the IOP, replacing each oracle message p_i by its commitment C_i , derives every verifier challenge from the transcript by the Fiat–Shamir transform of §4.3, and supplies, for each of the IOP’s queries, the claimed evaluation together with an evaluation proof (Eval). The verifier parses the transcript, recomputes the challenges, checks each evaluation proof (Verify), and runs the IOP’s accept-or-reject check on the opened evaluations (Figure 14).

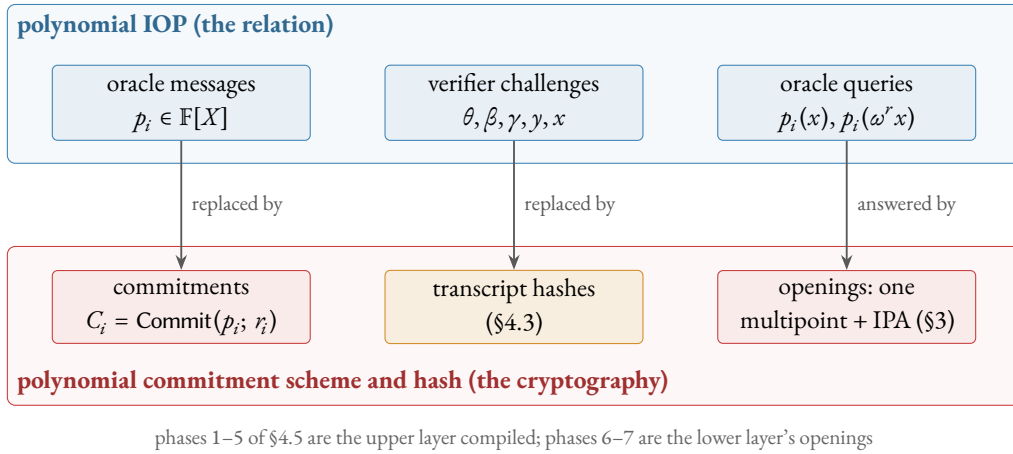


Figure 14. The compiler. The polynomial IOP (blue) is the information-theoretic layer: oracles, public-coin challenges and evaluation queries, sound by the Schwartz–Zippel lemma alone. The cryptographic layer (red and amber) realises it: each oracle becomes a commitment, each challenge a transcript hash, each query an opening, and Halo 2 collapses every opening into one. The two layers are separable: the same IOP compiles with any commitment scheme meeting the hypotheses of Remark 4.11.

PLONK as a polynomial IOP. The IOP that Halo 2 compiles is PLONK’s, in the PLONKish generality of §2.5. Its oracles, sent by the prover, are one polynomial per advice column; for each lookup, the two permuted columns A' and S' ; the permutation running products, one per chunk, and one lookup running product per lookup; and the quotient b , split into chunks below the degree bound. The fixed columns, the selectors among them, and the permuted-label polynomials are *preprocessed*, part of the verifying key rather than prover messages, which is why they are absent from the list. The verifier’s challenges, in order, are θ for the lookup compression, β and γ for the permutation and lookup products, y for the combination of all constraints into one gate equation, and x for the evaluation point. The final check is the gate equation $C(x) = b(x) Z_H(x)$, with C the y -combination of every gate, permutation and lookup constraint polynomial and Z_H the

vanishing polynomial of the rows, which the verifier evaluates from the oracles' values at x and at the rotations its constraints read: ωx or $\omega^{-1}x$ for an adjacent row, and $\omega^{-6}x$, the rotation to the boundary row of Remark 2.17, for the chunk-boundary identities of the permutation argument.

The map onto the phases. Phases 1 to 5 of §4.5 are this IOP compiled, with one addition: the commitments to the advice, the permuted lookup columns, the running products and the quotient chunks are the oracle messages replaced by commitments, and the random polynomial of phase 4, committed and evaluated at x , is the compiler's addition for zero knowledge (§4.6), not an IOP message; the challenges are the IOP's challenges squeezed from the transcript, and the claimed evaluations of phase 5 answer the IOP's oracle queries. Phases 6 and 7 bind those answers to the commitments: the multipoint reduction of §4.4, with its challenges $x_1, \dots, x_4$ and combined polynomial f^* , followed by the single inner-product opening of f^* . That is the bridge between the abstract compiler and the concrete protocol, and the phase numbering is the one Table 10 uses. Under the hypotheses of the *Crypto Guide*'s compilation recipe (Theorem “The compilation recipe, informal”) and of Remark 4.11, the compiled protocol has the IOP's relation and the commitment scheme's assumptions, transparency, proof size and verifier cost.

Remark 4.11 (The compilation principle, and what it does not say). Soundness of the compiled argument is not the sum of two numbers, and the notions under which it is proved deserve their names. A polynomial IOP is *round-by-round* knowledge sound, informally, when its partial transcripts admit a “doomed” labelling that a prover message cannot escape except with bounded probability over the next verifier challenge; a related *state-restoration* notion lets a prover restore an earlier verifier state and redraw its challenge, which is precisely what a prover attacking a transcript hash can do. Precise equivalence or implication theorems between the two notions require additional conditions on the protocol and on the extractor, and this volume does not identify them. The *Crypto Guide*'s compilation recipe (Theorem “The compilation recipe, informal”, §“SNARKs: succinct non-interactive arguments of knowledge”) assumes a complete, knowledge-sound public-coin polynomial IOP; a correct, evaluation-binding and extractable commitment scheme; compatible composition hypotheses; and a suitable multi-round Fiat–Shamir theorem in the random-oracle model. For a multi-round protocol such as Halo 2 this volume reads those hypotheses as three: the IOP has a round-by-round or state-restoration knowledge-soundness theorem; the commitment scheme is extractable and evaluation-binding in a form compatible with it; and the transcript satisfies the hypotheses of an appropriate multi-round Fiat–Shamir theorem. The conclusion is that the compiled protocol is a non-interactive argument of knowledge in the random-oracle model, by layered extraction: the commitment scheme's extractor recovers the committed polynomials, converting the argument's prover into an IOP prover; the multi-round Fiat–Shamir theorem transfers the round-by-round or state-restoration guarantee to the non-interactive setting; and evaluation binding ensures that the opened evaluations are those of the extracted polynomials. What it does *not* say is that the knowledge error is determined by two scalars, the IOP's knowledge error and the commitment scheme's extraction error, added together: the exact error and the query loss depend on the definitions and on the extraction model, and §5 names the theorems that fix them for the analysed protocol and classifies their instantiation to the deployed transcript.

Delegated results. The subsection proves nothing afresh. The commitment interface (Setup, Commit, Open, Eval, Verify) with its hiding, binding and extraction properties is the *Crypto Guide*’s (§“Polynomial commitment schemes”); the Fiat–Shamir transform and its random-oracle security are the same volume’s (§“The Fiat–Shamir transform: from interactive to non-interactive”); the definitions of completeness, soundness and knowledge soundness for interactive arguments used above are its (§“Interactive proofs, zero knowledge, and SNARKs” and §“Knowledge soundness and extractors”); and the Schwartz–Zippel lemma and the vanishing polynomial of the rows are the *Math Guide*’s (§“The Schwartz–Zippel lemma” and §“The evaluation domain H and its vanishing polynomial”). The volume adds the instantiation: which oracles, which challenges, which openings, and the one commitment scheme that realises them.

5 A rigorous treatment of knowledge soundness

Theorem 3.5 and Corollary 3.13 proved the inner-product opening knowledge sound for a verifier who draws his challenges: an extractor that runs the prover as a subroutine, rewinds it and feeds it fresh challenges recovers the committed vector or a nontrivial discrete-logarithm relation among the generators, which the hardness of the discrete logarithm on Vesta excludes. The deployed verifier draws nothing; every challenge is a hash of the transcript (§4.3), and Remark 4.11 named the hypotheses under which a compiled argument inherits its layers’ guarantees without fixing them. This section determines what the extractor establishes when every challenge is a hash of the transcript. It combines the rewinding-extractor framework of the *Crypto Guide*, the definition of knowledge soundness (§“Knowledge soundness and extractors”), the tree extraction of Sigma-protocols (§“Sigma-protocols”) and the Fiat–Shamir theorem with its stated caveats (§“The Fiat–Shamir transform: from interactive to non-interactive”), with the algebraic group model, named among the idealisations of §“The standard model and the random oracle model”. It lifts the interactive extractor to the non-interactive argument in two layers and names the loss of the naive lifting; it states the model in which the halo2 book proves its extraction theorem and classifies what that theorem does and does not establish for the deployed transcript; and it closes on the gap between concrete and asymptotic security. The running-time vocabulary, polynomial, negligible and PPT, is the *Math Guide*’s (§“Polynomial, exponential, and negligible functions” and §“Algorithms, running time, and PPT”); expected polynomial time is the *Crypto Guide*’s (§“Adversaries and the security parameter”, the remark on expected versus strict polynomial time); tightness and loss are the *Crypto Guide*’s (§“Reductions”, §“Concrete security and the bit-security budget”); the assumption is its standing one (§“Standing assumptions”).

5.1 From the interactive extractor to the non-interactive argument

What the interactive extractor costs. Theorem 3.5 extracts from a tree of accepting transcripts with three children at every node and one level per round: 3^k leaves for $k = \log_2 d$ rounds, that is $d^{\log_2 3}$ with $\log_2 3 = 1.58496 \dots$, polynomial in d though more than a binary tree would cost. At the deployed $d = 2^{11} = 2048$ the tree has $3^{11} = 177147$ leaves against the $2^{11} = 2048$ a binary tree would have; the extractor runs in expected polynomial time, the rewinding that assembles the tree being the *Crypto Guide*’s (§“Sigma-protocols”, the tree-extraction theorem cited

there). Its knowledge error is the $O(\log d/|\mathbb{F}|)$ of Theorem 3.5, one $O(1/|\mathbb{F}|)$ term per round for the challenges the siblings must avoid. The scale of these terms is worth fixing once: the proof system’s own allowances are linear in $n = d$, the $18431/|\mathbb{F}| \approx 2^{-239.8}$ of the identity check (§2.4), whose difference polynomial $C - bZ_H$ has degree at most $d_{\max} n - 1 = 18431$ at the deployed $d_{\max} = 9$ ($\deg C \leq d_{\max}(n - 1)$ by the proof of Theorem 4.7, and the $d_{\max} - 1$ committed chunks of the quotient have degree $< n$ each, §4.5); the order- $n/|\mathbb{F}|$ term of the multipoint reduction (Lemma 4.5); and $d/|\mathbb{F}| = 2^{11}/2^{254} = 2^{-243}$ is that linear allowance for the opening. The opening’s proved error sits below it, and quoting the linear figure for the opening loses nothing in a system whose other terms are already linear in d , as the proof of Theorem 3.5 noted. Every one of these is a Schwartz–Zippel term, not a security level, in the sense fixed in §1.2.

Layered extraction. The compiled argument of §4.7 was assembled in two layers, an information-theoretic polynomial IOP and a cryptographic commitment scheme, and its extractor is assembled in the same two layers, run backwards (Figure 15).

The lower layer recovers polynomials. The prover has sent commitments C_v and, in phases 6 and 7, one opening of the combined $F^* = F + \sum_i [x_4^i] Q_i$ with $Q_i = \sum_e [x_1^e] C_{v_{ie}}$ (Construction 4.4). Phase 7 runs the deployed opening of §3.8, and the extractor of Corollary 3.13, the tree extraction of Theorem 3.5 with two further levels, ξ and ζ , the latter in the root role of the resampled U , returns the coefficient vector behind F^* , a representation of F^* over $(\mathbf{G}, H)$, or a nontrivial discrete-logarithm relation among $\mathbf{G}$, H and U . To reach the individual commitments the extractor rewinds over the combining challenges as well: the vectors behind F^* for $m + 1$ distinct values of x_4 are $m + 1$ linear equations in the $m + 1$ unknown vectors behind F , $Q_1, \dots, Q_m$, whose coefficient matrix has the rows $(1, x_4, \dots, x_4^m)$ and is invertible because a nonzero polynomial of degree at most m has at most m roots (*Math Guide*, §“Roots and the factor theorem”), the argument of the proof of Lemma 3.4; the vectors behind Q_i for $|I_i|$ distinct values of x_1 likewise yield the vectors behind each C_v in group i . One C_v is the collapsed quotient $\sum_i [x^{ni}] \text{Commit}(b_i)$ of §4.5; its vectors for $d_{\max} - 1$ values of x with pairwise distinct x^n yield the vectors behind the chunks b_i , by the same argument with x^n in place of x_4 . The output is one polynomial of degree $< n$ per commitment the prover sent, and two different vectors extracted for one commitment would be a nontrivial discrete-logarithm relation among $\mathbf{G}$ and H , exactly as at the root of Theorem 3.5. This layer is the only place the discrete logarithm on Vesta enters, and it enters in only one way: the extractor either recovers the polynomials or exhibits a relation.

The upper layer recovers the witness. With the polynomials in hand, the prover’s claimed evaluations are the true evaluations of fixed polynomials except with the Schwartz–Zippel allowance of Lemma 4.5, since a claimed value that differs from the extracted polynomial’s value survives the reduction only with that probability; and “the true evaluations of fixed polynomials” is precisely the hypothesis under which §2 proved its theorems. Theorem 2.5 turns the accepted identity $C(x) = b(x) Z_H(x)$ into divisibility of C , Lemma 4.6 splits it into the vanishing of every constraint polynomial on the rows, Theorem 2.11 with Corollary 2.12 turns the running products into the wiring, and Theorem 2.15 with Proposition 2.16 turns the lookup identities into table membership, each with its stated term over $|\mathbb{F}|$; Theorem 2.2 reads gate satisfaction off divisibility. The witness is then read off directly: the advice columns are the values of the extracted advice polynomials on the rows. Nothing in this layer uses an assumption; the IOP’s extractor outputs the

oracles, which the layer below supplies. The consequence runs both ways. A break of the discrete logarithm on Vesta breaks the lower layer and nothing else, in the way §3.5 described, one commitment opening to any vector; and no assumption can rescue an identity that the upper layer does not actually enforce, because the upper layer's errors are counting statements about polynomials.

What is proved and what is described. In the interactive setting the composed extractor is a tree extractor in the sense of the *Crypto Guide* with a wider tree: three children at each of the k inner-product levels, two at the ζ level and two at the ξ level, $m + 1$ at the x_4 level, $\max_i |I_i|$ at the x_1 level, one x_1 serving every group, and, at the x level, $(d_{\max} - 2)n + 1 = 14337$ children: each value of x^n has at most n preimages, so only that many distinct challenges guarantee the $d_{\max} - 1$ pairwise distinct x^n the quotient step needs, as Theorem 3.5 took five children for three distinct squares. The two ζ children must moreover be nonzero, as in Corollary 3.13. Each polynomial the upper layer consumes is fixed by a binding commitment sent before every challenge that tests it: the advice before θ , the permuted lookup columns A' and S' before β and γ , the running products and the random polynomial before y , the quotient chunks before x (§4.5); the polynomial behind F and the mask behind S , committed after x , serve only inside the lower layer; and all of the consumed ones precede x , the first challenge at which the tree branches, so the upper layer's Schwartz–Zippel arguments apply on every branch without further rewinding. The inner-product part of that tree is Corollary 3.13, proved; the theorems of the upper layer are proved in §2; the accounting that joins them into one tree-extraction statement with one knowledge error is the compilation principle of Remark 4.11, whose hypotheses this volume names and does not discharge. In particular the compiled error is not asserted to be the sum of the layers' errors.

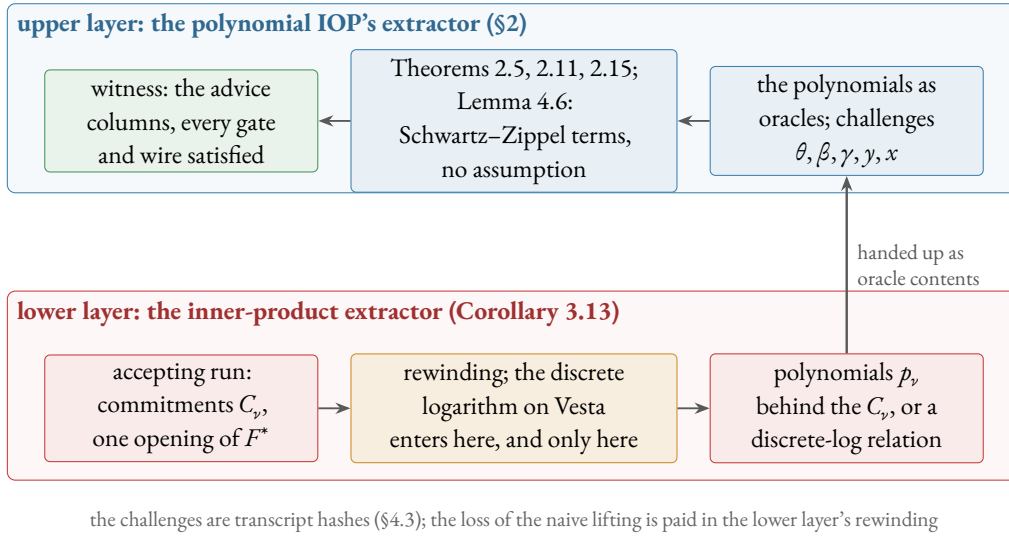


Figure 15. Layered extraction. The lower layer (red) is the cryptographic one: from an accepting run, the extractor of Corollary 3.13, rewinding over the opening's challenges, the combining challenges of the multipoint reduction and the evaluation challenge x , recovers the polynomial behind every commitment or exhibits a discrete-logarithm relation; the assumption (amber) lives here and nowhere else. The upper layer (blue) is the information-theoretic one: with the polynomials as oracle contents, the theorems of §2 read the witness (green) off them with Schwartz–Zippel terms only.

The naive lifting, and its loss. Everything above is interactive: the extractor chose the challenges it fed the prover. In the deployed argument the challenges are hashes, and the extractor of the *Crypto Guide*'s Fiat–Shamir theorem obtains its second transcript by *forking*: it finds, among the prover's at most Q hash queries, the critical one whose answer became the challenge, rewinds to it, and answers it afresh, succeeding with probability about $\varepsilon(\varepsilon/Q - 1/|\mathbb{F}|)$, so that the extractor's success is the prover's squared and divided by Q (§“The Fiat–Shamir transform: from interactive to non-interactive”, the knowledge-soundness clause of its theorem). That theorem covers one challenge. The tree of Theorem 3.5 needs three siblings at each of k levels, and a sibling at level j must share the transcript prefix through level $j - 1$, which in the hashed setting means agreeing with the prover's prefix and its earlier hash answers: in a transcript hash, changing one message redraws *all* later challenges, so siblings at round j can be found only among the prover's own queries. Let μ be the number of verifier challenges of the protocol being lifted. The simplest lifting guesses, among the at most Q queries, the μ that answered them, and runs the interactive extractor against the interactive prover that the guess defines; the guess is right with probability at least $Q^{-\mu}$, so the knowledge error is multiplied by Q^μ . Attema, Fehr and Klooß (*Fiat–Shamir Transformation of Multi-Round Interactive Proofs*) record this Q^μ as the loss for a general $(2\mu + 1)$ -move protocol; nesting the one-round forking level by level fares worse, since each level squares the success probability before dividing by Q . For the opening alone μ exceeds $k = \log_2 d$, the choice of U (in the deployed opening, ξ and ζ) being a challenge too, and μ grows once the proof system's own challenges are counted. The loss belongs to this analysis. For $(k_1, \dots, k_\mu)$ -special-sound protocols (*Crypto Guide*, §“Sigma-protocols”, the definition of tree special soundness) the same authors prove, in the random oracle model and with no idealisation of the group, a knowledge error that grows only linearly in Q . The rounds of the opening's extraction tree have that shape, but its root step, which rewinds the choice of U (in the deployed opening, ξ and ζ), and its alternative output, a discrete-logarithm relation, are not checked here against that theorem's hypotheses, and this volume does not assess that route.

With the deployed $k = \log_2 d = 11$, so $\mu > 11$ for the opening alone, and a query budget of $Q = 2^{80}$ hash evaluations, the factor Q^μ exceeds $Q^{11} = 2^{880}$; multiplied by the linear allowance $d/|\mathbb{F}| = 2^{-243}$ it exceeds 2^{637} , a bound above 1, which is no bound at all. Nor does the proved $O(\log d/|\mathbb{F}|)$ help: the challenge space is a 255-bit field with fewer than 2^{255} elements, so 2^{880} exceeds $|\mathbb{F}|$ itself, and a factor larger than $|\mathbb{F}|$ turns every term of the form $c/|\mathbb{F}|$ with $c \geq 1$ into a number above 1. Asymptotically the loss falls elsewhere. With Q and d polynomial in the security parameter λ (*Crypto Guide*, §“Adversaries and the security parameter”) and $|\mathbb{F}| = 2^{\Theta(\lambda)}$, the factor $Q^\mu = 2^{O(\log^2 \lambda)}$ is super-polynomial and sub-exponential in the *Math Guide*'s classification (§“Polynomial, exponential, and negligible functions”). The Schwartz–Zippel terms survive it: $Q^\mu O(\log d)/|\mathbb{F}| = 2^{O(\log^2 \lambda) - \Theta(\lambda)}$ is negligible. The reduction does not survive it: the guessing extractor succeeds only with probability reduced by Q^μ , or runs Q^μ times longer to compensate, and so is not PPT, and the prover's success is then bounded only by Q^μ times the discrete-logarithm advantage, which need not be negligible when that advantage is merely negligible. The interactive extractor is polynomial, $d^{\log_2 3}$ transcripts in expected polynomial time; the looseness arises in the naive analysis of the passage to the non-interactive argument.

What the tighter notions are for. Remark 4.11 named the two notions built to avoid this: *round-by-round* knowledge soundness, which labels partial transcripts as doomed and bounds the probability that one verifier challenge lifts the doom, and *state-restoration* soundness, which lets the prover restore an earlier verifier state and redraw its challenge, precisely the freedom a prover attacking a transcript hash has. Each exists to obtain a bound on the compiled argument that loses a factor linear in Q rather than Q^ϵ ; no equivalence between them is asserted here, as none was there. Claiming a single factor Q for a compiled protocol nevertheless requires the precise theorem’s state-restoration, transcript, challenge and extraction hypotheses to hold of that protocol, and this volume does not assert a factor- Q bound for Halo 2’s compiled protocol. The next subsection names the model in which the halo2 book proves such a bound, in the state-restoration form, for its abstract interactive protocol, what that analysis assumes, and why its instantiation to the deployed transcript remains unwritten.

5.2 The algebraic group model

The nested forking of the previous subsection rewinds because the extractor has no other way to learn what is behind a group element: three accepting continuations are the price of one representation. The *algebraic group model* of Fuchsbauer, Kiltz and Loss (*The Algebraic Group Model and its Applications*) removes the need by restricting the adversary’s group operations, and it enables tighter analyses for protocols that suit it. It is not by itself a theorem eliminating every Fiat–Shamir loss; what it eliminates is the rewinding, and what remains must still be proved.

Definition 5.1 (Algebraic adversary). An *algebraic adversary* against a group G is a PPT algorithm that, whenever it outputs a group element $P \in G$, additionally outputs a vector $\mathbf{a}$ of scalars with

$$P = \langle \mathbf{a}, \mathbf{Y} \rangle,$$

where $\mathbf{Y}$ is the vector of group elements it has previously seen: the generators it was given and the elements it has received. The vector $\mathbf{a}$ is the *representation* of P .

Where the model sits. The algebraic group model sits between the standard model, in which the adversary is any PPT algorithm and every primitive is concrete code it may inspect (*Crypto Guide*, §“The standard model and the random oracle model”), and the generic group model, in which the adversary may perform group operations only through an oracle on opaque handles and never exploits the representation of an element (the same section, the remark on the generic and algebraic group models). An algebraic adversary is less restricted than a generic one, since it may compute on the bit representation of group elements however it likes; it must only be able to explain each element it outputs as a combination of elements it has seen. The model is an idealisation (the same remark): a proof in it says nothing about an adversary that outputs a group element it cannot explain. Figure 16 draws the three models as nested restrictions on the adversary and places this section’s analyses on them; the random oracle model, which idealises the hash rather than the group, is an independent axis, and an analysis may combine either group model with it.

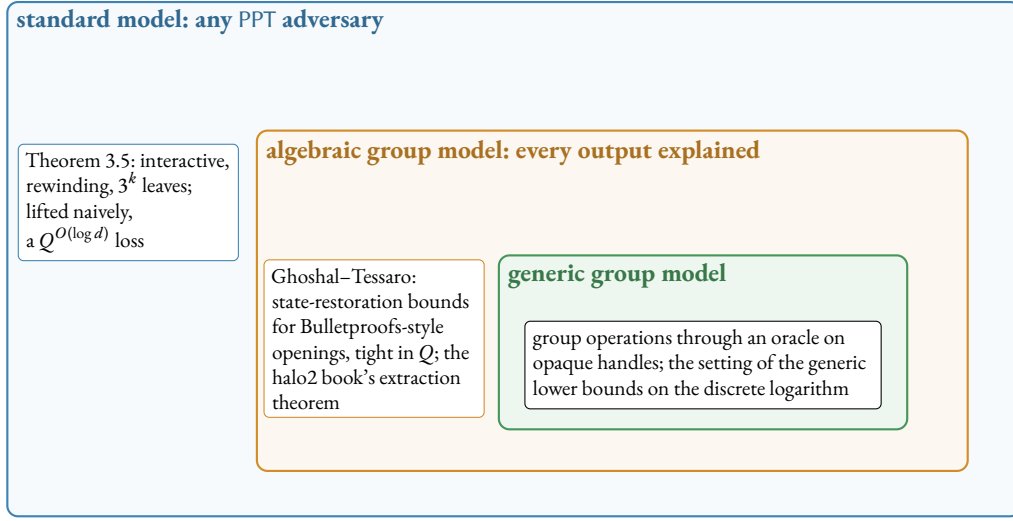


Figure 16. Three models as nested restrictions on the adversary. Every adversary admitted by an inner ring is admitted by the rings around it, so a proof in an outer ring is the stronger statement and a bound in an inner ring is the easier to make tight. The interactive extractor of Theorem 3.5 lives in the standard model for uniform generators; the tight state-restoration bounds of Ghoshal and Tessaro, and the halo2 book’s extraction theorem that invokes them, live in the algebraic group model (amber). The generic group model (green) is where the discrete logarithm’s own lower bounds are proved.

Online extraction in the algebraic group model. In this model the prover outputs every group element with its representation, and the extractor reads the witness off the representations of a single accepting transcript instead of assembling it from rewind ones; no rewinding occurs, and the price is the assumption that the prover is algebraic. For the deployed opening, equation (29), the halo2 book’s *Protocol Description* chapter carries this out (“Witness-extended Emulation”, the definition of the bad-challenge sets): comparing the two sides of the accepting equation coordinate by coordinate either exhibits a nontrivial discrete-logarithm relation among the generators or, unless some challenge falls in a set of *bad challenges* fixed by the messages before it, forces the representation of P to open to the claimed value. The analysed protocol draws every challenge from one space $\mathcal{C}$ of field elements, none zero or a row, and each round challenge u_j further so that its factor $1 + u_j x_3^{2^{j-1}}$ of $g_{\text{dep}}(x_3)$ in (26) is nonzero. Each round challenge u_j has at most $4n$ bad values in $\mathcal{C}$, and ζ and ξ have one each; the book bounds the bad sets of the proof system’s other challenges, $x_1, \dots, x_4, x$ and those of the committed rounds, by the degrees of the identities they test, the largest being $n_g(n-1)$, the bound on the constraint polynomial’s degree in each indeterminate, provided $n_g(n-1) \geq 4n$, as at the deployed $n_g = 9$, where $18423 > 4n = 8192$; the book admits $n_g \geq 4$, for which this can fail.

Ghoshal and Tessaro, and the deployed stance. Ghoshal and Tessaro (*Tight State-Restoration Soundness in the Algebraic Group Model*) carry this analysis out for Bulletproofs-style inner-product arguments: they prove tight state-restoration soundness in the algebraic group model and derive Fiat–Shamir security bounds from it under their stated hypotheses, the transfer from the state-restoration game to the compiled argument being their theorem. The deployed

analysis follows them. The protocol specification defines the proving system by reference to the halo2 book (protocol specification, § 5.4.10.3, “Halo 2”), and the book’s *Protocol Description* chapter fixes the framework: it adopts state-restoration soundness because the concrete protocol is realised through the Fiat–Shamir transformation and a cheating prover can fork the transcript at will, and it invokes Ghoshal and Tessaro’s theorem for the passage from that game to soundness of the compiled argument. Its abstract interactive protocol is stated for a relation of the form “a fixed multivariate constraint polynomial in the column polynomials and the challenges vanishes on the rows”, with the column polynomials as the witness, over public parameters of $n + 2$ group elements, $\mathbf{G}$, U and the blinding generator H (written W there).

Definition 5.2 (State-restoration witness-extended emulation). Let Π be a public-coin interactive argument for a relation $\mathcal{R}$ with r challenges from a space $\mathcal{C}$. A *state-restoration prover* $\mathcal{P}^*$, given the public parameters, outputs a statement ϕ and then queries an oracle on pairs (τ, a) , where τ is a transcript prefix the oracle has recorded (initially only the empty one) and a is a next prover message; it may thus extend any earlier prefix. In the *real* game the oracle answers a message of rounds 1 to r with a fresh uniform $c \in \mathcal{C}$ and records (τ, a, c) , and answers a final message with the verifier’s decision on (τ, a) , recording (τ, a) . In the *ideal* game an algorithm $\mathcal{E}$, given the public parameters and ϕ and keeping state across queries, receives each query (τ, a) , for an algebraic prover together with the representations (Definition 5.1) of the group elements in a , returns the answer, which is recorded with the query, and at the end outputs w . For a distinguisher $\mathcal{D}$ given the recorded transcripts,

$$\text{Adv}_{\Pi, \mathcal{R}}^{\text{srwee}}(\mathcal{P}^*, \mathcal{D}, \mathcal{E}) := \Pr[E_{\text{real}}] - \Pr[E_{\text{ideal}}],$$

where E_{real} is the event that $\mathcal{D}$ outputs 1 in the real game, and E_{ideal} the event that $\mathcal{D}$ outputs 1 in the ideal game and, if an accepting transcript was recorded, $(\phi, w) \in \mathcal{R}$.

Theorem 5.3 (Extraction for the abstract protocol; interface). *Let Π be the halo2 book’s abstract interactive argument for its relation $\mathcal{R}$, over a group $\mathbf{G}$ of prime order with scalar field $\mathbb{F}$, for n rows, a positive integer n_g with $n_g(n - 1) \geq 4n$ and a constraint polynomial of degree at most $n_g(n - 1)$ in each indeterminate, with challenges from $\mathcal{C}$. There is an extractor $\mathcal{E}$ such that for every non-uniform algebraic state-restoration prover $\mathcal{P}^*$ making at most Q queries to its oracle there is an adversary $\mathcal{B}$ with*

$$\text{Adv}_{\Pi, \mathcal{R}}^{\text{srwee}}(\mathcal{P}^*, \mathcal{D}, \mathcal{E}) \leq Q \varepsilon_{\text{bad}} + \text{Adv}_{\mathbf{G}, n+2}^{\text{dlrel}}(\mathcal{B}), \quad \varepsilon_{\text{bad}} \leq \frac{n_g(n - 1)}{|\mathcal{C}|},$$

for every computationally unbounded distinguisher $\mathcal{D}$. Here ε_{bad} bounds the fraction of bad challenges after every partial transcript, and $\text{Adv}_{\mathbf{G}, n+2}^{\text{dlrel}}(\mathcal{B})$ is the probability that $\mathcal{B}$, given $n + 2$ uniform elements of $\mathbf{G}$, outputs a nontrivial discrete-logarithm relation among them.

The theorem is quoted as an interface and not proved in this volume: it is the halo2 book’s (*Protocol Description*, “Witness-extended Emulation”), proved there by instantiating Theorem 1 of Ghoshal and Tessaro with the bad-challenge sets described above. Its extractor works online, from the representations of a single transcript, and never rewinds. At the deployed $n_g = d_{\text{max}} = 9$

and $n = 2^{11}$, $\varepsilon_{\text{bad}} \leq 18423/|C| \approx 2^{-239.8}$, and $Q \varepsilon_{\text{bad}} \approx 2^{-159.8}$ at $Q = 2^{80}$ queries.

The instantiation gap, classified. Applying those bounds to the deployed Halo 2 transcript requires matching the precise folding, masking, challenge distribution and composition assumptions of the analysed protocol to the deployed one, and this volume does not prove that instantiation theorem. Four mismatches are visible from the earlier sections. The book’s abstract relation sees the permutation and lookup arguments of §2 only through the constraint polynomial and its committed columns; the Orchard circuit’s actual gates and table columns (§4.1) and its chunked running products (“The deployed chunking” in §2.6) instantiate that abstraction and must be checked against its degree hypotheses. The book’s challenges exclude zero, the rows and, for each u_j , the value that zeroes its factor of $g_{\text{dep}}(x_3)$, while the deployed transcript squeezes full field elements with no rejection (§3.8), so the deployed challenge distribution differs from the analysed one by the excluded events. The analysed folding is the deployed normalisation of Remark 3.9, and the masking is the deployed mask of §3.8, so those two match; the hash, however, is BLAKE2b (§4.3), and the transfer theorem’s random oracle must be instantiated by it. And the book’s theorem is for the interactive argument against a state-restoration prover, the compiled bound following by the transfer theorem under its hypotheses. In the classification of §1.2, the status of the transfer is *designed-but-unspecified*: the bounds exist for the analysed protocol and the design intends them for the deployed one, but the theorem that carries them across is not written, here or in the specification.

5.3 Concrete versus asymptotic security

The *Crypto Guide* distinguishes a reduction’s existence from its tightness (§“Reductions”, the definition of loss and the remark on why loss matters concretely) and reads security as a budget of bits (§“Concrete security and the bit-security budget”): a scheme inherits the strength of its assumption only up to the loss of the reduction that connects them. Two tightness gaps separate the proved statements from the deployment.

The first gap: the naive lifting. The naive Fiat–Shamir knowledge-soundness analysis of the opening loses the factor $Q^u > Q^{\log d}$ of §5.1, exponential in the round count $\log d$ with base the query count Q : above 2^{880} at the deployed size for a 2^{80} -query prover, against a challenge space of fewer than 2^{255} elements. The interactive extraction itself is polynomial, $d^{\log_2 3} = 177147$ transcripts at $d = 2048$ in expected polynomial time, with the proved knowledge error $O(\log d/|F|)$ of Theorem 3.5. The gap lies in the naive analysis of the lifting, and the notions of §5.1 and the model of §5.2 exist to close it.

The second gap: the model. Each analysis cited in this section that avoids the naive loss relies on an idealisation: the halo2 book’s extraction theorem (Theorem 5.3) on the algebraic group model, and its passage to the compiled argument on the random oracle model as well; the special-soundness analysis of Attema, Fehr and Klooß on the random oracle model alone. No standard-model tight bound is claimed for the deployment, by this volume or by the analyses it cites; this is a statement about the design’s analyses, not a specification claim, and the specification makes none. Read against the bit-security budget, the consequence is concrete. The discrete-logarithm layer

offers about 2^{126} operations of classical security (§3.5; *Crypto Guide*, §“Standing assumptions”); a reduction tight in Q lets the argument inherit that strength, less the algebraic term $Q \varepsilon_{\text{bad}}$ of Theorem 5.3, about $2^{-159.8}$ at 2^{80} queries, which is not the bottleneck; a reduction losing 2^{880} lets it inherit nothing. For the analysed protocol the number is therefore the assumption's, in the algebraic group and random oracle models; for the deployed argument it is so only through the instantiation that §5.2 classifies designed-but-unspecified.

The stance. Proved without idealisation of the adversary or of the transcript hash are the interactive statement, Theorem 3.5, for uniform generators (the deployed hashed generators add its hypothesis that hash-to-curve is a random oracle), and the information-theoretic layer of §2; described, with the hypotheses named, are the layered extraction that joins them and the naive lifting's loss; cited, with its instantiation to the deployed transcript classified designed-but-unspecified, is the halo2 book's bound in the algebraic group model.

6 The verifier's decision and its implication chain

This section adds no theorem: it lists the verifier's checks in the order he performs them, assembles the implication chain from an accepted proof back to the claim, each link labelled by the statement that proves it and by its error term, repeats the chain for the Orchard Action circuit, and maps the five reductions of §1.2 to the formal statements that own them. Nothing here depends on the recursion of §7, which Orchard does not deploy.

6.1 The verifier's checks, read in order

What he holds. The verifier of the toy holds the fixed data of the circuit, the five selector polynomials and the permuted-label polynomials of Construction 2.10; the public input, the column $(0, 0, 0, -35)$ interpolated to $\text{PI}(X)$ (§4.2); and the proof. The proof is a byte string (§4.3): commitments, claimed evaluations, and one opening, in the order of the phases of §4.5 (Table 10), counted by kind in Table 8. He never sees a column polynomial, a blinder or the advice; he sees curve points and field elements, and he performs the checks below in this order.

1. **The challenges.** He first commits to the instance column himself, from the public input and with the fixed blinder 1 (§4.2), so that the public input never comes from the prover. He then replays the transcript (§4.3): he absorbs the key digest, that instance commitment and each prover message in turn, and squeezes every challenge at its prescribed point. In the toy the point $z = 20$ was drawn by decree; in the deployed protocol the evaluation point x is a hash of everything before it.
2. **The identity at the point.** From the claimed evaluations and the fixed polynomials he evaluates himself, he forms both sides of the gate identity. In the toy this is the one field equation (5): $G(20) = 75$ against $t(20) Z_H(20) = 67 \cdot 46 = 75$, accepted; the $c_0 = 10$ cheat of Example 2.3 gives 20 against 64, rejected. In the deployed protocol the identity is $C(x) = b(x) Z_H(x)$, with C the y -combination of every constraint polynomial and b the chunked quotient, and it is enforced by opening the collapsed quotient commitment at x to the value $C(x)/Z_H(x)$ that

the verifier computes himself from the evaluations of phase 5, inside the multipoint opening (phases 6 and 7 of §4.5).

3. **The wiring.** He evaluates the boundary and update identities of the permutation argument (7) at the point and, in the deployed form, the chunk-boundary identities of §2.6 and the boundary identity $\ell_{\text{last}}(Z^2 - Z)$ of Remark 2.17. In the toy the ratio of the two products (6) at $\beta = \gamma = 2$ is 1 for the honest trace and 61 for the tampered $b_1 = 4$ (Example 2.8).
4. **The tables.** He evaluates the four lookup identities (9) and, in the deployed form, the boundary identity $\ell_{\text{last}}(Z^2 - Z)$ of Remark 2.17 at the point, five for each lookup argument. The toy's grid has none; the two-bit check of Example 2.13 showed the shape, with both products equal to 82 and the forged 7 stranded in every arrangement. In the deployed protocol items 3 and 4 are not separate checks: their identities are summands of C and are enforced by the one check of item 2; only the toy checks them separately.
5. **The openings.** Every evaluation he received for the three checks above was *claimed*. He now binds each claim to its commitment: the multipoint reduction of Construction 4.4 folds every claim into one statement, and the inner-product equation (29) discharges it with one multi-scalar multiplication of length n . In the toy, the opening of $a(X)$ at $z = 20$ to the value 59 closes at (54, 39) on both sides (Table 5) and the false value 60 moves the left-hand side to (62, 15); the deployed opening of the same commitment closes at (5, 72) and rejects 60 likewise (Table 7). Among the claims is $\pi(x)$, which the opening binds to his own instance commitment (§4.2).

If every check passes he accepts. The accepted checks imply the following chain, read backwards from the last check to the first; Figure 17 draws it.

The chain, read backwards.

1. *The identity check passed at his random point.* Because the commitment binds, the polynomials behind the openings were fixed before the point was chosen: the Pedersen vector commitment is binding under the discrete logarithm on Vesta (the *Crypto Guide*, §“Pedersen vector commitments”, recalled in §2.9), and the opening argument is knowledge sound (Theorem 3.5 and Corollary 3.13), so from any prover who passes the opening an extractor recovers the committed vector and the value it opens to, or else a discrete-logarithm relation among the hashed generators; its knowledge error is $O(\log d/|\mathbb{F}|)$, and the multipoint reduction that fed it adds one Schwartz–Zippel allowance of order $n/|\mathbb{F}|$ (Lemma 4.5). With a hash in place of coins the ordering holds by construction, every commitment being an input of the hash that produced the challenge testing it (§4.3). Because the point was random and a false identity disagrees with the true one at all but $\deg D$ points, passing supports $G(X) = t(X) Z_H(X)$ as polynomials except with the error $\deg D/|\mathbb{F}|$ of Theorem 2.5 for this fixed check: 11/97 for the toy's degree budget, 3/97 for the particular cheat of Example 2.6, about 2^{-240} at Orchard scale, one term in the sense §1.2 fixed. In the deployed form the identity is $C = b Z_H$, and the challenge y that combined every constraint into C costs one more term, $(T - 1)/|\mathbb{F}|$ for T constraint polynomials (Lemma 4.6).

2. *The identity holds as polynomials.* Then $Z_H \mid G$, so G vanishes on every row (Theorem 2.2, equivalence (3)), so every gate equation holds at every row (Theorem 2.1). Both steps are exact equivalences with no error term: in the toy, the four rows of Table 1 each satisfy (1). In the deployed form every constraint polynomial combined into C vanishes on the active rows: every custom gate, and the permutation and lookup identities that the next two links consume.
3. *The permutation identities passed.* Except with the error $2N/|F|$ of Theorem 2.11 over β, γ , with N the number of participating cells, twelve in the toy, and, in the deployed form, $N/|F|$ more for the challenges at which a product factor vanishes (Corollary 2.12), the copy constraints hold: the per-row gates are wired into one coherent computation, the witness 3 of row 0 is the witness of rows 1 and 2, and each row's output is the next row's input. The lookup identities passed likewise: except with the error $4n/|F|$ of Theorem 2.15, and $n(m-1)/|F|$ more for the θ -compression of (10) (Proposition 2.16) when a lookup has $m > 1$ columns, every looked-up value is in its table.
4. *The public input fixed the target at 35.* In the toy, 35 enters the gate polynomial G through $\text{PI}(X)$, which the verifier interpolates himself (§4.2), so it enters the identity of link 1. In the deployed circuit the instance polynomial $\pi(X)$ of Definition 4.2 is the verifier's own: he committed to it from the values he intends to verify against, and its designated cells are copy-constrained to the advice cells that carry the same values, so the permutation of link 3 forces agreement. There is no error term of its own. The relation certified is the one at the public input the verifier supplied (§4.2).

Together with the observation that closed §2.1, that filling the advice columns *is* knowing the witness, because those cells are the wires carrying it and its powers, the four links say: there exists an assignment of the advice columns, a value of the witness, making the entire arithmetisation of $X^3 + X + 5 = 35$ hold, at the public 35.

From existence to knowledge. The chain so far establishes that satisfying advice *exists*. That the prover *knows* it is the extraction question. The extractor of Theorem 3.5 and Corollary 3.13 recovers, from any prover who passes the opening, the committed vector and its blinder: it establishes knowledge of committed polynomial data, and nothing more. It does not by itself prove that those polynomials encode a satisfying circuit witness; that is the upper layer's task in the layered extraction of Remark 4.11, where the commitment scheme's extractor turns commitments into polynomials and the polynomial IOP's extractor turns polynomials into a witness. Under the composed hypotheses named there, a round-by-round or state-restoration knowledge-soundness theorem for the IOP, extractability and evaluation binding for the commitment scheme, and a multi-round Fiat–Shamir theorem for the transcript, an extractor for the *full* protocol obtains such a witness; §5 weighs those hypotheses against the deployed transcript, and in particular states in which model the tight bound is proved.

What he has not learnt. For the interactive protocol and an honest verifier the transcript reveals nothing beyond the truth of the statement (Proposition 4.9, cited, and Corollary 4.10); for the non-interactive deployed proof this is stated, not proved (§4.6). Every commitment the prover

sent is a uniform group element (perfect hiding, recalled in §4.6); the verifying key's and the instance commitments carry the fixed blinding factor 1 and commit only public data; the evaluations of each blinded column are uniform by Lemma 4.8; the final scalar of the opening, which folding alone would have leaked as a linear form of the coefficients (Remark 3.7), is masked by the random polynomial $s(X)$ vanishing at the opening point (§3.6, in the deployed form of §3.8); and the joint transcript is simulated by the witness-free simulator of Proposition 4.9, within the distance of Corollary 4.10 for the deployed challenges. The toy has no blinding rows and is not zero knowledge: its claimed value $a(20) = 59$ alone singles out the witness 3 among the three roots 3, 18, 76 of $X^3 + X + 5 = 35$ in $\mathbb{F}_{97}$, and indeed among all 97 candidates; the blinding rows of Lemma 4.8 are what make the deployed evaluations uniform. The deployed mask makes the opening's own scalar a fresh uniform value: $c = 20$ under one mask (Table 7) and $c' = 46$ under another (§3.8), both accepted.

The error budget. The verifier has checked a small transcript, $2720 + 2272a$ bytes for a Actions (Table 8), and a logarithmic opening of $2k + 1 = 23$ points. Each link of the chain carries its own error term, which bounds the failure of that link alone: the identity's $\deg D/|\mathbb{F}|$ (Theorem 2.5); the y -combination's $(T - 1)/|\mathbb{F}|$ (Lemma 4.6); the permutation's $2N/|\mathbb{F}|$ (Theorem 2.11) and $N/|\mathbb{F}|$ for the challenges at which a product factor vanishes (Corollary 2.12); the lookup's $4n/|\mathbb{F}|$ (Theorem 2.15) and the θ -compression's $n(m - 1)/|\mathbb{F}|$ (Proposition 2.16); the multipoint reduction's allowance of order $n/|\mathbb{F}|$ (Lemma 4.5); and the opening's $O(\log d/|\mathbb{F}|)$ (Theorem 3.5 and Corollary 3.13). Combining them into one knowledge error for the protocol is the compilation principle of Remark 4.11, whose hypotheses §5 names and does not discharge; no sum of the terms is asserted. Beyond these terms lie the loss of replacing coins by a hash, of which the grinding route alone multiplies a term by the query budget Q , so that 2^{-240} becomes 2^{-176} after 2^{64} attempts (§4.3, Proposition 4.3), while the multi-round transform's full loss is weighed in §5, which asserts no factor- Q bound for the compiled protocol; and the assumptions beneath the chain, the discrete logarithm on Vesta that makes the commitments bind and the extractor's alternative output infeasible (Theorem 3.5 and Corollary 3.13), the random-oracle model for the transcript hash, and the extraction model in which the non-interactive bound is proved (§5). The isolated 2^{-240} is not the verdict's error: it is one Schwartz–Zippel term for one fixed lie, read as §1.2 fixed.

6.2 The Action circuit at deployed scale

Nothing in the chain used that the statement was small. Replace the four gates by the constraints of the Orchard Action statement, the nine conditions the protocol specification imposes on the public input and the witness (protocol specification, § 4.18.4, “Action Statement (Orchard)”: the integrity of the old and new note commitments, the Merkle path from the spent commitment to the anchor unless the spent value is 0 (the *Crypto Guide*, §“Merkle trees and commitments to sets”), the opening of the net value commitment, a Pedersen commitment to the value difference (the *Crypto Guide*, §“The Pedersen commitment”), the nullifier's derivation, the spend authority, that the randomised verification key rk is the re-randomisation of the spend validating key ak by a witnessed randomiser (the *Crypto Guide*, §“Key re-randomisation and unlinkability”), the integrity of the diversified address, that the spent note's address (g_d, pk_d) satisfies $pk_d = [ivk] g_d$ for

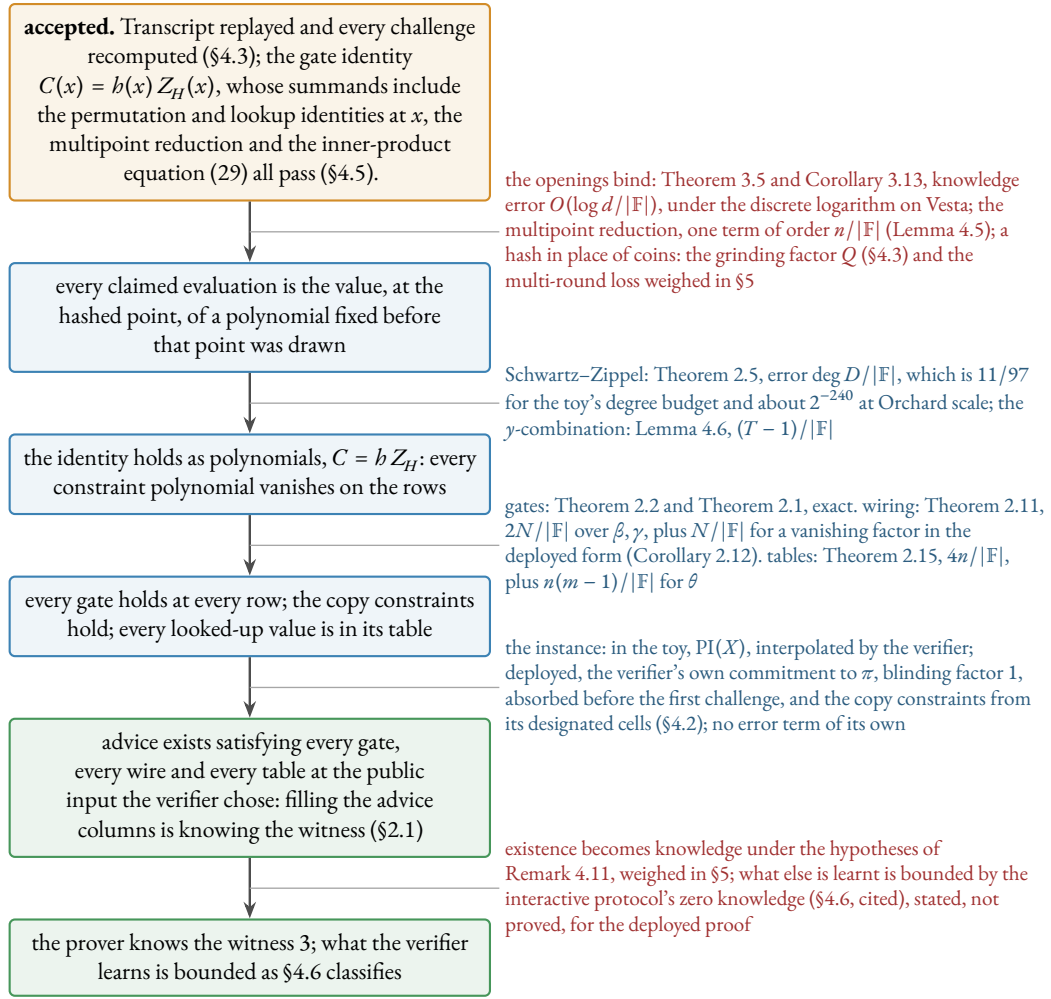


Figure 17. The implication chain from the verifier's accepted checks (amber) back to the claim (green), read downwards; each arrow carries the statement that proves it and its error term. Blue labels are algebraic links whose only losses are Schwartz–Zippel terms or none; red labels are the two links that rest on an assumption or a model hypothesis, the binding of the openings at the top and the extraction of the witness at the bottom.

the incoming viewing key ivk derived in the witness from ak and further key material (the *Crypto Guide*, §“Shielded-note delivery as deployed hybrid encryption”) unless that derivation fails, and the two enable flags, which force the spent value to 0 unless $\text{enableSpends} = 1$ and the created value to 0 unless $\text{enableOutputs} = 1$. From NU6.3 the circuit checks a tenth condition on the further public bit $\text{disableCrossAddress}$ (§ 4.6; ZIP 258): when it is set, the receivers (g_d, pk_d) of the spent and created notes coincide (§4.2). Each condition unfolds into the gates of the deployed circuit (§4.1), and the grid grows accordingly (Table 11): the four rows become $2^{11} = 2048$; the one universal gate of degree 3 becomes custom gates reading rotations, under the degree bound $d_{\max} = 9$, the incomplete addition of Theorem 4.1 among them; the quotient t of degree 5 becomes b of degree up to 16375, cut into 8 chunks (Theorem 4.7); the four-class permutation on twelve cells becomes fifteen equality-enabled columns in three running products; a Sinsemilla generator table of 2^{10} rows, reached by three lookup arguments, joins the permutation (§4.1.3; the hash is the

Crypto Guide's, §“Sinsemilla: an algebraic hash-based commitment”); the four openings at z become 49 claimed evaluations per Action and 45 shared (Table 10), collapsed into one opening of 23 points; and the point $z = 20$ drawn by decree becomes an x squeezed from the BLAKE2b transcript. The spine is identical: arithmetise into a grid, interpolate the columns, compress “all rows correct” into one identity, $G = t Z_H$ in the toy and $C = b Z_H$ in the deployed protocol, commit, and test at a random point.

Table 11. The toy and the deployed Action circuit, object by object. Every row is the same object at two scales; every number is one already printed in the section cited, where it is computed by script or, for the 2^{10} rows of the Sinsemilla table, recorded from the definition of Sinsemilla.

	the toy	the Orchard Action
field	$\mathbb{F}_{97}$	$\mathbb{F}_{p_{\text{Pallas}}}$, a prime of 255 bits
commitment group	$y^2 = x^3 + 3$ over $\mathbb{F}_{97}$, 97 points (§3.7)	Vesta, of order p_{Pallas} (§3.8)
rows	$n = 4$, $H = \{1, 22, 96, 75\}$	$n = 2^{11} = 2048$
columns	3 advice, 5 selectors, 1 instance	10 advice, 29 fixed, 1 instance (§4.1)
gates	one universal gate (1), degree 3, rotation 0	custom gates at rotations, $d_{\text{max}} = 9$ (Definition 2.7; the deployed value, §4.1.4)
identity	$G = t Z_H$; $\deg G = 9$, $\deg t = 5$	$C = b Z_H$ with C the y -combination; $\deg b \leq 16375$, 8 chunks (Theorem 4.7)
wiring	a permutation of twelve cells, four copy classes (b_3, c_3 fixed), one running product (§2.6)	fifteen equality-enabled columns, three running products
table facts	none on the grid; Example 2.13	three lookup arguments into a 2^{10} -row Sinsemilla table (§4.1.3)
the point	$z = 20$, by decree	x squeezed from the BLAKE2b transcript (§4.3)
one-term error	11/97; 3/97 for the $c_0 = 10$ cheat	about 2^{-240} ; 2^{-176} after 2^{64} grinding attempts
openings	a, b, c, t at z ; four cross-term points and S per opening (§3.7)	49 claimed evaluations per Action and 45 shared, one opening of 23 points (Tables 10 and 8; the collapse, Construction 4.4)
proof	—	2720 + 2272 a bytes, 4992 for one Action (Table 8)

The algebraic core of Action verification. For the verification of an Action proof that the consensus rules require (protocol specification, § 4.6, “Action Descriptions”), the randomised polynomial-identity test of link 1 is the algebraic core: a Schwartz–Zippel argument on one identity at one hashed point. The full verifier of §4.5 evaluates that identity’s permutation and lookup summands from the claimed evaluations, binds every claimed evaluation through the multipoint reduction, recomputes every transcript challenge, and closes the single inner-product equation with its one multi-scalar multiplication of length n , before concluding that the circuit’s constraints hold at the public input it verified against, the anchor, the net value commitment, the nullifier, the randomised verification key, the extracted note commitment and the flags of §4.2. Under the knowledge soundness whose status §5 classifies (designed-but-unspecified for the deployed transcript), an accepted Action proof certifies, for a known witness, that the constraints of the circuit whose verifying key the consensus rules fix (protocol specification, § 4.6) hold. That these imply the conditions of the Action statement (protocol specification, § 4.18.4) is a property of that circuit, which this volume does not verify and which the specification records as having failed for the Orchard circuit deployed before NU6.2 (the same section); value balance additionally requires the

binding signature (protocol specification, § 4.14, “Balance and Binding Signature (Orchard)”), and privacy rests also on components outside the proof, such as note encryption.

What the formal treatment supplies. Each of the five reductions of §1.2, “The chain of reductions”, is owned by a formal statement. Table 12 lists, reduction by reduction, what the toy exhibited and which statement carries it; the last two rows point beyond the spine, to the extractor that turns existence into knowledge and to the recursion, which Orchard does not deploy and on which nothing above depends.

Table 12. The map from the worked example to the formal statements that own it, in the order of the reductions of §1.2. The fifth reduction, from the random check to committed openings, is split into its parts.

reduction	the toy exhibits	the statement that owns it
computation $\rightarrow$ grid	Table 1, Figure 2; knowing the witness is filling the advice	Definition 2.7 (§2.1, §2.5)
grid $\rightarrow$ polynomials	the nine interpolants; G of degree 9 vanishing on H	Theorem 2.1 (§2.2)
vanishing $\rightarrow$ identity	$t = G/Z_H$ of degree 5; the gap $D = 24(1 + X + X^2 + X^3)$	Theorem 2.2 (§2.3)
identity $\rightarrow$ one random check	$z = 20$ accepted; 3/97 and 11/97; Figure 3	Theorem 2.5 (§2.4); Lemma 4.6 and Theorem 4.7 (§4.5)
random check $\rightarrow$ committed openings		
binding	$\mathbf{a} = (90, 61, 22, 24)$ sealed in $P = (39, 25)$	the <i>Crypto Guide</i> , §“Pedersen vector commitments”, recalled in §2.9
the opening	Tables 4 and 5; $v' = 60$ rejected	Construction 3.2, Theorem 3.3, Lemma 3.4, Theorem 3.5, Corollary 3.13 (§3)
the wiring	product 1 honest, 61 with $b_1 = 4$	Theorem 2.11 (§2.6)
the tables	$82 = 82$; the forged 7 stranded	Theorem 2.15 (§2.8)
the public 35	$\text{PI} = (0, 0, 0, -35)$	Definition 4.2 (§4.2)
the removed verifier	z by decree against x by hash; $Q\varepsilon$	the <i>Crypto Guide</i> , §“The Fiat–Shamir transform: from interactive to non-interactive”, applied in §4.3
many openings, one hiding	— the leaked linear form; $c = 20$ against $c' = 46$	Construction 4.4 and Lemma 4.5 (§4.4) Remark 3.7 and the mask (§3.6, §3.8); Lemma 4.8 (§4.6)
the pattern	—	Remark 4.11 and “The map onto the phases” (§4.7)
“the prover knows the witness”	—	the extractor in the algebraic group model and the Fiat–Shamir loss (§5, “The algebraic group model”)
beyond the spine	—	the accumulation fold, Proposition 7.5, the accumulation scheme cited as Theorem 7.4, and recursion over the cycle of curves, none deployed by Orchard, which uses one direction of the cycle only (§7.3, “Accumulation and the Halo trick”; §7.4, “The half Orchard uses”)

7 Recursive proof composition and accumulation schemes

This section states recursive proof composition and accumulation for the inner-product argument; Orchard deploys neither. The protocol specification uses Halo 2 for one purpose, to prove and verify Action statements (protocol specification, § 4.1.13, “Zero-Knowledge Proving System”), and requires every Action proof to be valid (§ 4.6, “Action Descriptions”); the design proposal that introduced the pool states that it makes no use of the proof system’s support for recursive proofs (ZIP 224, under “Proving system”). A verifier holding several proofs may still check them together. The complete verification equation E_i of each proof is a multi-scalar multiplication over all d generators that should evaluate to $\mathcal{O}$ (§3.8.1, “Batch verification”); for independent uniform $\mu_1, \dots, \mu_b \in \mathbb{F}$ the verifier of b proofs checks $\sum_i [\mu_i] E_i = \mathcal{O}$ instead. If some $E_j \neq \mathcal{O}$, then E_j generates the group of prime order, so for any fixed values of the other factors exactly one μ_j satisfies the combined check, and a false batch passes with probability $1/|\mathbb{F}|$ (Lemma 3.14). Batch verification verifies the batch and emits nothing; it is not the accumulation developed here. In the classification of §1.2, the Pasta curves and the use Orchard makes of them (§7.4) are *specified*; recursion and accumulation over those curves are *designed-but-unspecified*: their constructions are published and the halo2 book describes them (*Recursion* chapter), and ZIP 224 leaves them to future protocol updates, but no specification defines a recursive statement or an accumulation rule for Zcash. Theorem 7.4 and Proposition 7.5 below are theorems, the first cited and the second proved.

7.1 Recursion and its two classical obstacles

The setting. Consider a computation iterated many times,

$$z_m = f^{(m)}(z_0) := f(f(\dots f(z_0) \dots)),$$

one step f applied m times to an initial state, and suppose a verifier wants to be sure that z_m is the honest result without recomputing the m steps. Valiant’s *incrementally verifiable computation* asks for a proof of z_m that is updated step by step, each update costing about one step’s worth of proving and the finished proof no larger than the proof of a single step. The more general setting, in which computations branch and merge and each step consumes the proofs of its predecessors, is the *proof-carrying data* of Chiesa and Tromer: every message in a distributed computation carries a proof that it was produced correctly from messages that themselves carried such proofs.

The construction. The natural construction is *recursive proof composition*: proving, inside one proof, that another proof verifies. Write $\mathcal{V}$ for the verifier of a non-interactive argument (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge”), and suppose its algorithm is itself arithmetised, in the sense of §2, into a PLONKish circuit of Definition 2.7, the *verifier circuit*: its instance columns hold the statement and its advice columns hold a proof, and its gates are satisfiable exactly when $\mathcal{V}$ accepts that proof of that statement. At step $i + 1$ the prover proves the compound statement

“ $z_{i+1} = f(z_i)$, and there exists a proof π_i that the verifier circuit accepts for the statement of step i .”

The proof π_i is advice of the step- $(i + 1)$ circuit, never sent onward; the proof π_{i+1} that results attests to step $i + 1$ and, through the embedded verifier, to π_i , hence to step i , and so on down to z_0 . One proof attests to the entire history, and the cost of producing π_{i+1} is the cost of one step of f plus one run of the verifier circuit, independent of how long the history is. The requirement on which everything rests is the one just used: the argument’s verifier must be efficiently expressible as a circuit. A verifier whose circuit is as large as the computation it checks would make each step as expensive as the whole history, and the construction would buy nothing.

Two obstacles. The classical instantiations meet that requirement badly, in two different ways.

1. *Pairings in a circuit, and a ceremony.* The pairing-based systems, Groth16, which Sapling deploys, and PLONK compiled with the KZG commitment (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge” for Groth16 and the bilinear pairing; §“Polynomial commitment schemes”, under “KZG: evaluation in the exponent”), have constant-size proofs and verifiers that run in time independent of the circuit. But those verifiers evaluate pairings, and a pairing lands in a target group that lives in an extension field $\mathbb{F}_{p^e}$ of the curve’s base field, of degree e equal to the curve’s embedding degree (*Crypto Guide*, §“Instantiation on elliptic curves; the Pasta curves”, the large-embedding-degree criterion). A circuit whose native arithmetic is one prime field must emulate arithmetic in another field with many gates per operation, so a pairing is costly to express inside a circuit. These particular systems also need a trusted setup, the ceremony that §3.8.1 contrasted with hashed generators.
2. *A linear-time verifier.* The transparent systems built on the inner-product argument of §3 need no pairing and no ceremony, but their verifier is linear in the size of the committed vectors: the one multi-scalar multiplication $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle$ of length d that §3.4 singled out. Inside a verifier circuit that multiplication is d scalar multiplications of curve points, each a double-and-add with $\ell - 1$ doublings for an ℓ -bit scalar (*Math Guide*, §“Scalar multiplication and double-and-add”), so the verifier circuit grows linearly in d , the length of the committed vectors of the proof it verifies. A linear-time check dominates the verifier circuit and negates the per-step economy that recursion was meant to deliver.

Deferred verification. Halo, the construction of Bowe, Grigg and Hopwood (*Halo: Recursive Proof Composition without a Trusted Setup*) from which Halo 2 descends, *defers* the linear-time part of verification instead of recomputing it at every step, and uses the inner-product argument, in which no pairing occurs. Bünz, Chiesa, Mishra and Spooner formalised the deferral as an *accumulation scheme* (*Proof-Carrying Data from Accumulation Schemes*; BCMS20), the notion the next subsection defines; their Theorem 7.1 is quoted below, without proof, as Theorem 7.4.

7.2 Accumulation schemes

The idea is to separate what a verifier must check at each step from what can be checked once, at the end of a chain. Fix a *predicate* Φ , a property of an input q decidable in polynomial time, for example “this inner-product opening is correct” of a transcript of Construction 3.2. An accumulation scheme replaces a sequence of checks of Φ by a sequence of accumulation steps and one run of a decider.

Definition 7.1 (Accumulation scheme). Let λ be the security parameter (*Crypto Guide*, §“Adversaries and the security parameter”) and Φ a predicate on inputs q . An *accumulation scheme* for Φ is a triple (P, V, D) of polynomial-time algorithms with access to a common random oracle, whose *accumulators* are bit strings.

1. The *accumulation prover* P , on input q and an old accumulator acc_{old} , outputs a new accumulator acc and an accumulation proof π_V .
2. The *accumulation verifier* V , on input $(q, \text{acc}_{\text{old}}, \text{acc}, \pi_V)$, outputs a bit; the bit 1 asserts that acc correctly accumulates q and acc_{old} .
3. The *decider* D , on input an accumulator, outputs a bit; an accumulator is *valid* if D outputs 1 on it.

The scheme is *complete* if, whenever $\Phi(q) = 1$, $D(\text{acc}_{\text{old}}) = 1$ and (acc, π_V) is an output of P on $(q, \text{acc}_{\text{old}})$, both $V(q, \text{acc}_{\text{old}}, \text{acc}, \pi_V) = 1$ and $D(\text{acc}) = 1$. It is *sound* if for every adversary of size polynomial in λ that outputs $(q, \text{acc}_{\text{old}}, \text{acc}, \pi_V)$ the probability, over the random oracle and the adversary’s coins, of

$$V(q, \text{acc}_{\text{old}}, \text{acc}, \pi_V) = 1 \wedge D(\text{acc}) = 1 \wedge (\Phi(q) = 0 \vee D(\text{acc}_{\text{old}}) = 0)$$

is negligible in λ .

The definition is that of BCMS20 (Section 4.1) with one input and one old accumulator per step; theirs admits any number of each and adds a generator and an indexer, which fix the public parameters and keys left implicit here. For recursion the size of an accumulator must also be bounded independently of the number of inputs accumulated into it (BCMS20, Definition 5.1).

Soundness carries the checks along a chain.

Lemma 7.2 (Soundness along a chain). *Let (P, V, D) be sound in the sense of Definition 7.1, let acc_0 be valid, and let m be polynomial in λ . For any efficient algorithm producing $q_1, \dots, q_m$ and $\text{acc}_1, \dots, \text{acc}_m$, where acc_i accumulates q_i and acc_{i-1} , such that V accepts every step and $D(\text{acc}_m) = 1$, the probability that $\Phi(q_i) = 0$ for some i is at most m times the soundness error of the adversary that outputs step i for a uniform $i \in \{1, \dots, m\}$; it is therefore negligible.*

Proof. If some q_i violates Φ , then either acc_i is valid and step i exhibits the event of the soundness definition, or some later step $j > i$ turns an invalid acc_{j-1} into a valid acc_j and exhibits it. Let $\mathcal{A}$ be the algorithm that produces the chain and outputs step i for a uniform $i \in \{1, \dots, m\}$; since the bad event forces some step to exhibit the soundness event, its probability is at most m times the soundness error of $\mathcal{A}$, which is negligible because m is polynomial (*Math Guide*, §“Polynomial, exponential, and negligible functions”, the closure properties). $\square$

The verifier of each step runs the accumulation verifier and never the decider. In a recursive chain the accumulation verifier runs inside the circuit of the step’s proof, so the argument also needs the knowledge soundness of each step’s proof, to recover what that circuit checked; BCMS20 prove the resulting proof-carrying data secure for computations of constant depth,

which for a chain means constant length, because the extractor is applied recursively once per level (Theorem 5.2 and Remark 5.3).

For an inner-product polynomial commitment such a scheme is known under the hypothesis of Theorem 7.4 below, and it resolves the second obstacle of §7.1 where it arose: the verifier circuit runs the $O(\log d)$ part of the opening check, the rounds, the scalars and the small combinations of points, and accumulates the $O(d)$ part, the claim about $G^{(0)}$, into a running accumulator; the decider pays that multi-scalar multiplication once, at the end of the chain. The next subsection defines the accumulator, states the scheme as an interface theorem, and works the fold on the toy curve.

7.3 Accumulation and the Halo trick

The asymmetry, recalled. Everything the verifier of Construction 3.2 does to decide the check (15), or its deployed form (29), is logarithmic in d : $3k - 2$ field multiplications for $g(x; \mathbf{u})$, a combination of the $2k$ received points and a handful of fixed ones with $O(k)$ scalar multiplications, and a comparison of two curve points. The single exception is the evaluation

$$G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = \text{Commit}(g(X; u_1, \dots, u_k); 0),$$

a multi-scalar multiplication of length d over *all* the generators, which §3.4 named the one linear step and recognised as the unblinded commitment to the polynomial g whose *value* the verifier had just computed cheaply. Verifying an Action proof as the consensus rules require includes that step for every proof; batch verification, recalled at the start of this section, evaluates one combined multiplication for the proofs of a batch, and the batch is verified and nothing survives it. Accumulation takes the same kind of linear combination and does something else with it: it carries the combined claim forward as data, unverified, and pays for it once at the end of a chain. Figure 18 sets the two side by side; the rest of this subsection is the right-hand panel, and none of it is deployed by Orchard.

The atomic accumulator. The claim to be deferred has a fixed shape, and the accumulator is that shape written down.

Definition 7.3 (Atomic accumulator for the inner-product argument). An *accumulator instance* is a tuple

$$\text{Acc} := (G^{(0)}, u_1, \dots, u_k) \in \mathbb{G} \times (\mathbb{F}^\times)^k,$$

a curve point and the k round challenges of one opening. It is *valid* if and only if

$$G^{(0)} = \langle \mathbf{s}(\mathbf{u}), \mathbf{G} \rangle = \text{Commit}(g(X; u_1, \dots, u_k); 0), \quad (35)$$

with $\mathbf{s}(\mathbf{u})$ and g the structured scalars and the polynomial of Theorem 3.3, in the symmetric normalisation of Construction 3.2. The deferred claim concerns the *unblinded* commitment: the blinder of the opened commitment was discharged by the $[r^*]H$ term of (15), or the $[f]H$ term of (29), and plays no part in it. From here on the unary $\text{Commit}(\cdot)$ abbreviates $\text{Commit}(\cdot; 0)$.

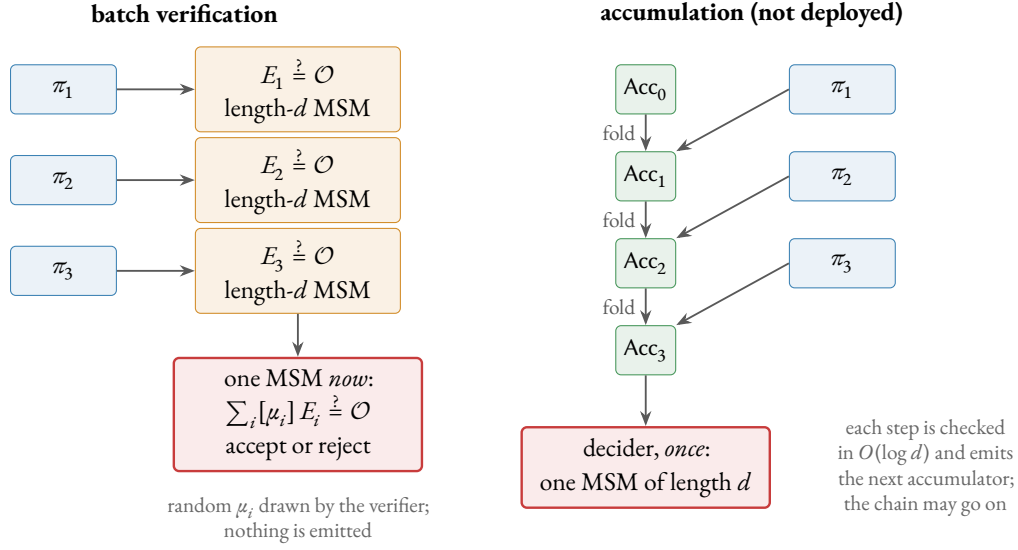


Figure 18. Left: batch verification. Each proof's complete verification equation E_i is a multi-scalar multiplication over the d generators (amber); the verifier combines them with fresh random factors and evaluates one multiplication immediately (red), then accepts or rejects. Right: accumulation. Each proof's deferred claim is accumulated into a running accumulator (green), each step being checked at logarithmic cost; the accumulator is emitted for the next step, and a decider evaluates one length- d multiplication at the end of the chain. Both take random linear combinations; only the right-hand scheme carries a claim forward.

The deployed analogue replaces $\mathbf{s}(\mathbf{u})$ and g by their forms in Remark 3.9; Proposition 7.5 and, with verifier-drawn challenges, the closure argument below hold for it verbatim, with Corollary 3.13 in place of Theorem 3.5, since they use only the linearity of the commitment, the degree of g and the knowledge soundness of the opening.

A verifier who is handed a point $G^{(0)}$ together with the challenges can complete the check (15) in $O(\log d)$ operations *provided* the tuple is valid; validity is precisely what he has not paid for. The scheme quoted next lets him defer that payment.

Theorem 7.4 (Accumulation for the inner-product argument; interface). *Let PC_{DL} be the inner-product polynomial commitment of BCMS20 (Appendix A.2), for polynomials of degree below d over a group $\mathbb{G}$ of prime order, and suppose that PC_{DL} is a polynomial commitment scheme in the random oracle model, its openings in particular being extractable. Then (BCMS20, Theorem 7.1) there is an accumulation scheme in the sense of Definition 7.1, in the random oracle model, for the predicate “this PC_{DL} opening is correct”, whose accumulators are themselves PC_{DL} openings, such that:*

1. *the accumulation prover P runs in time dominated by $O(d)$ scalar multiplications in $\mathbb{G}$;*
2. *the accumulation verifier V runs in time dominated by $O(\log d)$ scalar multiplications in $\mathbb{G}$;*
3. *the decider D is the opening check of PC_{DL} : a succinct part, which yields a tuple of the form of Definition 7.3 in the normalisation of Remark 3.9, followed by the validity check (35) of that tuple, one multi-scalar multiplication of length d .*

The theorem is quoted as an interface and not proved in this volume. Its hypothesis, the extractability of PC_{DL} openings, is stated by BCMS20 as a conjecture from the binding of the Pedersen commitment, the extractor they record for the Fiat–Shamir-compiled opening running in superpolynomial time (BCMS20, Appendix A.3.2). The scheme PC_{DL} is neither Construction 3.2, whose fold is symmetric, nor the deployed opening of Remark 3.9, which blinds every commitment and subtracts $[v] G_0$ where PC_{DL} adds a multiple of its inner-product generator (the halo2 book’s *Comparison to other work* chapter); no theorem transferring it to either is proved here or cited, and in the classification of §1.2 that transfer, like accumulation over these curves, is designed-but-unspecified. What an accumulation step does for Construction 3.2, and why its random combination is sound, is worked out below at the level the volume proves things.

The accumulation step in the verifier circuit. Consider a Halo 2 verifier circuit, expressed over the scalar field of one Pasta curve and verifying a proof produced over the other curve, an arrangement whose necessity §7.4 explains. At step i of a chain the circuit receives the previous proof π_{i-1} , whose opening is an instance of Construction 3.2, and the accumulator Acc_{i-1} exposed as a public input of π_{i-1} , of which it uses the instance on the curve π_{i-1} is committed on (see below), and does three things.

1. *Run the succinct part.* It recomputes the challenges $\mathbf{u}'$ of the opening from the transcript, evaluates $g(x; \mathbf{u}')$ with $O(k)$ scalar operations, and forms the left-hand side of (15) and every term of the right-hand side except the one on the final generator, here written $G'^{(0)}$, with $O(k)$ point operations. The point operations are native, since the points of π_{i-1} have coordinates in the circuit’s field. The challenges, $g(x; \mathbf{u}')$ and the scalar products lie in the scalar field of the curve π_{i-1} is committed on, the other Pasta field; they are computed non-natively, or exposed as public inputs and checked by the next circuit of the chain, which is native to that field (“What the cycle does and does not buy” in §7.4; the halo2 book’s *Recursion* chapter).
2. *Witness the expensive part.* It does not compute $G'^{(0)}$. The prover of the step supplies the claimed point as advice, and the circuit uses it to complete the check, which is then passed *conditionally on the validity of the fresh tuple* $(G'^{(0)}, \mathbf{u}')$ of that opening.
3. *Accumulate.* The step now holds two claims of the shape (35): the inherited instance $(G^{(0)}, \mathbf{u})$ of Acc_{i-1} and the fresh $(G'^{(0)}, \mathbf{u}')$. For a challenge ρ derived by the random oracle from both tuples, the two reduce to the single claim

$$G^{(0)} + [\rho] G'^{(0)} = \text{Commit}(g(X; \mathbf{u}) + \rho g(X; \mathbf{u}')) = (\mathbf{s}(\mathbf{u}) + \rho \mathbf{s}(\mathbf{u}'), \mathbf{G}), \quad (36)$$

whose coefficient vector on the right anyone holding both challenge tuples can recompute. Proposition 7.5 shows that a false input survives the combination for at most one ρ . The step’s prover then opens the combined point at a fresh point, outside the circuit, and the circuit checks that opening succinctly as in item 1; the tuple the opening leaves behind is the new accumulator Acc_i (over a 2-cycle, its instance on that curve), exposed as a public input of the proof π_i the step produces. The paragraph “Closure: the prover-assisted step” below describes that opening.

Over a 2-cycle of curves (§7.4) the proofs alternate between the two curves, and the accumulator is a pair of instances, one per curve. The step that verifies a proof committed on one curve folds that

proof's fresh tuple into the instance on the same curve, which was last updated two steps earlier and forwarded unchanged by the intervening step, and it forwards the other curve's instance in the same way, without point arithmetic. Figure 19 draws the chain. At its end a single multi-scalar multiplication of length d , run outside any circuit, checks the last accumulator and with it every opening in the history: for the scheme of Theorem 7.4 by Lemma 7.2.

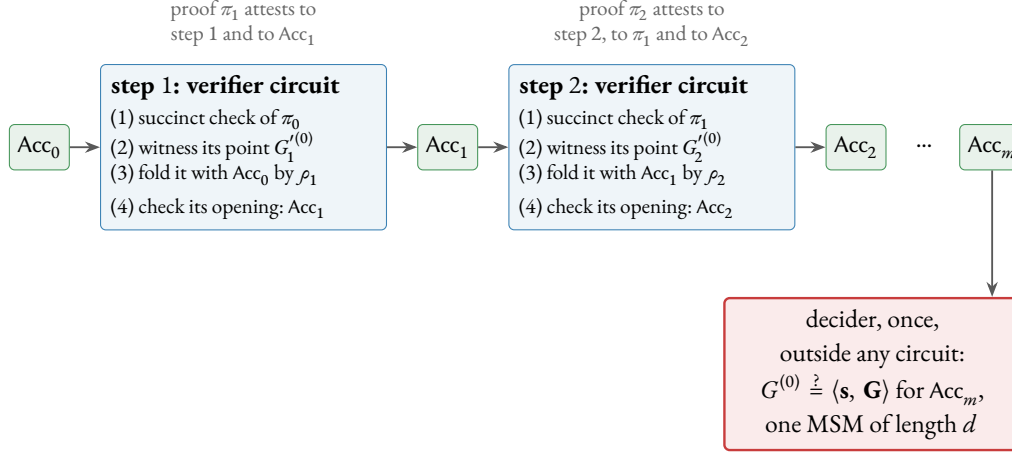


Figure 19. A chain of recursively composed proofs. Each step's verifier circuit (blue) runs the logarithmic part of the previous proof's opening check, witnesses the expensive point instead of computing it, combines the resulting tuple with the accumulator (green) it inherited by (36), and checks succinctly the opening of the combined point at a fresh point, whose tuple is the new accumulator, a public input of the step's own proof. Over a cycle of curves each Acc_i is a pair of instances, one per curve: each step folds into the instance on the curve of the proof it verifies and passes the other through unchanged. The decider (red) pays one length- d multi-scalar multiplication at the end of the chain, and no circuit ever contains one.

Why the fold is sound. The soundness of the fold (36) is one line of linear algebra in the group.

Proposition 7.5 (Soundness of the ρ -fold). *Let $(G^{(0)}, \mathbf{u})$ and $(G'^{(0)}, \mathbf{u}')$ be two accumulator instances and ρ uniform in $\mathbb{F}$, drawn after both are fixed. If both instances are valid, the folded equation (36) holds for every ρ . If at least one is invalid, it holds for at most one value of ρ , hence with probability at most $1/|\mathbb{F}|$.*

Proof. Set $\Delta := G^{(0)} - \text{Commit}(g(X; \mathbf{u}))$ and $\Delta' := G'^{(0)} - \text{Commit}(g(X; \mathbf{u}'))$, the two errors. By linearity of the commitment (*Crypto Guide*, §“Pedersen vector commitments”, the remark that linearity survives), $\text{Commit}(g + \rho g') = \text{Commit}(g) + [\rho] \text{Commit}(g')$, so the left-hand side of (36) minus the right is $\Delta + [\rho] \Delta'$, and the equation holds exactly when

$$\Delta + [\rho] \Delta' = \mathcal{O}. \quad (37)$$

If both instances are valid, $\Delta = \Delta' = \mathcal{O}$ and the equation holds for all ρ . Otherwise two cases. If $\Delta' \neq \mathcal{O}$, then, $\mathbb{G}$ being cyclic of prime order, Δ' generates $\mathbb{G}$ and the map $\rho \mapsto [\rho] \Delta'$ is a bijection $\mathbb{F} \rightarrow \mathbb{G}$ (*Math Guide*, §“The mixed inner product with group elements”, the one-dimensional vector space $\mathbb{G}$ over $\mathbb{F}$), so exactly one ρ sends Δ' to $-\Delta$: the discrete logarithm of $-\Delta$ to the base

Δ' , which exists whether or not anyone can compute it. If $\Delta' = \mathcal{O}$, then $\Delta \neq \mathcal{O}$ and (37) holds for no ρ . A uniform ρ hits the at most one bad value with probability at most $1/|\mathbb{F}|$. $\square$

Example 7.6 (The fold on the toy curve). The fold is run on the 97-point curve and the four generators $G_0, \dots, G_3$ of §3.7, with $d = 4$ and the symmetric structured scalars of Theorem 3.3, on two challenge tuples, $(u_1, u_2) = (52, 44)$ with $\mathbf{s} = (80, 10, 68, 57)$ and $G^{(0)} = \langle \mathbf{s}, \mathbf{G} \rangle = (21, 10)$, and $(u'_1, u'_2) = (41, 39)$ with $\mathbf{s}' = (64, 11, 53, 47)$ and $G'^{(0)} = (23, 2)$, both valid by construction, for every $\rho \in \mathbb{F}_{97}$. With both instances valid the folded equation (36) holds for all 97 values of ρ . Spoiling an instance by adding a nonzero multiple of G_0 to its point makes its error Δ a nonzero point, and the surviving values of ρ are counted again. With the first instance false and the second valid, no ρ survives: this is the case $\Delta' = \mathcal{O} \neq \Delta$ of the proof. With the second false and the first valid, exactly one survives, $\rho = 0$, the value that discards the second instance altogether. With both false exactly one survives, $\rho = 50$, the discrete logarithm of $-\Delta$ to the base Δ' on this curve, where such logarithms are a short search. In every spoiled case the count is at most one of 97: the bound $1/|\mathbb{F}|$ of Proposition 7.5, attained.

Closure: the prover-assisted step. The folded claim (36) is not an accumulator instance. Its coefficient vector $\mathbf{s}(\mathbf{u}) + \rho \mathbf{s}(\mathbf{u}')$ is in general not the structured vector of any single challenge tuple, so carrying it means carrying both tuples, and after m folds all m of them: the decider would still pay one multiplication of length d , but the accumulator would grow with the chain, against the size bound that recursion requires (§7.2). The tuple type of Definition 7.3 is therefore *not* closed under verifier-side linear combination alone. It is closed under one more step, which the prover assists. Write $A := G^{(0)} + [\rho] G'^{(0)}$ for the combined point and $a(X) := g(X; \mathbf{u}) + \rho g(X; \mathbf{u}')$ for the combined polynomial, of degree below d . The combined claim says exactly that $A = \text{Commit}(a)$. The step's prover opens A by the opening protocol of Construction 3.2 at a fresh point x' , derived by the random oracle from A and both tuples, claiming the value $a(x')$, which the verifier computes for himself in $O(k)$ field operations from the two tuples, since $g(x'; \mathbf{u})$ and $g(x'; \mathbf{u}')$ are each a product of k factors. That opening does two things at once. First, it carries the combined claim up to a blinder.

Proposition 7.7 (Prover-assisted step, verifier-drawn challenges). *Under the hypotheses of Theorem 3.5, let ρ, x' and the opening's challenges be drawn uniformly by the verifier, in this order, ρ after the two tuples are fixed. There is an extractor which, given rewindable access to a step prover accepted with non-negligible probability, runs in expected polynomial time and outputs either blinders δ, δ' with $G^{(0)} = \text{Commit}(g(X; \mathbf{u}); \delta)$ and $G'^{(0)} = \text{Commit}(g(X; \mathbf{u}'); \delta')$, or a nontrivial discrete-logarithm relation among $\mathbf{G}, H, U, U'$. Its knowledge error is $O(\log d / |\mathbb{F}|) + (d - 1)/|\mathbb{F}| + 1/|\mathbb{F}|$.*

Proof. By Theorem 3.5 the opening yields a representation $A = \langle \mathbf{a}^*, \mathbf{G} \rangle + [r_A]H$, unique since a second one would be a discrete-logarithm relation among $\mathbf{G}$ and H , and certifies $a^*(x') = a(x')$ for the polynomial a^* with coefficient vector $\mathbf{a}^*$; two polynomials of degree below d that differ agree at a uniform x' , drawn after A was fixed, with probability at most $(d - 1)/|\mathbb{F}|$ (*Math Guide*, §“The Schwartz–Zippel lemma”), so except with that probability $\mathbf{a}^*$ is the coefficient vector of a and $A = \text{Commit}(a; r_A)$. The blinder r_A is the prover's and need not be 0. Rewinding the

choice of ρ and extracting again at a second value $\tilde{\rho} \neq \rho$ gives representations of $\mathcal{A}$ and of $\tilde{\mathcal{A}} := G^{(0)} + [\tilde{\rho}] G'^{(0)}$, whose difference is $[\rho - \tilde{\rho}] G'^{(0)}$; solving yields $G'^{(0)} = \text{Commit}(g(X; \mathbf{u}); \delta')$ and $G^{(0)} = \text{Commit}(g(X; \mathbf{u}); \delta)$ for blinders δ, δ' that the extractor computes. This is the argument of Proposition 7.5 applied to the $\mathbf{G}$ -coordinates of the representations: a combination correct at two values of ρ has both errors zero. It is all the chain needs: a tuple whose point is $\text{Commit}(g(X; \mathbf{u}); \delta)$ still certifies the opening it came from, because, writing c for the final scalar that (15) calls a ,

$$[c] \text{Commit}(g; \delta) + [r^*]H = [c] \text{Commit}(g) + [r^* + c\delta]H,$$

so that opening satisfies the honest check with the blinder $r^* + c\delta$ in place of r^* , and Theorem 3.5 applies to it; in (29) the offset joins the term $[f]H$ likewise. The chain thus carries the claim that each deferred point's $\mathbf{G}$ -coordinates are $\mathbf{s}(\mathbf{u})$; the exact equality (35), blinder 0 included, is required only of the last accumulator, which the decider checks and an honest prover satisfies. An intermediate tuple may miss (35) by such a multiple $[\delta]H$ without any opening it serves being false. $\square$

Second, the opening leaves behind a deferred claim of the atomic form: it ends, as every opening does, in a point $G''^{(0)}$ and challenges $\mathbf{u}''$ that the verifier circuit witnesses rather than computes. The accumulator after the step is again one tuple. This is the accumulation step of BCMS20 (Section 7.1), which likewise derives the combining challenge and the fresh point by the random oracle from the tuples and the combined point; there the accumulator is the opening itself, and its succinct part runs at the next step. With ρ, x' and the opening's challenges drawn by a verifier, the step's soundness is Proposition 7.7. With random-oracle challenges, the step's soundness is BCMS20's Theorem 7.1 for PC_{DL} under its conjectured extractability (Theorem 7.4); for Construction 3.2 it is designed-but-unspecified.

What the accumulator carries. The deferred equation (35) is a claim of *commitment equality*, equivalently a claim about the value of one multi-scalar multiplication, for a coefficient vector $\mathbf{s}(\mathbf{u})$ that is public and structured: anyone holding the challenges can write it down. It is not itself an evaluation opening; no polynomial is being evaluated at any point when the decider runs, only a point recomputed and compared. The accumulation protocol reduces this claim, and carries it along the chain, through the prover-assisted opening just described, which turns each commitment equality into one evaluation claim at a fresh point and each opening back into one commitment equality. The Orchard verifier, once more, does none of this: it computes $G^{(0)}$ and compares.

7.4 A cycle of curves for efficient native recursion

The verifier circuit of §7.3 was “expressed over the scalar field of one Pasta curve and verifying a proof produced over the other”. This subsection explains why the two curves are needed, what the arrangement buys, and where Orchard already uses half of it.

Two fields per curve, recalled. A curve E over a prime field has two fields attached to it (*Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”): the *base field*, in which its points' coordinates lie and in which the group law's slopes, squarings and inversions are computed, and the *scalar field*, the integers modulo the prime order of its group, in which the scalars of $[k]P$ live.

Write E_p for a curve over $\mathbb{F}_p$ and $q := \#E_p(\mathbb{F}_p)$ for its prime order, so that its base field is $\mathbb{F}_p$ and its scalar field $\mathbb{F}_q$. A PLONKish circuit of Definition 2.7 is defined over one prime field $\mathbb{F}$, its *native field*: its cells hold elements of $\mathbb{F}$ and its gates are polynomial identities over $\mathbb{F}$, so an addition or multiplication in $\mathbb{F}$ costs one cell or one gate. Arithmetic in any *other* prime field is *non-native*: an element of the other field must be carried as several $\mathbb{F}$ -cells, each range-checked by the lookups of §2.7, and each of its operations becomes many gates with carries and reductions. Which field a circuit is native to is not a free choice. Its column polynomials are committed with the inner-product commitment of §3 on some curve, and a Pedersen commitment to a vector of $\mathbb{F}$ -elements needs a group whose scalar field is $\mathbb{F}$; so *the native field of a circuit is the scalar field of the curve its proof is committed on*, exactly as the Action circuit, native to $\mathbb{F}_{p_{\text{Pallas}}}$, is committed on Vesta, whose order is p_{Pallas} (§3.8).

Why one curve cannot host its own verifier. Take a proof committed on E_p . Its group elements, the column and quotient commitments, the cross terms (L_j, R_j) and the deferred point of its opening, are points of E_p with coordinates in $\mathbb{F}_p$. The in-circuit folding of §7.3 adds and scales such points, which is base-field arithmetic in $\mathbb{F}_p$, so a verifier circuit handles them natively only if its native field is $\mathbb{F}_p$. But a circuit committed on E_p is native to the *scalar* field $\mathbb{F}_q$: its challenges, its structured scalars and every field operation of the succinct part are $\mathbb{F}_q$ -arithmetic. One curve would host both jobs only if $\mathbb{F}_p = \mathbb{F}_q$, that is, $\#E_p(\mathbb{F}_p) = p$. Curves with exactly p points exist; by Hasse’s theorem (*Math Guide*, §“The group structure of $E(\mathbb{F}_p)$ ”), which writes $\#E_p(\mathbb{F}_p) = p + 1 - t$ with the trace t , they are the curves with $t = 1$, called *anomalous*. On an anomalous curve the discrete logarithm is computable in polynomial time, the attack of Smart, of Semaev and of Satoh and Araki, and the *Crypto Guide* lists “not anomalous” among the criteria a curve must meet to be used at all (§“Instantiation on elliptic curves; the Pasta curves”, the anti-Smart criterion). So no *secure* curve has $\mathbb{F}_p = \mathbb{F}_q$, and no one curve can host both the coordinates of its proofs and its own verifier circuit natively.

The 2-cycle. The efficient resolution uses two curves that trade fields. A *2-cycle* of elliptic curves is a pair (E_p, E_q) with

$$\#E_p(\mathbb{F}_p) = q \quad \text{and} \quad \#E_q(\mathbb{F}_q) = p,$$

both primes: the scalar field of each curve is the base field of the other. Then:

1. a circuit committed on E_q has native field $\mathbb{F}_p$, the scalar field of E_q ; and $\mathbb{F}_p$ is the coordinate field of E_p , so that circuit manipulates the points of a proof produced over E_p natively, and verifies such a proof at the cost §7.3 counted, the proof’s scalars aside;
2. the proof that circuit produces is committed on E_q , and its points have coordinates in $\mathbb{F}_q$, the native field of a circuit committed on E_p ; so a circuit over $\mathbb{F}_q$ handles its points natively in turn;
3. the proofs of a chain alternate between the two curves, each step handling its predecessor’s points natively (Figure 20).

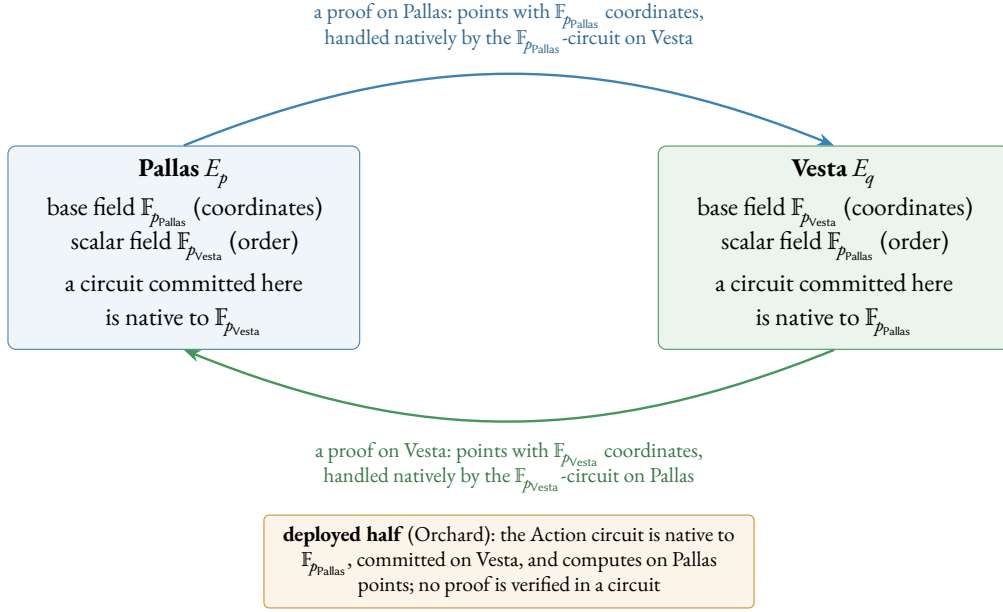


Figure 20. The Pasta 2-cycle. The base field of each curve (its coordinate field) is the scalar field of the other, so a circuit committed on Vesta (green) is native to $\mathbb{F}_{p_{Pallas}}$ and handles Pallas points (blue) natively, and symmetrically. In a recursive chain the proofs alternate along the two arrows. Orchard (amber) uses one direction of the cycle only: Pallas arithmetic inside a Vesta-committed circuit, with no verifier circuit at all.

What the cycle does and does not buy. The cycle makes the *preceding* curve’s coordinate arithmetic native; it does not identify the two scalar fields, which remain distinct primes. Challenges squeezed in the previous proof’s field and the scalars derived from them are $\mathbb{F}_p$ -elements that the $\mathbb{F}_q$ -native circuit must handle: they need compatible encodings across the two fields, range checks where an element of the larger field is carried in the smaller, and, where a genuine $\mathbb{F}_p$ multiplication is unavoidable, non-native scalar arithmetic. Nor is the cycle a logical necessity. A single curve could verify its own proofs with non-native coordinate arithmetic throughout, at a cost of many gates per point operation multiplied by the $O(\log d)$ point operations of the succinct part; that is possible and substantially more expensive. The cycle is the efficient native design, not the only correct one.

The deployed instance. The Pasta cycle of the *Math Guide* (§“Base fields, scalar fields, and the Pasta cycle”; §“Pallas and Vesta assembled”) is a 2-cycle: Pallas is defined over $\mathbb{F}_{p_{Pallas}}$ with group order p_{Vesta} , and Vesta over $\mathbb{F}_{p_{Vesta}}$ with group order p_{Pallas} , both curves $y^2 = x^3 + 5$ (protocol specification, § 5.4.9.6, “Pallas and Vesta”). The cycle follows from the specification’s two primes and nothing else. With

$$p_{Pallas} = 0x40000000\ 00000000\ 00000000\ 00000000\ 224698fc\ 094cf91b\ 992d30ed\ 00000001,$$

$$p_{Vesta} = 0x40000000\ 00000000\ 00000000\ 00000000\ 224698fc\ 0994a8dd\ 8c46eb21\ 00000001,$$

the point $(-1, 2)$ lies on Pallas, $[p_{Vesta}](-1, 2) = \mathcal{O}$ there, and exactly one multiple of p_{Vesta} lies in the Hasse interval $[p + 1 - 2\sqrt{p}, p + 1 + 2\sqrt{p}]$ for $p = p_{Pallas}$. That settles the order: the order

of the point $(-1, 2)$ divides the prime p_{Vesta} and is not 1, so it is p_{Vesta} , which therefore divides $\#\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ by Lagrange’s theorem (*Math Guide*, §“Cosets and Lagrange’s theorem”); the group order lies in an interval of length $4\sqrt{p}$, far shorter than p_{Vesta} , which contains one multiple of p_{Vesta} ; hence $\#\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) = p_{\text{Vesta}}$. The symmetric check on Vesta shows that the Vesta group has order p_{Pallas} . The traces are

$$\begin{aligned} t_{\text{Pallas}} &= p_{\text{Pallas}} + 1 - p_{\text{Vesta}} = -86663725065984043395317759, \\ t_{\text{Vesta}} &= p_{\text{Vesta}} + 1 - p_{\text{Pallas}} = 86663725065984043395317761, \end{aligned}$$

each far inside the Hasse bound and neither equal to 1: neither curve is anomalous, as the *Crypto Guide*’s criterion requires. The two traces sum to 2, as they must for any 2-cycle, since $(p + 1 - q) + (q + 1 - p) = 2$; the toy curve of §3.7, $y^2 = x^3 + 3$ over $\mathbb{F}_{97}$ with 97 points, has trace $79 + 1 - 97 = -17$, and the case excluded in that subsection, a curve over $\mathbb{F}_{97}$ with 97 points and so trace 1, is exactly the anomalous one.

The cycle in the accumulation chain. The Pasta cycle renders each step of §7.3 native. A proof committed on Vesta is produced by a circuit native to $\mathbb{F}_{p_{\text{Pallas}}}$; that circuit verifies the previous proof of the chain, committed on Pallas, whose group elements, the column and quotient commitments, the pairs (L_j, R_j) and the point of the Pallas accumulator instance it forwards, have coordinates in exactly $\mathbb{F}_{p_{\text{Pallas}}}$, so the point arithmetic of the succinct checks and of the fold (36) is native. The next proof, committed on Pallas by a circuit native to $\mathbb{F}_{p_{\text{Vesta}}}$, verifies the Vesta proof by the symmetric argument. This cross-curve alternation avoids non-native emulation of the preceding curve’s coordinate field; as “What the cycle does and does not buy” said, it does not eliminate the scalar-field conversions.

The half Orchard uses. This is the part of the cycle in consensus. The Action circuit is native to $\mathbb{F}_{p_{\text{Pallas}}}$ and committed on Vesta (§3.8), and the statement it proves manipulates Pallas points: the note commitment cm , a Pallas point whose x -coordinate cm_x is a public input; the randomised verification key rk ; and the net value commitment cv^{net} , the last two exposed by their coordinates in the instance column of §4.2. Their coordinates lie in $\mathbb{F}_{p_{\text{Pallas}}}$, the circuit’s native field, by the very coincidence the cycle is built on, so the circuit computes on them natively, with gates such as the incomplete-addition gate of §4.1.2. The protocol specification records the arrangement: Vesta for the proof system, Pallas for the application circuit, both curves designed to be efficiently implementable inside a circuit though only Pallas is so used (protocol specification, § 5.4.9.6, “Pallas and Vesta”); ZIP 224 says the same in the language of this subsection, that the proposal uses half of the cycle, Pallas being the curve embedded in Vesta’s circuits, and that the full cycle is left for future use (ZIP 224, under “Curves”). One direction of Figure 20 is thus in consensus today; the return arrow, and everything in this section that depends on it, is not.